claude-windows / e663947f-2127-4383-9905-650340ff9863

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e663947f-2127-4383-9905-650340ff9863.jsonl`
- SHA-256: `5348cea02bb1f124cf679bbee5b7a664faadf3948f102aadd2a231987bbe38f5`
- Source modified: `2026-07-09T08:10:15+00:00`
- Imported at: `2026-07-09T08:30:30+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `e663947f-2127-4383-9905-650340ff9863`

Transcript

1. user (2026-07-09T06:23:48.042Z)

The cloud-serving worker infer path (`backend/worker/__main__.py`) has three linked observability gaps, surfaced during a live production CUJ on 2026-07-09 (video GX010155, video_id `e2e144175c8d05451903ae44a9ce0aa0`, worker execution `rally-worker-dpssz`, deployed image `sha256:1838c07f?` from `origin/master @ 00e8f92`). Fix all three in ONE PR (branch from `origin/master`).

CONTEXT: `phase_placement.validation/segment/compile` are all `cloud_run_l4`, so the control-plane delegates validation to the L4 worker and dispatches it WITHOUT `--skip-validation` (verified in the execution args). The worker therefore DOES run the full validator suite and re-computes the checksum ? but none of it is observable.

GAP 1 ? validation invisible on a PASS. In `backend/worker/__main__.py` (~lines 281?343) the worker runs `validator_registry.run_all_with_context(...)` unless `--skip-validation`, but only `print()`s on `[worker] validation SKIPPED` or `[worker] validation FAILED`; a PASS logs nothing. And results are persisted only to the ephemeral worker DB via `db.add_video(video_id, ..., [r.model_dump() for r in val_results])` ? they are NOT in the pushed `report.json` (which contained only `video_id, status, segment_count, video_uri, segments`). So a passing validation is invisible in both logs and the runlog. Users legitimately thought validation was skipped.

GAP 2 ? no per-stage wall-clock timing. There is no timing around checksum (`compute_video_id`), validation (`run_all_with_context`), or segmentation (`segmenter.process_video`). In the CUJ, segmentation took ~10 min with nothing recorded.

GAP 3 ? the #534 live progress heartbeat + telemetry does NOT fire on the native-L4 path. Evidence: over a ~10-min GPU pass, `gs://khelsutra-rally-serving/outputs/<video_id>/run_telemetry.jsonl` contained ONLY its meta header (116 bytes: `{"meta":{"label":"l4-infer-?","gpu_name":"NVIDIA L4","cpu_count":8,...}}`) with ZERO snapshots; there were NO `[worker] progress:` lines in stdout; and no `<video_id>_detector_progress.json` was ever written to GCS. The snapshots + heartbeat are driven by the detector progress callback `_progress` (`backend/worker/__main__.py` ~line 388, passed to the segmenter as `progress=_progress` ~line 434), which was never invoked. Root-cause in `backend/pipeline/detectors/native_wasb_runner.py` (~lines 284?440): it is meant to stream `wasb_infer` stdout and parse the periodic `done/total (pct%)` lines (emitted every `log_every_batches`=50) to call the callback ? verify/repair that path so the callback fires ~every `detector_progress_sample_sec` (15s).

FIX (single PR):
1. Clear per-stage log lines in the worker infer flow, emitted even on success: `[worker] checksum: video_id=? (Xs)`; `[worker] validating (N checks)?` then `[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`; `[worker] segmentation (<segmenter>) done in Zs ? R rallies`.
2. Time each stage with `time.perf_counter()` and persist BOTH a `timings` block AND the validation results (per-validator name/passed/message/score) into the pushed `report.json` (built/pushed via `_serialize_and_push_outputs`, ~lines 125?158) so runlogs capture time-per-stage + validator outcomes.

3. Make the native-L4 detector progress callback actually fire so `run_telemetry.jsonl` gets real GPU/velocity snapshots, the `[worker] progress:` heartbeat appears in stdout, and `<video_id>_detector_progress.json` is written to GCS during the run. Ensure `RunTelemetry.snapshot(...)` (`backend/utils/run_telemetry.py`) is invoked from the callback.

VERIFY: add offline unit tests (load the module by file path, mirroring `tests/test_nightly_regression.py`) for the timing/report-serialization additions and the progress-line parsing. Run `python -m ruff check .` (install ruff==0.15.16, CI's pinned version), `mypy backend training`, and `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70`. NB: on Windows there are 2 pre-existing `tests/test_storage.py` GCSFuse failures unrelated to this work (fixed separately in #548) ? they pass on Linux/CI. The live-L4 heartbeat itself needs an owner-driven CUJ to fully confirm end-to-end; state clearly what was and wasn't verified. Open a PR to master.

2. assistant (2026-07-09T06:23:55.245Z)

I'll start by getting oriented ? the memory tells me the primary working directory is a stub and the real code is elsewhere. Let me confirm the actual repo location and read the session handoff.

3. user (2026-07-09T06:23:56.311Z)

<system-reminder>This memory is 2 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>

```
1       ---
2       name: actual-repo-location
3       description: "The badminton-highlight-indexer code lives under khelsutra-guru/, not the
primary working dir stub"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: df5d5fb2-534a-447a-8181-b222f94655f3
8       ---
9
10      The primary working directory `C:\Users\avidu\Projects\badminton-highlight-indexer` is a
near-empty
11      **stub** ? it contains only `AGENTS.md`, an empty `.claude/`, and
`output/nightly_regression/`
12      (a `LATEST_STATUS.txt`). The actual `khelsutra-guru/badminton-highlight-indexer`
codebase lives at:
13
14        `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`
15
16      (git remote `github.com/Khelsutra/badminton-highlight-indexer`, default branch
`master`).
17
18      **How to apply:** when a task references this repo's code (backend/worker,
pipeline/segmenters,
19      config.json, run_all_tests.py, etc.), `cd` to the khelsutra-guru path and work there ?
the stub dir
20      has none of it. Confirmed 2026-07-06 while fixing the wf294 silent-detector-failure
issue.
21
22      Repo conventions verified there: CI = ruff 0.15.16 (`ruff check .`), mypy 2.1.0 (`mypy
backend
23      training`; `mypy.ini` quarantines several modules incl. `backend.worker.__main__`,
`gemini`,
24      `yolo_hybrid` via `ignore_errors=True`), full suite via `python run_all_tests.py`,
coverage floor
25      `--cov-fail-under=70` (actual ~86%). Segmenters use a `Success`/`Failure` Result
contract
26      ([[efficient-timeouts-and-retries]] is unrelated).
27
```

4. user (2026-07-09T06:23:57.327Z)

```
1       ---
2       name: session-handoff-gpu-resident
3       description: "Start here to pick up the GPU-resident WASB rollout (issue #506) ?
implementation PR #507 is open; golden gate next"
4       metadata:
5         type: project
6       ---
7
8       # Session Handoff ? GPU-resident WASB inference (rollout)  ·  issue #506 · PR #507
MERGED
9
10      ## LATEST 2026-07-07 (end of session) ? all serving fixes merged + fresh redeploy
11      Every gap surfaced this session is merged to `origin/master @ ef41473`: **#511**
(compute=local fails fast on the weightless control-plane) + **#516** (SPA compute-picker
defaults to CLOUD on hosted) = the compute-picker footgun; **#512** (validators share ONE decode
pass ? measured ~3.1× on the 5.3K clip: warm 179s?58s on a 16-core box; projects ~6.5min?~2min
on the 2-vCPU control-plane); **#513** (worker skips redundant re-validation on dispatch ? also
fixes the default-blur-threshold reject); **#515** (worker no-done-marker fastfail). **Clean
rebuild from `ef41473` ? redeploy control-plane + worker to the new digest (D3)** ? see the
DEPLOY_REPORT / [[gpu-vm-setup-gotchas]]. #509 forwarding was PROVEN live by the CUJ (worker
args carried `decode_backend=nvdec_resident`+batch64+prefetch4). Two follow-ups SPAWNED:
**task_5acf4b46** (clean 2?3 video validation-speedup closure test + DEPLOY_REPORT) and
**task_4585aa00** (cache validation results by video_id ? skip re-validation on reupload; the
missing cache that made CUJ retries re-pay validation). Validation is NOT cached across
reuploads today (only a whole-pipeline cache gated on a prior SUCCESSFUL segmentation).
12
13      ## SHIPPED 2026-07-07 ? GPU-resident NVDEC decode is LIVE in cloud-serving
14      Full arc done: PR #507 (feature) + PR #508 (cloud-serving preset default =
nvdec_resident) both merged to master (44a2f47, 143dbb9). The `rally-worker` Cloud Run Job
(asia-southeast1) runs the new image `rally-worker:latest` `sha256:465a9250?` (torch 2.6 +
PyNvVideoCodec 2.1.0 + baked NVDEC libs) with `WASB_DECODE_BACKEND=nvdec_resident` set ? **nvdec
is the live serving default**, validated on the real L4 (GX010128 served trajectory reproduced
the VM: 26479?26481 visible; cpu+nvdec+canary smokes all ?). Record:
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md`.
15      - **ROLLBACK levers:** `gcloud run jobs update rally-worker --region asia-southeast1
--remove-env-vars WASB_DECODE_BACKEND` (? torch-2.6 cpu path, same image); or `--image ?/rally-
worker@sha256:3bfefaf6?` (? prior torch-1.11 image).
16      - **PRs:** #507 (feature) + #508 (preset default) + #509 (dispatch forwarding + release
hardening) all merged. master @ 66756ad.
17      - **DONE ? control-plane + worker both on image D2 `sha256:09d2edf7` (built from master
66756ad = #507+#508+#509).** Redeployed 2026-07-07: `gcloud run services update rally-control-
plane --image ?:latest` ? rev **00017-qbd** (100% traffic, booted healthy: uvicorn up, cloud-
serving profile ACTIVE, edge-auth on); `gcloud run jobs update rally-worker --image ?:latest`.
So the #509 dispatch **forwarding** + #508 preset default are now LIVE in the control-plane
image. NB the FIRST control-plane deploy this session used the stale `465a9250` (built from
44a2f47, pre-#508/#509) ? rev 00016; superseded by 00017. Rollback: `gcloud run services update-
traffic rally-control-plane --to-revisions rally-control-plane-00015-kp9=100` (last pre-nvdec
rev) or `--to-revisions ?-00016-?` .
18      - **`WASB_DECODE_BACKEND=nvdec_resident` env is still set on the worker Job** ? now
REDUNDANT with the forwarded --set + preset (both say nvdec, env just wins), kept as belt-and-
suspenders + manual override. Safe to remove ONLY after a real /api/process dispatch confirms
the forwarding path live (needs a CF Access JWT ? not verifiable this session; forwarding is
deployed + unit-tested but not live-dispatch-verified).
19      - `DEPLOY_REPORT_?gpu-resident-nvdec_2026-07-07.md` is current: PR #510 added the §5
"Update" (D2 redeploy of control-plane rev 00017 + worker) and corrected §6. All four PRs
merged: #507 #508 #509 #510; master @ 3be0935.
20      - **Wrinkles fixed (#509):** dispatch now forwards
indexing.wasb.{decode_backend,batch_size,prefetch_batches} (worker has no DEPLOYMENT_PROFILE ?
never applied the preset; batch tuning was silently lost too); added `.gcloudignore` (Cloud
Build had none ? relied on the .gitignore fallback to skip 8 GB output/scratch); added
`deploy/cloudrun/post_deploy_smoke.ps1`. See [[gpu-vm-setup-gotchas]].
21      - WASB-C3 weights-license gate still open (unchanged; pre-existing).
```

22
23   ## Live CUJ 2026-07-07 ? release PROVEN, 3 pre-existing gaps surfaced (all filed as spawned tasks)
24   Owner ran a real /api/process CUJ ("Video 1 - Badminton.MP4", a 5312x2988 **10-bit HEVC** 120Mbps clip). Outcomes:
25   - **#509 forwarding PROVEN live**: the cloud dispatch put `--set indexing.wasb.decode_backend=nvdec_resident --set ?batch_size=64 --set ?prefetch_batches=4` into the worker exec args. And a sharp-clip green run on the D2 worker (exec `rally-worker-ql56s`, env-driven nvdec) = 19 rallies, success. Serving path is GREEN for valid footage.
26   - **Gap 1 ? slow validation (task_dcee8bf0):** ~125 random `cap.set(POS_FRAMES)` seeks across scene_cut(100)/blur(15)/relevance(10), each their own VideoCapture, on 5.3K 10-bit HEVC on the CPU-only control-plane ? ~6.5 min. Fix: single sequential/keyframe pass.
27   - **Gap 2 ? compute=local footgun (task_81572190):** SPA sent compute=local ? in-process on the weightless control-plane ? "weights not found" fail. Guard/redirect it.
28   - **Gap 3 ? worker re-validates with DEFAULT thresholds (task_72d98f83):** control-plane min_blur=8.0 (baked) NOT forwarded; worker re-validates at default 20.0 and REJECTED this soft clip (sharpness 11.9) ? rc2. This is what failed the CUJ on the cloud path. Fix: skip worker re-validation on dispatch (control-plane is authoritative) or forward validation config.
29   - **Validation-latency design (my rec):** don't move validation to GPU standalone. (1) stop the double-validation, (2) kill the seeks (sequential/keyframe), (3) end-state = ffprobe metadata pre-gate on control-plane + fold content checks (blur/court) INTO the worker's existing GPU decode pass (decode once, validate+detect together). All 3 gaps are pre-existing, NOT caused by the nvdec release.
30
31   ## (historical) release/rollout phase ? superseded by SHIPPED above
32   PR #507 squash-merged to master as **44a2f47** (branch deleted). Pre-merge gate reproduced locally: full suite 1764 passed / coverage 85.02% / ruff+mypy clean; CI 11/11 green; 2 optional review follow-ups landed (PyNvVideoCodec==2.1.0 pin in both deploy files; decode_backend persisted in native-runner detection_params). Real-L4 validation + golden gate PASS + latency/resource A/B all done (see PR #507 comment + below). **Remaining = the serving release (image rebuild + gated nvdec flip); decode_backend still defaults cpu so nothing in prod has changed yet.**
33
34   ### Serving release plan (live infra: Job `rally-worker` + service `rally-control-plane`, asia-southeast1; ONE image `rally-worker:latest` drives both ? gen_service_yaml.py:193)
35   - **Phase A (ship image, no behavior change):** `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-client-0306026826 .` ? new `rally-worker:latest` (torch2.6+pynvc2.1.0+NVDEC libs). Push does NOT change prod (Job/service pin a digest). Cut over via `gcloud run jobs update rally-worker --region asia-southeast1 --image ?:latest` + redeploy service (`gen_service_yaml.py` ? `services replace`). decode_backend defaults cpu ? cpu path under torch2.6 (validated F1 0.576?0.582). Smoke 1 clip. Rollback: `jobs update --image <old digest>`.
36   - **Phase B (enable nvdec, gated + reversible):** flip via WORKER env `WASB_DECODE_BACKEND=nvdec_resident` (native runner reads env first ? no code/preset change, instant rollback by removing it). ? THE untested risk: NVDEC on the real Cloud Run L4 ? the image bakes libnvcuvid 580.159.03; Cloud Run's host-driver ABI is unverified there (only tested on a self-managed L4 VM). MUST smoke 1 clip on the Job before trusting it. Then canary; then durable default = add `decode_backend: nvdec_resident` to the cloud-serving preset (presets.py) + rebuild + Launch Report.
37   - Cost: build ~$0.3; each Job smoke ~$0.4?0.7 (L4). Rollback levers at every step.
38
39
40   ## You are here
41   The implementation is DONE and **PR #507 is open** (branch `feat/506-gpu-resident-nvdec`, 2026-07-07): default-OFF `indexing.wasb.decode_backend = "cpu" | "nvdec_resident"`, the validated NVDEC?affine-`grid_sample`?GPU-normalize path wired into `backend/pipeline/detectors/wasb_infer.py` (`NvdecResidentFrontend` + `run_video_nvdec_resident`, same resumable cache keyed by decode_backend), native-runner threading + healthcheck gates (CUDA + PyNvVideoCodec), and the env bump (wasb env ? py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec; libnvcuvid/libnvidia-encode staged in `deploy/cloudrun/Dockerfile` + `deploy/digitalocean/gpu_setup.sh`). Verified locally: 1764 tests green, coverage 85% (floor 70), ruff/mypy/link/leak clean; a 14-agent adversarial review confirmed 7 minor findings, all fixed (notably `padding_mode="zeros"` = cv2 BORDER_CONSTANT for non-16:9 sources). Research evidence unchanged: [[gpu-resident-modernization]], issue #506 comments (F1 0.574 ? baseline 0.582 on GX010128 at SERVED, ~1.7×, GPU 47?79%).

```
42
43      ## Real-L4 validation ? DONE + golden gate PASS 2026-07-07 (this session, ~$3 of the
$100 budget; VM torn down)
44      Validated end-to-end on an L4 (`rally-506`, torn down clean, no orphan disks) ? full
evidence in the PR #507 comment. All GREEN: (1) `gpu_setup.sh` builds the env clean (py3.10 +
torch 2.6.0+cu124 cuda True + PyNvVideoCodec 2.1.0 + NVDEC lib extraction); (2) on-box **parity
gate PASS** (`pytest -k parity`, 480s ? needed test-name fix 9267255, the documented `-k parity`
selected 0 tests); (3) crash/resume byte-identical; (4) cpu?nvdec cache isolation correct; (5)
**Cloud Run image builds + `--gpus all` smoke PASS** (baked NVDEC libs decode; native
healthcheck ok with nvdec_resident); throughput 43.7 fps / GPU 82% on GX010128.
45      **Golden gate @ SERVED = PASS** (10 baseline-comparable clips, 445 rallies): mean
?f1@0.5 **+0.020**, mean ?tolF1 +0.006, worst ?0.009 (**GX010128 must-pass: 0.574 vs 0.582 ?**),
8/10 improved-or-flat. The other 5 golden clips are 4K-sourced (baselines are 1080p) so a vs-
baseline number would conflate resolution with decode ? the parity gate already covers nvdec?cpu
on identical pixels, so they add nothing to the verdict.
46      **Latency + resource A/B** (2nd L4 `rally-506b`, torn down; GX010128, batch 8, telemetry
in `gs://?nightly/dev/506/results/bench/`): nvdec **43.7 fps @ GPU 83% / CPU 30%** vs cpu-
stream(no-prefetch) 15.0 fps @ GPU 28% / CPU 49%; GPU mem +170 MiB (of 24GB), RAM ~2?3 GB flat.
The win is the **bottleneck flip (GPU?, CPU?)**; the 2.9× is vs the untuned streaming path ? vs
prefetch-tuned cpu the realistic end-to-end is ~1.6?1.8× (#506 spike). All in the PR #507
comment.
47
48      ## Staged assets (durable)
49      - **Golden video working set**: all 14 originals in `gs://khelsutra-rally-
corpus/videos/golden/<name>.mp4` + the 4 baseline-source 1080p proxies in `videos/golden_1080p/`
(mahadevpura_2, Badminton_BXH_2, Boxhill_Doubles, GX020094). Frame-count-verified against each
golden trajectory. (GOLDEN_VIDEOS.md could record this GCS location in a follow-up ? kept out of
PR #507 to not muddy its scope.)
50      - Gate trajectories: `gs://khelsutra-rally-nightly/dev/506/results/golden/*_nvdec.csv`.
Re-score with repo-root `score_golden_gate.py` (untracked dev tool, like `score_local.py`;
labels from `?corpus/labels/`, baselines from `?corpus/trajectories/`).
51
52      ## Next steps
53      1. **Merge PR #507** ? real-L4-validated + golden-gate PASS; CI green locally.
54      2. **Flip the default**: owner Launch Report ? set `decode_backend=nvdec_resident` in
the cloud-serving preset (`backend/config/presets.py`). The gate that gated this is now green.
55
56      ## Ramp-up kit
57      - PR #507 (full change description + verification record) and issue #506 (all comments ?
measured evidence).
58      - New tests: `tests/test_wasb_nvdec_resident.py` (grid/cv2 parity incl. non-16:9, cache
keying, CLI guards, the skippable on-box parity gate).
59      - Reference code still in `scratch/gpu_resident/` (`exp_t2_full.py`, `score_local.py`,
`t2setup_env.sh`, `telemetry.sh`).
60
61      ## Key decisions (don't relitigate)
62      - GPU resize MUST be affine `grid_sample` matching `cv2.warpAffine` ? NOT
`F.interpolate` (F1 0.58?0.07; only shows on hard clips). Locked by unit tests now.
63      - `padding_mode="zeros"` (cv2's BORDER_CONSTANT(0)), not "border" ? matters only for
non-16:9 sources; identical to the validated run on 16:9.
64      - PyNvVideoCodec (MIT) for GPU decode; TorchCodec couldn't load on the DL image.
65      - Score with SERVED preset (cap6+R4); raw (0.02,2.0,2.0) is degenerate.
66      - Default-OFF; nvdec/cpu detection caches never cross-replay (manifest keyed by
decode_backend).
67      - WASB weights/NYCU-dataset license (WASB-C3) is a separate launch gate, not a perf
blocker.
68
```

5. user (2026-07-09T06:23:57.609Z)

```
<system-reminder>This memory is 2 days old. Memories are point-in-time observations, not live
state ? claims about code behavior or file:line citations may be outdated. Verify against
current code before asserting as fact.</system-reminder>
1       ---
2       name: efficient-timeouts-and-retries
```

```
3       description: How the owner wants Claude to handle waits, retries, timeouts, and Windows
gcloud SSH ? fast, visible, diagnostic
4       metadata:
5         type: feedback
6       ---
7
8       The owner strongly dislikes slow, silent retry loops. On 2026-07-06, diagnosing a GPU
VM, I burned ~3 minutes on a 12x15s SSH-readiness loop that suppressed stderr with 2>/dev/null,
so it "failed" invisibly while the owner had already SSH'd into the box manually. The owner
interrupted repeatedly ("be really smart at your job").
9
10      **Why:** wasteful polling plus hidden errors makes me look slow and not smart; the owner
watches actively and interrupts. One visible failure I actually diagnose beats many blind
retries.
11
12      **How to apply:**
13      - ONE attempt with visible stderr, read the real error, then fix ? do not blind-retry a
broken command.
14      - Short, reasonable timeouts (seconds, not minutes). Escalate only with a concrete
reason.
15      - Never hide stderr (2>/dev/null) on a command whose success is uncertain.
16      - Root cause: on Windows, `gcloud compute ssh` uses PuTTY plink.exe/pscp.exe, which
reject OpenSSH `-o` flags and mishandle some `~` remote paths. Use plain `gcloud compute ssh
<vm> --zone --project --command="..."`; to push a file, base64-encode into --command (echo B64 |
base64 -d > file), not scp. To launch detached work without plink hanging ~60s, redirect all fds
and close stdin.
17      - For long remote work: run detached (nohup, log to file/GCS), poll a status file; avoid
many SSH round-trips.
18
```

6. assistant (2026-07-09T06:24:06.291Z)

The memory confirms the real code is at `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`. Let me check its git state and pull per convention. Let me also start reading the target files in parallel.

7. user (2026-07-09T06:24:10.818Z)

```
=== branch ===
codex/issue-523-sd-reel-plan-clear
=== status ===
?? "SESSION_HANDOFF (1).md"
=== last commit ===
a460d74 chore(handoff): close issue #523 and move to next open bugs
=== fetch ===
=== behind/ahead vs origin/master ===
6       10
```

8. user (2026-07-09T06:24:18.429Z)

```
=== origin/master HEAD ===
00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
2084d56 fix(storage): build GCSFuse fused path with OS-native separators (#548)
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
=== does 00e8f92 match? ===
00e8f92
```

9. user (2026-07-09T06:24:18.455Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
```

```
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      from typing import Any, List
28
29      from backend import storage
30      from backend.compute import ComputeJobSpec, get_compute_backend
31      from backend.config import (
32          StartupCheckError,
33          enforce_startup_checks,
34          load_config,
35          probe_cuda_available,
36      )
37      from backend.infrastructure.database import Database
38      from backend.pipeline.segmenters.base import segmenter_registry
39      from backend.pipeline.validators.base import registry as validator_registry
40      from backend.reporting import emit_indexer_report
41      from backend.results import Failure
42      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("backend.worker")
46
47
48      def _coerce(v: str) -> Any:
49          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50          low = v.lower()
51          if low in ("true", "false"):
52              return low == "true"
53          for cast in (int, float):
54              try:
55                  return cast(v)
56              except ValueError:
57                  pass
58          return v
59
60
61      def _apply_overrides(config: Any, sets: List[str]) -> None:
62          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63          for kv in sets or []:
64              key, sep, val = kv.partition("=")
```

```
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
```

```
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
```

```
        from the local 2/3), never a raw traceback:
          * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
            running (the shim best-effort cancels it).
          * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
            crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
            ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
        from backend.compute import RemoteExecutionFailed
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
            skip_validation=bool(getattr(args, "skip_validation", False)),
        )
        try:
            result = compute.run_infer(spec)
        except RemoteExecutionFailed as e:
            logger.error(
                "[worker] remote execution terminated without a completion marker: %s", e
            )
            return 5
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
```

```
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
        # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
        if getattr(args, "skip_validation", False):
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
                "gate and already validated this input."
            )
            val_results, val_context = [], {}
        else:
            val_results, val_context = validator_registry.run_all_with_context(
```

```
                local_video, config
            )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
```

```
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
```

```
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
        """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
        resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
        A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
        local = storage.fetch(
            spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
        )
        with open(local, encoding="utf-8") as f:
            data = json.load(f)
        if not isinstance(data, dict):
            raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
        return data


    def _write_compile_marker(
        done_uri: str,
        result: dict,
        video_id: str,
        returncode: int,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "filename": result.get("filename", ""),
                "rendition": result.get("rendition", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
```

```
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        rendition = str(spec.get("rendition") or "original")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))

        print(
            f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
            f"encoder={stitching.encoder} rendition={rendition} out={out_name}"
        )

        result = orchestration._stitch_reel(
            config,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
```

```
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
            rendition,
            # On the L4 worker the bucket mirror is the ONLY reel delivery ? the worker's
local file is
            # discarded when the Job exits. A lost mirror MUST fail the stitch (not a
404-on-"success").
            require_mirror=True,
        )
        status = str(result.get("status") or "failed")
        rc = 0 if status == "success" else 3
        if status != "success":
            print(f"[worker] compile FAILED: {result.get('message')}")

        # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
        if getattr(args, "done_uri", None):
            _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
        return rc


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
```

```
WRONG
563         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
564         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
565         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
566         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
567         rather than guesses.
568         """
569         import json
570         import subprocess
571
572         try:
573             out = subprocess.check_output(
574                 [
575                     ffprobe_bin(),
576                     "-v",
577                     "error",
578                     "-select_streams",
579                     "v:0",
580                     "-show_entries",
581                     "stream=r_frame_rate,width",
582                     "-of",
583                     "json",
584                     path,
585                 ],
586                 stderr=subprocess.DEVNULL,
587             ).decode()
588             st = (json.loads(out).get("streams") or [{}])[0]
589             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
590             fps = float(num) / float(den or "1")
591             width = float(st.get("width") or 0)
592             if fps > 0 and width > 0:
593                 return fps, width
594         except (
595             subprocess.SubprocessError,
596             ValueError,
597             KeyError,
598             OSError,
599             json.JSONDecodeError,
600         ):
601             pass
602         return None
603
604
605     def _resolve_train_detector(args: argparse.Namespace, config: Any):
606         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
607         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
608         stub=CPU mock). Returns the runner, or prints an error + returns None.
609
610         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
611         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
612         no shuttle detector and is rejected.
613         """
614         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
615
616         seg_cls = segmenter_registry.get(args.segmenter)
617         if not seg_cls:
618             print(
619                 f"[worker] unknown segmenter: {args.segmenter!r} "
```

```
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(
            u
            for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
            if u.lower().endswith(".csv")
        )
        if len(label_uris) < 2:
            print(
```

```
                f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                f"under {corpus}/labels/"
            )
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
            local_label = storage.fetch(
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
```

```
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

```

```
        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
            action="store_true",
            help="skip the worker's re-validation: the dispatcher (control-plane) already
```

```
ran the "
866             "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
867             "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
868             "`worker infer` CLI still validates.",
869         )
870         pi.set_defaults(func=cmd_infer)
871
872         pc = sub.add_parser(
873             "compile",
874             help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
875         )
876         pc.add_argument(
877             "--spec-uri",
878             required=True,
879             help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
880             "config + source uri + video_id)",
881         )
882         pc.add_argument(
883             "--out-uri",
884             default="",
885             help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
886             "Also forwarded via --set storage.output_root; either resolves the reel path.",
887         )
888         pc.add_argument(
889             "--done-uri",
890             default=None,
891             help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
892             "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
893         )
894         pc.add_argument(
895             "--config", default=None, help="config.json path (default: ./config.json)"
896         )
897         pc.add_argument(
898             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
899         )
900         pc.add_argument(
901             "--set",
902             action="append",
903             default=[],
904             metavar="k=v",
905             help="config override, e.g. --set storage.output_root=gs://bucket",
906         )
907         pc.set_defaults(func=cmd_compile)
908
909         pt = sub.add_parser(
910             "train",
911             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
912         )
913         pt.add_argument(
914             "--corpus-uri",
915             required=True,
916             help="prefix containing labels/ (+ videos/ for "
917             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
918         )
919         pt.add_argument(
920             "--out-uri",
921             required=True,
922             help="output prefix (weights/<version>/ is appended)",
```

```
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
        entry), not at import, so importing the module never reconfigures a caller's
logging."""
        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
        # configured logging at import), which would leave the level at WARNING and re-
suppress the
        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
        logging.basicConfig(
            level=logging.DEBUG if verbose else logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%H:%M:%S",
            force=True,
        )


    def main(argv=None) -> int:
        args = build_parser().parse_args(argv)
```

```
979         _setup_logging(getattr(args, "verbose", False))
980         return args.func(args)
981
982
983     if __name__ == "__main__":
984         sys.exit(main())
985
```

10. assistant (2026-07-09T06:24:49.365Z)

Origin/master is at `00e8f92` ? matches the task. Note the current branch `codex/issue-523-sd-reel-plan-clear` is **6 behind** origin/master, so it's missing PR #546 (the #534 heartbeat work) ? that's why the file I just read has no `_progress` callback. The task's line numbers reference origin/master. Per branching-safety convention, let me cut my branch from `origin/master` explicitly and read the real target code there.

11. user (2026-07-09T06:24:52.289Z)

```
branch 'fix/worker-l4-infer-observability' set up to track 'origin/master'.
Switched to a new branch 'fix/worker-l4-infer-observability'
=== now on ===
fix/worker-l4-infer-observability
=== HEAD ===
00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
```

12. assistant (2026-07-09T06:24:57.968Z)

Now on a clean branch off `origin/master`. Let me read the real target code (the origin/master version with the `_progress` callback) and the two other files central to the fix.

13. user (2026-07-09T06:24:59.411Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      import time
28      from typing import Any, List, Optional
```

```
29
30      from backend import storage
31      from backend.compute import ComputeJobSpec, get_compute_backend
32      from backend.config import (
33          StartupCheckError,
34          enforce_startup_checks,
35          load_config,
36          probe_cuda_available,
37      )
38      from backend.infrastructure.database import Database
39      from backend.pipeline.segmenters.base import segmenter_registry
40      from backend.pipeline.validators.base import registry as validator_registry
41      from backend.reporting import emit_indexer_report
42      from backend.results import Failure
43      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
44      from backend.utils.hashing import compute_video_id
45      from backend.utils.run_telemetry import RunTelemetry
46
47      logger = logging.getLogger("backend.worker")
48
49
50      def _coerce(v: str) -> Any:
51          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
52          low = v.lower()
53          if low in ("true", "false"):
54              return low == "true"
55          for cast in (int, float):
56              try:
57                  return cast(v)
58              except ValueError:
59                  pass
60          return v
61
62
63      def _apply_overrides(config: Any, sets: List[str]) -> None:
64          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true).
65          Supports dotted paths into plain dict fields (e.g. storage.gcs.mount_root=...) for
#535."""
66          for kv in sets or []:
67              key, sep, val = kv.partition("=")
68              if not sep:
69                  raise ValueError(f"--set expects k=v, got {kv!r}")
70              parts = key.split(".")
71              obj = config
72              for p in parts[:-1]:
73                  obj = getattr(obj, p)
74              last = parts[-1]
75              coerced = _coerce(val)
76              if isinstance(obj, dict):
77                  obj[last] = coerced
78              else:
79                  setattr(obj, last, coerced)
80
81
82      def _write_intervals_csv(segments: List[dict], path: str) -> None:
83          """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
84          artifact (same schema as the rally-annotator labels)."""
85          with open(path, "w", newline="") as f:
86              w = csv.writer(f)
87              w.writerow(["start", "end", "shot_count", "ending_reason"])
88              for s in segments:
89                  w.writerow(
```

```
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.

        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
        ``status="failed"`` report.json in the bucket ? the control plane's source of truth
        (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
        report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
        EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(
            video_id,
            segments,
            status,
            os.path.join(out_local, "report.json"),
            video_uri=video_uri,
```

```
        )
        out_prefix = out_uri.rstrip("/")
        pushed: List[str] = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)
        return pushed


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
        (returns True, ZERO work) when no deployment.profile is selected.

        The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
        torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
        WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
        no-op of work, not just of output."""
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        cuda = (
            probe_cuda_available() if profile else None
        )  # torch-free on the default-OFF path
        try:
            enforce_startup_checks(config, cuda_available=cuda, logger=logger)
            return True
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

        A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
        from the local 2/3), never a raw traceback:
          * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
            running (the shim best-effort cancels it).
          * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
            crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
            ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
        from backend.compute import RemoteExecutionFailed
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
            skip_validation=bool(getattr(args, "skip_validation", False)),
```

```
        )
        try:
            result = compute.run_infer(spec)
        except RemoteExecutionFailed as e:
            logger.error(
                "[worker] remote execution terminated without a completion marker: %s", e
            )
            return 5
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
```

```
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
        # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
        if getattr(args, "skip_validation", False):
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
                "gate and already validated this input."
            )
            val_results, val_context = [], {}
        else:
            val_results, val_context = validator_registry.run_all_with_context(
                local_video, config
            )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
```

```
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            out_local = os.path.join(scratch, "outputs", video_id)  # for sidecar +
telemetry
            os.makedirs(out_local, exist_ok=True)

            # Live telemetry black-box for L4 detector runs (#534): velocity + GPU/CPU/RAM
samples.
            # Written under outputs/ so the final push will mirror it; snapshots driven from
detector %.
            telem: Optional[RunTelemetry] = None
            try:
                telem = RunTelemetry(
                    os.path.join(out_local, "run_telemetry.jsonl"),
                    label=f"l4-infer-{video_id[:8]}",
                )
            except Exception:  # noqa: BLE001 - telemetry must never break a production run
                telem = None

            out_prefix = getattr(args, "out_uri", None)
```

```
            if out_prefix:
                out_prefix = out_prefix.rstrip("/")

            # Configurable sampling for sidecar upload + telemetry snapshots (#534).
            # Local sidecar file is always kept up-to-date (cheap). Only the costly remote
put
            # and nvidia-smi-using snapshot are rate-limited. 0/negative = every callback.
            sample_sec = 15.0
            try:
                idx_cfg = getattr(config, "indexing", None)
                if idx_cfg is not None:
                    raw = getattr(idx_cfg, "detector_progress_sample_sec", 15.0)
                    sample_sec = float(raw) if raw is not None else 15.0
            except Exception:  # noqa: BLE001
                sample_sec = 15.0

            last_sample_t = [0.0]

            def _progress(s: str) -> None:
                print(f"[worker] progress: {s}")
                # Write sidecar for cloud/L4 poll to surface live detector progress in job
status (#534).
                # CP will read outputs/<video_id>/*_detector_progress.json during poll.
                # The *local* file is always written so the latest state is available in
scratch.
                now = time.time()
                do_sample = sample_sec <= 0 or (now - last_sample_t[0] >= sample_sec)

                pfile = os.path.join(out_local, f"{video_id}_detector_progress.json")
                try:
                    with open(pfile, "w") as pf:
                        json.dump({"msg": s, "t": now}, pf)
                except Exception:
                    pass  # best effort

                heavy_done = False
                # LIVE push to storage (the part visible to CP poll during the run).
                if out_prefix and do_sample:
                    try:
                        storage.put(
                            pfile,
                            f"{out_prefix}/outputs/{video_id}/{os.path.basename(pfile)}",
                            config=storage_cfg,
                        )
                        heavy_done = True
                    except Exception:
                        pass  # best-effort live heartbeat

                # Drive RunTelemetry snapshots (includes nvidia-smi + file rewrite) only on
sample.
                if telem and s and "detector" in (s or "") and do_sample:
                    try:
                        m = re.search(r"(\d+)/(\d+)", s)
                        done = int(m.group(1)) if m else None
                        total = int(m.group(2)) if m else None
                        telem.snapshot(done=done, total=total, phase="detector")
                        heavy_done = True
                    except Exception:  # noqa: BLE001
                        pass

                if heavy_done:
                    last_sample_t[0] = now

            result = segmenter_cls(db, config).process_video(
                video_id,
```

```
                local_video,
                max_frames=args.max_frames,
                progress=_progress,
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
```

```
487             str(getattr(args, "video_uri", "") or ""),
488         )
489
490         print(
491             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
492         )
493         for u in pushed:
494             print("   ", u)
495
496         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
497         # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
498         # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
499         # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
500         if getattr(args, "done_uri", None):
501             _write_done_marker(
502                 args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
503             )
504         return 0
505
506
507     def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
508         """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
509         resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
510         A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
511         local = storage.fetch(
512             spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
513         )
514         with open(local, encoding="utf-8") as f:
515             data = json.load(f)
516         if not isinstance(data, dict):
517             raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
518         return data
519
520
521     def _write_compile_marker(
522         done_uri: str,
523         result: dict,
524         video_id: str,
525         returncode: int,
526         storage_cfg: dict,
527         scratch: str,
528     ) -> None:
529         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
530         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
531         try:
532             marker = {
533                 "video_id": video_id,
534                 "returncode": returncode,
535                 "status": result.get("status", "failed"),
536                 "download_url": result.get("download_url", ""),
537                 "reel_uri": result.get("reel_uri", ""),
538                 "filename": result.get("filename", ""),
539                 "rendition": result.get("rendition", ""),
540                 "message": result.get("message", ""),
541             }
```

```
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        rendition = str(spec.get("rendition") or "original")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))

        print(
            f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
            f"encoder={stitching.encoder} rendition={rendition} out={out_name} "
            f"ranges={time_pairs}"
        )

        result = orchestration._stitch_reel(
            config,
            stitching,
            video_id,
            source_ref,
```

```
            time_pairs,
            out_name,
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
            rendition,
            # On the L4 worker the bucket mirror is the ONLY reel delivery ? the worker's
local file is
            # discarded when the Job exits. A lost mirror MUST fail the stitch (not a
404-on-"success").
            require_mirror=True,
        )
        status = str(result.get("status") or "failed")
        rc = 0 if status == "success" else 3
        if status != "success":
            print(f"[worker] compile FAILED: {result.get('message')}")

        # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
        if getattr(args, "done_uri", None):
            _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
        return rc


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
```

```
647
648         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
649         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
650         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
651         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
652         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
653         rather than guesses.
654         """
655         import json
656         import subprocess
657
658         try:
659             out = subprocess.check_output(
660                 [
661                     ffprobe_bin(),
662                     "-v",
663                     "error",
664                     "-select_streams",
665                     "v:0",
666                     "-show_entries",
667                     "stream=r_frame_rate,width",
668                     "-of",
669                     "json",
670                     path,
671                 ],
672                 stderr=subprocess.DEVNULL,
673             ).decode()
674             st = (json.loads(out).get("streams") or [{}])[0]
675             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
676             fps = float(num) / float(den or "1")
677             width = float(st.get("width") or 0)
678             if fps > 0 and width > 0:
679                 return fps, width
680         except (
681             subprocess.SubprocessError,
682             ValueError,
683             KeyError,
684             OSError,
685             json.JSONDecodeError,
686         ):
687             pass
688         return None
689
690
691     def _resolve_train_detector(args: argparse.Namespace, config: Any):
692         """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
693         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
694         stub=CPU mock). Returns the runner, or prints an error + returns None.
695
696         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
697         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
698         no shuttle detector and is rejected.
699         """
700         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
701
702         seg_cls = segmenter_registry.get(args.segmenter)
703         if not seg_cls:
```

```
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(
            u
            for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
            if u.lower().endswith(".csv")
        )
```

```
        if len(label_uris) < 2:
            print(
                f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                f"under {corpus}/labels/"
            )
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
            local_label = storage.fetch(
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
```

```
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
```

```
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
```

```
950             action="store_true",
951             help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
952             "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
953             "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
954             "`worker infer` CLI still validates.",
955         )
956         pi.set_defaults(func=cmd_infer)
957
958         pc = sub.add_parser(
959             "compile",
960             help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
961         )
962         pc.add_argument(
963             "--spec-uri",
964             required=True,
965             help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
966             "config + source uri + video_id)",
967         )
968         pc.add_argument(
969             "--out-uri",
970             default="",
971             help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
972             "Also forwarded via --set storage.output_root; either resolves the reel path.",
973         )
974         pc.add_argument(
975             "--done-uri",
976             default=None,
977             help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
978             "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
979         )
980         pc.add_argument(
981             "--config", default=None, help="config.json path (default: ./config.json)"
982         )
983         pc.add_argument(
984             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
985         )
986         pc.add_argument(
987             "--set",
988             action="append",
989             default=[],
990             metavar="k=v",
991             help="config override, e.g. --set storage.output_root=gs://bucket",
992         )
993         pc.set_defaults(func=cmd_compile)
994
995         pt = sub.add_parser(
996             "train",
997             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
998         )
999         pt.add_argument(
1000             "--corpus-uri",
1001             required=True,
1002             help="prefix containing labels/ (+ videos/ for "
1003             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
1004         )
1005         pt.add_argument(
1006             "--out-uri",
```

```
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
        entry), not at import, so importing the module never reconfigures a caller's
logging."""
        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
        # configured logging at import), which would leave the level at WARNING and re-
suppress the
        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
        logging.basicConfig(
            level=logging.DEBUG if verbose else logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%H:%M:%S",
            force=True,
        )


```

```
1063    def main(argv=None) -> int:
1064        args = build_parser().parse_args(argv)
1065        _setup_logging(getattr(args, "verbose", False))
1066        return args.func(args)
1067
1068
1069    if __name__ == "__main__":
1070        sys.exit(main())
1071
```

14. assistant (2026-07-09T06:25:12.861Z)

Now I have the origin/master version with the `_progress` callback (lines 388?428) and telemetry (lines 359?368). Let me read the three other key files: the native runner (where the callback should be invoked), the telemetry module, and the validator base (for the result structure).

15. user (2026-07-09T06:25:13.957Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8       see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
```

```
    not the numbers.
    """

    import logging
    import os
    import re
    import shutil
    import subprocess
    from dataclasses import dataclass
    from typing import Callable, List, Optional, Tuple

    from backend.config.models import IndexingConfig
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext

    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    logger = logging.getLogger(__name__)

    # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
    # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
    _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")

    # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
    # so the "cuda True/False" string they parse can never drift apart.
    _CUDA_PROBE_CODE = (
        "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
    )

    # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
    # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
    _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")

    # Probe for the GPU-resident decode dep (#506): a wasb env without PyNvVideoCodec must
fail the
    # healthcheck, not the paid inference subprocess. (The NVDEC driver libs ? libnvcuvid ?
are only
    # exercised at decoder INIT, so a missing lib still surfaces in the subprocess; this
catches the
    # common misconfig, a wasb env built without the package.)
    _PYNVC_PROBE_CODE = "import PyNvVideoCodec"


    @dataclass
    class NativeWasbConfig:
        """Resolved settings for a Linux-native WASB run.

        Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
        over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
        defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
        """

        repo_dir: str = "~/models/WASB-SBDT"  # native clone of nttcom/WASB-SBDT
        python_bin: str = (
            "python"  # the `wasb` conda env python (invoked by path, no activate)
        )
        weights_path: str = ""  # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
        sport: str = "badminton"
```

```
        device: str = "cuda"  # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
        scratch_dir: str = ""  # frame-extraction scratch (env); "" = next to the output dir
        timeout_sec: int = 1800  # 30 min; a hung GPU call is killed (0 = no timeout)
        keep_frames: bool = (
            False  # retain the decoded frame cache after success (debug only)
        )
        # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
        # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
        # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
        stream_video: bool = True
        # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
        # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
        batch_size: int = 0
        prefetch_batches: int = 0
        # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
        # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
        # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
        decode_backend: str = "cpu"
        log_every_batches: int = 50  # emission rate of "D/T (pct%)" progress lines from
wasb_infer

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
            if isinstance(idx_cfg, dict):
                idx_cfg = IndexingConfig(**idx_cfg)
            w = idx_cfg.wasb
            env = os.environ.get
            # stream_video is tri-state in config: None/absent => native default (stream);
an
            # explicit True/False still wins (e.g. False for a container cv2 can't seek).
            sv = w.stream_video
            return cls(
                repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
                python_bin=env("WASB_PYTHON") or w.python_bin,
                weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
                sport=w.sport,
                device=env("WASB_DEVICE") or w.device,
                scratch_dir=env("WASB_SCRATCH")
                or env("RALLY_SCRATCH")
                or env("TMPDIR")
                or "",
                timeout_sec=w.timeout_sec,
                keep_frames=w.keep_frames,
                stream_video=(True if sv is None else bool(sv)),
                batch_size=int(w.batch_size or 0),
                prefetch_batches=int(w.prefetch_batches or 0),
                decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
                log_every_batches=int(w.log_every_batches or 50),
            )


    class NativeWasbRunner(DetectorRunner):
        """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""

        def __init__(self, cfg: NativeWasbConfig):
            self.cfg = cfg
            # Cached CUDA-availability probe (used to resolve device="auto"). Set by
```

```
healthcheck;
145             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
146             self._cuda_available: Optional[bool] = None
147
148         # --- low-level native exec (the single place tests patch)
-------------------------
149         def _run(
150             self, argv: List[str], cwd: Optional[str] = None
151         ) -> subprocess.CompletedProcess:
152             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
153             return subprocess.run(
154                 argv,
155                 cwd=cwd,
156                 capture_output=True,
157                 text=True,
158                 timeout=(self.cfg.timeout_sec or None),
159             )
160
161         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
162         def _probe_cuda(self) -> bool:
163             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
164             (missing python / timeout / import error) is treated as 'no CUDA'."""
165             try:
166                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
167             except (FileNotFoundError, subprocess.TimeoutExpired):
168                 return False
169             return res.returncode == 0 and "cuda True" in (res.stdout or "")
170
171         def _cuda_is_available(self) -> bool:
172             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
173             if self._cuda_available is None:
174                 self._cuda_available = self._probe_cuda()
175             return self._cuda_available
176
177         def _effective_device(self) -> str:
178             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
179             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
180             if self.cfg.device == AUTO:
181                 return DeviceContext(AUTO, self._cuda_is_available()).effective
182             return self.cfg.device
183
184         def _repo_src(self) -> str:
185             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
186
187         def _copy_infer_script(self) -> bool:
188             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
189             repo_src = self._repo_src()
190             try:
191                 os.makedirs(repo_src, exist_ok=True)
192                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
193                 return True
194             except OSError as e:
195                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
196                 return False
197
198         def _rm_rf(self, path: Optional[str]) -> None:
199             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
```

```
            if path:
                shutil.rmtree(path, ignore_errors=True)

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
            if not self.cfg.weights_path:
                return False, (
                    "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
                )
            weights = os.path.expanduser(self.cfg.weights_path)
            if not os.path.exists(weights):
                return False, f"WASB weights not found at {weights}"
            repo_src = self._repo_src()
            if not os.path.isdir(repo_src):
                return False, (
                    f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
                    f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
                )
            # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
            # argparse rc=2 from wasb_infer.py at run time.
            if self.cfg.device not in VALID_DEVICES:
                return False, (
                    f"unknown device {self.cfg.device!r} ? valid: {',
'.join(VALID_DEVICES)}."
                )
            if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
                return False, (
                    f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
                    f"{', '.join(_VALID_DECODE_BACKENDS)}."
                )
            try:
                res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
            except FileNotFoundError:
                return (
                    False,
                    f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
                )
            except subprocess.TimeoutExpired:
                return False, "native torch healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return (
                    False,
                    f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
                )
            # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
            self._cuda_available = "cuda True" in out
            ctx = DeviceContext(self.cfg.device, self._cuda_available)
            if ctx.cuda_required_but_missing:
                # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
                # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
                return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
            if ctx.fell_back:
                logger.warning(
                    "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
                    "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
                    out,
                )
```

```
            # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
the
            # config validator already rejects) cannot run the GPU-resident path ? fail loud
            # instead of letting wasb_infer.py die mid-run on a paid box.
            if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
                return False, (
                    f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
                    f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
                )
            if self.cfg.decode_backend == "nvdec_resident":
                try:
                    nv = self._run([self.cfg.python_bin, "-c", _PYNVC_PROBE_CODE])
                except (FileNotFoundError, subprocess.TimeoutExpired):
                    return False, "PyNvVideoCodec probe failed to run (python
missing/timeout)."
                if nv.returncode != 0:
                    return False, (
                        f"decode_backend=nvdec_resident but `{_PYNVC_PROBE_CODE}` failed in
"
                        f"{self.cfg.python_bin!r}: {(nv.stderr or nv.stdout or
'').strip()[-300:]} "
                        f"? the wasb env needs PyNvVideoCodec (deploy/cloudrun/Dockerfile /
"
                        f"gpu_setup.sh stage it)."
                    )
            return True, out

        def run_predict(
            self,
            video_win_path: str,
            output_win_dir: str,
            video_id: Optional[str] = None,
            progress: Optional[Callable[[str], None]] = None,
        ) -> Optional[str]:
            """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.

            Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
            runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
            same downstream contract, so the parity comparison is apples-to-apples.
            """
            output_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_dir, exist_ok=True)
            # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
            norm = str(video_win_path).replace("\\", "/")
            stem, _ext = os.path.splitext(os.path.basename(norm))
            # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
            key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
            out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

            if not self._copy_infer_script():
                logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
                return None

            weights = os.path.expanduser(self.cfg.weights_path)
            argv = [
                self.cfg.python_bin,
                "wasb_infer.py",
                "--video",
                os.path.abspath(str(video_win_path)),
            ]
```

```
            frames_dir: Optional[str] = None
            if self.cfg.decode_backend == "nvdec_resident":
                # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
itself
                # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
                # --stream-video nor --frames_out_dir applies and there is no frame cache to
clean.
                argv += ["--decode-backend", self.cfg.decode_backend]
            elif self.cfg.stream_video:
                # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
                argv.append("--stream-video")
            else:
                scratch = self.cfg.scratch_dir or output_dir
                frames_dir = os.path.join(scratch, f"{key}_frames")
                argv += ["--frames_out_dir", frames_dir]
            # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
            # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
            device = self._effective_device()
            if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
                # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
a
                # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
                logger.error(
                    "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
                    "%r ? aborting before the WASB subprocess.",
                    device,
                )
                return None
            argv += [
                "--weights",
                weights,
                "--sport",
                self.cfg.sport,
                "--device",
                device,
                "--out",
                out_csv,
            ]
            # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
            # byte-parity guarantee) is unchanged when the knobs are unset.
            if self.cfg.batch_size:
                argv += ["--batch-size", str(self.cfg.batch_size)]
            if self.cfg.prefetch_batches and self.cfg.decode_backend != "nvdec_resident":
                # Prefetch overlaps CPU-side batch assembly with GPU compute ? meaningless
for
                # the GPU-resident decoder (there is no CPU assembly), whose wasb_infer
branch
                # ignores the flag; keep the argv honest instead of passing a dead knob.
                argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
            if self.cfg.log_every_batches and self.cfg.log_every_batches > 0:
                argv += ["--log-every-batches", str(self.cfg.log_every_batches)]

            logger.info(
                "Running native WASB inference (device=%s%s) on %s ...",
                device,
                " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
                os.path.basename(norm),
            )
```

```
365
366             res: Optional[subprocess.CompletedProcess] = None
367             try:
368                 if progress is None:
369                     # Fast path: capture at end (existing behaviour, byte-identical for
tests).
370                     res = self._run(argv, cwd=self._repo_src())
371                 else:
372                     # Live progress path (#534): stream stdout so we can parse the periodic
373                     # "done/total (pct%) ..." lines already emitted by wasb_infer.py and
call
374                     # the callback. This gives the SPA a real heartbeat during the long GPU
pass
375                     # instead of a spinner. Still respects timeout.
376                     proc = subprocess.Popen(
377                         argv,
378                         cwd=self._repo_src(),
379                         stdout=subprocess.PIPE,
380                         stderr=subprocess.PIPE,
381                         text=True,
382                     )
383                     assert proc.stdout is not None
384                     try:
385                         for line in proc.stdout:
386                             line = line.rstrip()
387                             if line:
388                                 # Parse existing wasb_infer progress lines for detector
heartbeat.
389                                 # Example: "1234/5000 (24%) | 2.3 win/s | elapsed 45s | ETA
120s"
390                                 m = re.search(r"(\d+)/(\d+)\s+\((\d+)%\)", line)
391                                 if m:
392                                     done, total, pct = m.groups()
393                                     try:
394                                         progress(f"detector {pct}% ({done}/{total})")
395                                     except Exception:  # noqa: BLE001
396                                         pass  # progress must never kill the run
397                                 # Also log for the container/runlog.
398                                 logger.info(line)
399                         # Wait for completion (communicate for stderr/returncode).
400                         _, stderr = proc.communicate()
401                         res = subprocess.CompletedProcess(
402                             args=argv, returncode=proc.returncode, stdout="", stderr=stderr
or ""
403                         )
404                     finally:
405                         if proc.poll() is None:
406                             proc.kill()
407             except FileNotFoundError:
408                 logger.error(
409                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
410                 )
411                 return None
412             except subprocess.TimeoutExpired:
413                 logger.error(
414                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec
415                 )
416                 return None
417
418             if res is None or res.returncode != 0:
419                 logger.error(
420                     "WASB native inference failed:\n%s",
421                     (getattr(res, "stderr", None) or getattr(res, "stdout", None) or
"")[-2000:],
422                 )
```

```
423                 return None
424             if not os.path.exists(out_csv):
425                 logger.error(
426                     "WASB native finished but expected CSV not found at %s", out_csv
427                 )
428                 return None
429             logger.info("WASB trajectory CSV: %s", out_csv)
430 
431             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
432             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
433             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
434             if not self.cfg.keep_frames and frames_dir:
435                 logger.info(
436                     "Cache hygiene: removing frame cache for %s "
437                     "(set indexing.wasb.keep_frames=true to retain).",
438                     key,
439                 )
440                 self._rm_rf(frames_dir)
441             return out_csv
442 
443         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
444         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
445         trajectory_to_action_windows = staticmethod(
446             TrackNetRunner.trajectory_to_action_windows
447         )
448 
```

16. assistant (2026-07-09T06:25:48.732Z)

The native runner's streaming path (lines 366?406) reads **only `proc.stdout`** and parses `(\d+)/(\d+)\s+\((\d+)%\)`. The critical question for GAP 3: how does `wasb_infer.py` actually emit its progress lines ? to stdout via `print`, or to stderr via `logging`? And does the `nvdec_resident` branch (the live serving default) emit them at all? Let me read `wasb_infer.py`, the segmenter plumbing, the telemetry module, and the validator base.

17. user (2026-07-09T06:25:50.274Z)

```
1       """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2 
3       A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
die with
4       **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
event). When
5       that happens the only forensic evidence is whatever was last flushed to disk. This
module keeps a
6       tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
is a
7       bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
4.8 GB VRAM
8       free, CPU 60 %, RAM 21/32 GB ? then vanished."
9 
10      It is the **observability** companion to the `ExecutionPolicy` *control* layer
11      (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
signals
12      that explain *why* a run slowed or died, and lets us compare execution velocity across
quality
13      iterations.
14 
15      Design:
16      - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
from an
```

```
17        existing progress callback; fps is derived from the delta since the previous snapshot.
18        ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
sample, so
19        the last line before a kill captures the machine state at death.
20      - **Capped + rotating**: a fixed ``meta`` header line (machine / GPU / start) is always
kept; recent
21        snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
is rewritten
22        atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
written.
23      - **Graceful degradation**: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
24        from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25        hiccup must not break the run it observes.
26      - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27        and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29      All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30      """
31
32      from __future__ import annotations
33
34      import json
35      import os
36      import shutil
37      import subprocess
38      import time
39      from typing import Any, Callable, Dict, List, Optional
40
41      GpuFn = Callable[[], Optional[Dict[str, Any]]]
42      SysFn = Callable[[], Optional[Dict[str, Any]]]
43
44      # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
45      _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
46      _THROTTLE_CLOCK_FRAC = 0.8  # SM clock below this fraction of max => flagged throttled
47      _THROTTLE_TEMP_C = (
48          84.0  # ... or temperature at/above this (laptop Max-Q throttle territory)
49      )
50
51
52      def _num(s: str) -> Optional[float]:
53          try:
54              return float(s)
55          except (TypeError, ValueError):
56              return None
57
58
59      def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
60          """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
61
62          Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
63          derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
64          """
65          exe = shutil.which("nvidia-smi")
66          if not exe:
67              return None
68          try:
69              out = subprocess.run(
```

```
                [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
                capture_output=True,
                text=True,
                timeout=timeout,
                check=True,
            ).stdout.strip()
        except (subprocess.SubprocessError, OSError):
            return None
        row = out.splitlines()[0] if out else ""
        parts = [p.strip() for p in row.split(",")]
        if len(parts) < 7:
            return None
        vals = [_num(p) for p in parts[:7]]
        temp, util, power, sm, sm_max, used, free = vals
        throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
        if sm is not None and sm_max:
            throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
        return {
            "util_pct": util,
            "temp_c": temp,
            "power_w": power,
            "sm_mhz": sm,
            "sm_max_mhz": sm_max,
            "vram_used_mb": used,
            "vram_free_mb": free,
            "throttled": throttled,
        }


    def system_stats() -> Optional[Dict[str, Any]]:
        """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
raises."""
        try:
            import psutil  # optional dep (BSD-3); only used when present
        except ImportError:
            return None
        try:
            vm = psutil.virtual_memory()
            return {
                "cpu_pct": psutil.cpu_percent(interval=None),
                "ram_used_gb": round(vm.used / 1e9, 2),
                "ram_total_gb": round(vm.total / 1e9, 2),
                "ram_pct": vm.percent,
            }
        except Exception:  # noqa: BLE001 - psutil edge cases must not break the run
            return None


    def gpu_name() -> Optional[str]:
        """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
        exe = shutil.which("nvidia-smi")
        if not exe:
            return None
        try:
            out = subprocess.run(
                [exe, "--query-gpu=name", "--format=csv,noheader"],
                capture_output=True,
                text=True,
                timeout=4.0,
                check=True,
            ).stdout
        except (subprocess.SubprocessError, OSError):
            return None
        return out.splitlines()[0].strip() if out else None

```

```
134
135     class RunTelemetry:
136         """A small rotating JSONL runlog of execution velocity + machine health.
137
138         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
msg)`` at
139         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
event lines
140         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
141         """
142
143         def __init__(
144             self,
145             path: str,
146             label: str = "run",
147             max_bytes: int = 10_000,
148             gpu_fn: GpuFn = gpu_stats,
149             sys_fn: SysFn = system_stats,
150             clock: Callable[[], float] = time.time,
151         ) -> None:
152             self.path = path
153             self.label = label
154             self.max_bytes = max_bytes
155             self._gpu_fn = gpu_fn
156             self._sys_fn = sys_fn
157             self._clock = clock
158             self._t0 = clock()
159             self._last_done: Optional[int] = None
160             self._last_t: Optional[float] = None
161             self._lines: List[
162                 str
163             ] = []  # recent body lines (on-disk ring, trimmed to max_bytes)
164             self._meta = json.dumps(
165                 {
166                     "meta": {
167                         "label": label,
168                         "started": self._iso(self._t0),
169                         "pid": os.getpid(),
170                         "gpu_name": gpu_name(),
171                         "cpu_count": os.cpu_count(),
172                     }
173                 },
174                 separators=(",", ":"),
175             )
176             self._write()
177
178         # -- public API ---------------------------------------------------------
179         def snapshot(
180             self,
181             done: Optional[int] = None,
182             total: Optional[int] = None,
183             phase: Optional[str] = None,
184             **extra: Any,
185         ) -> Dict[str, Any]:
186             """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
187             now = self._clock()
188             rec: Dict[str, Any] = {
189                 "t": self._iso(now),
190                 "elapsed_s": round(now - self._t0, 1),
191             }
192             if phase is not None:
193                 rec["phase"] = phase
194             if done is not None:
195                 rec["done"] = done
```

```
                if total:
                    rec["total"] = total
                    rec["pct"] = round(100.0 * done / total, 1)
                if (
                    self._last_done is not None
                    and self._last_t is not None
                    and now > self._last_t
                ):
                    rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
                self._last_done, self._last_t = done, now
            rec.update(extra)
            self._attach_health(rec)
            self._append(rec)
            return rec

        def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
            """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
            now = self._clock()
            rec: Dict[str, Any] = {
                "t": self._iso(now),
                "elapsed_s": round(now - self._t0, 1),
                "event": kind,
            }
            if msg:
                rec["msg"] = msg
            rec.update(extra)
            self._attach_health(rec)
            self._append(rec)
            return rec

        # -- internals ------------------------------------------------------
        def _attach_health(self, rec: Dict[str, Any]) -> None:
            gpu = self._safe(self._gpu_fn)
            sys_ = self._safe(self._sys_fn)
            if gpu is not None:
                rec["gpu"] = gpu
            if sys_ is not None:
                rec["sys"] = sys_

        def _append(self, rec: Dict[str, Any]) -> None:
            self._lines.append(json.dumps(rec, separators=(",", ":")))
            budget = self.max_bytes - len(self._meta) - 1
            while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
                self._lines.pop(0)  # drop oldest body line; the meta header is always kept
            self._write()

        def _write(self) -> None:
            try:
                tmp = self.path + ".tmp"
                with open(tmp, "w", encoding="utf-8") as f:
                    f.write(self._meta + "\n")
                    for s in self._lines:
                        f.write(s + "\n")
                os.replace(tmp, self.path)
            except OSError:
                pass  # telemetry must never break the run it observes

        @staticmethod
        def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
            try:
                return fn()
            except Exception:  # noqa: BLE001 - a collector hiccup must not break the run
                return None

```

```
260         @staticmethod
261         def _iso(t: float) -> str:
262             return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
263
```

18. user (2026-07-09T06:25:50.631Z)

```
1       import hashlib
2       import json
3       import logging
4       from abc import ABC, abstractmethod
5       from typing import Any, List
6
7       from pydantic import BaseModel
8
9       from backend.pipeline.validators.frame_sampler import (
10          FRAME_META_KEY,
11          FRAME_STATES_KEY,
12          SampleMeta,
13          run_frame_sampling,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bumped when validator LOGIC (not config) changes the verdict for identical
bytes+config ? e.g. a
19      # validator is rewritten, or the sampling arithmetic changes. It rides inside
config_fingerprint, so
20      # a bump invalidates every cached validation verdict (see backend/orchestration.py's
validation cache
21      # + backend/infrastructure/database.py v10). The validator SET and each validator's
config-derived
22      # thresholds are already folded into the fingerprint; this covers changes the config
can't express.
23      _VALIDATION_FINGERPRINT_SCHEMA = 1
24
25
26      class ValidationResult(BaseModel):
27          validator_name: str
28          passed: bool
29          message: str
30          details: dict[str, Any] = {}
31
32
33      class VideoValidator(ABC):
34          @property
35          @abstractmethod
36          def name(self) -> str:
37              """Unique identifier for this validator."""
38              pass
39
40          @property
41          @abstractmethod
42          def description(self) -> str:
43              """Human-readable explanation of what this validator checks."""
44              pass
45
46          @abstractmethod
47          def validate(
48              self, video_path: str, config: Any, context: dict[str, Any]
49          ) -> ValidationResult:
50              """
51              Executes validation.
52              - video_path: Local path to the MP4 file.
53              - config: AppConfig (typed) ? callers pass the typed model; attribute access
```

```
              preferred over .get() dict chains.
            - context: Dynamic context dict shared across validators (stores shared values
              like ffprobe metadata).
            """
            pass


    class FrameSamplingValidator(VideoValidator):
        """A validator whose work is sampling decoded frames.

        The three frame-decoding validators (scene_cut, blur, relevance) used to each
        open their own ``cv2.VideoCapture`` and seek to their sample frames ? ~125
        random ``POS_FRAMES`` seeks across three decode opens, which cost minutes on
        long-GOP 4K clips (see ``frame_sampler``). Subclasses instead declare *which*
        frame indices they want (:meth:`sample_indices`) and reduce those frames
        online (:meth:`_init_state` / :meth:`_consume`), so the registry can decode
        the whole set **once** in a single shared forward pass.

        :meth:`validate` uses the shared pass' result when the registry ran it; when
        a subclass is invoked standalone (direct ``.validate`` call, no shared
        context) it decodes its own indices in one pass via the same engine. Either
        way the frames ? and therefore the pass/fail decision ? are identical to the
        old per-validator seeks.
        """

        @abstractmethod
        def sample_indices(self, total_frames: int, fps: float) -> List[int]:
            """Absolute frame indices this validator samples, in ascending order.

            Pure and deterministic in ``(total_frames, fps)`` ? the shared pass unions
            these across validators to drive one decode, and the regression suite pins
            them to the legacy per-validator seek arithmetic.
            """
            ...

        @abstractmethod
        def _init_state(self, config: Any) -> Any:
            """Fresh accumulator for one run (may pre-resolve config-derived params)."""
            ...

        @abstractmethod
        def _consume(self, state: Any, frame_idx: int, frame: Any) -> None:
            """Fold one sampled frame (BGR ndarray) into ``state``. Called in index
order."""
            ...

        @abstractmethod
        def _finalize(
            self, state: Any, meta: SampleMeta, config: Any, context: dict[str, Any]
        ) -> ValidationResult:
            """Render the ValidationResult from the accumulated ``state`` + decode meta."""
            ...

        def validate(
            self, video_path: str, config: Any, context: dict[str, Any]
        ) -> ValidationResult:
            states = context.get(FRAME_STATES_KEY)
            if states is not None and self.name in states:
                # Shared single pass already decoded and reduced our frames.
                meta = context[FRAME_META_KEY]
                state = states[self.name]
            else:
                # Standalone: decode just this validator's indices in one pass.
                meta, own_states = run_frame_sampling(video_path, [self], config)
                state = own_states[self.name]
```

```
            return self._finalize(state, meta, config, context)


    class ValidationRegistry:
        def __init__(self):
            self._validators: List[VideoValidator] = []

        def register(self, validator: VideoValidator):
            """Registers a new validator."""
            self._validators.append(validator)

        def run_all_with_context(self, video_path: str, config: Any):
            """Runs all validators and returns ``(results, context)``.

            The per-run ``context`` accumulates shared values (notably
            ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
            discards it; callers that want media metadata for an observability report
            use this variant instead of re-probing.
            """
            results = []
            context: dict[
                str, Any
            ] = {}  # Shared context (e.g. for caching ffprobe outputs)
            self._prepare_frame_samples(video_path, config, context)
            for val in self._validators:
                try:
                    res = val.validate(video_path, config, context)
                    results.append(res)
                except Exception as e:
                    results.append(
                        ValidationResult(
                            validator_name=val.name,
                            passed=False,
                            message=f"Validator threw an unexpected error: {str(e)}",
                        )
                    )
            return results, context

        def _prepare_frame_samples(
            self, video_path: str, config: Any, context: dict[str, Any]
        ) -> None:
            """Decode every frame-sampling validator's frames in ONE shared pass.

            Populates ``context`` with the shared decode meta + per-validator
            accumulator state; each :class:`FrameSamplingValidator` then finalizes
            from that instead of opening its own capture. Best-effort: any failure
            here is swallowed so validators transparently fall back to a standalone
            pass (the run must never crash on a decode hiccup)."""
            samplers = [
                v for v in self._validators if isinstance(v, FrameSamplingValidator)
            ]
            if not samplers:
                return
            try:
                meta, states = run_frame_sampling(video_path, samplers, config)
            except Exception:
                logger.exception(
                    "Shared frame-sampling pass failed for %s; validators will fall "
                    "back to standalone decode.",
                    video_path,
                )
                return
            context[FRAME_META_KEY] = meta
            context[FRAME_STATES_KEY] = states

```

```
183         def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:
184             """Runs all registered validators in sequence, accumulating results."""
185             results, _ = self.run_all_with_context(video_path, config)
186             return results
187 
188         def config_fingerprint(self, config: Any) -> str:
189             """Stable hash of EVERYTHING that determines a validation verdict for fixed
video bytes.
190 
191             Powers the video_id-keyed validation-result cache (backend/orchestration.py): a
reupload of
192             identical content reuses its stored verdict ONLY while this fingerprint still
matches, so any
193             change that could flip a decision correctly forces a re-validation (never serve
a stale
194             PASS/FAIL). What goes in ? and why each matters:
195 
196             * the whole ``validation`` config block ? every threshold the validators read
197               (min_blur_threshold / min_fps / min_resolution / max_scene_cuts_per_minute /
pts gap /
198               relevance court-color bounds). Hashing the full model (not a hand-picked
subset) means a
199               NEW validation knob is covered automatically.
200             * ``sport`` ? the relevance validator keys its court-color profile off
``config.sport`` (a
201               top-level field, NOT under ``validation``), so switching sports changes the
verdict.
202             * the sorted validator NAMES ? adding/removing a validator changes which checks
run.
203             * a schema version (:data:`_VALIDATION_FINGERPRINT_SCHEMA`) ? a manual escape
hatch for
204               validator LOGIC changes the config can't express.
205 
206             Accepts a typed ``AppConfig`` (attribute access) or a plain dict;
``default=str`` keeps the
207             dump total on any exotic value. Deterministic: ``sort_keys`` fixes ordering."""
208             validation: Any = getattr(config, "validation", None)
209             if validation is None and isinstance(config, dict):
210                 validation = config.get("validation")
211             validation_repr = (
212                 validation.model_dump(mode="json")
213                 if hasattr(validation, "model_dump")
214                 else validation
215             )
216             sport: Any = getattr(config, "sport", None)
217             if sport is None and isinstance(config, dict):
218                 sport = config.get("sport")
219             payload = {
220                 "schema": _VALIDATION_FINGERPRINT_SCHEMA,
221                 "validators": sorted(v.name for v in self._validators),
222                 "sport": sport,
223                 "validation": validation_repr,
224             }
225             blob = json.dumps(
226                 payload, sort_keys=True, default=str, separators=(",", ":")
227             )
228             return hashlib.sha256(blob.encode("utf-8")).hexdigest()
229 
230 
231     # Shared global registry
232     registry = ValidationRegistry()
233 
```

19. assistant (2026-07-09T06:26:02.690Z)

`RunTelemetry.snapshot(done, total, phase=...)` matches the `_progress` call site. `ValidationResult` has `validator_name/passed/message/details` ? **no top-level `score`**; the "score" the fix wants must come out of `details`. Now the two decisive questions for GAP 3: (a) does `process_video` actually forward `progress` down to `run_predict`, and (b) how/where does `wasb_infer.py` emit the `done/total (pct%)` lines ? stdout or stderr, and does the nvdec branch emit them? Let me search.

20. user (2026-07-09T06:26:04.827Z)

```
180:        progress: Optional[Callable[[str], None]] = None,
300:        csv_path = runner.run_predict(video_path, out_dir, video_id=video_id,
progress=progress)
```

21. user (2026-07-09T06:26:06.542Z)

```
22:    ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
24:    most ``--flush-every`` windows of GPU work instead of the whole pass.
46:        --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-every
1000] [--fresh]
49:import argparse
168:        f.flush()
277:                    break  # truncated trailing line (crash mid-flush)
289:    """Append window records as JSONL and durably flush (bounded loss on crash)."""
292:    fh.flush()
303:        f.flush()
524:    log_every_batches: int = 50,
526:    flush_every: int = 1000,
624:        logger.info(
629:            logger.info("cache incompatible with this run -> starting fresh")
678:        log_every = max(1, log_every_batches)
679:        flush_n = max(1, flush_every)
681:        logger.info(
729:                    if len(pending) >= flush_n or done >= windows_total:
734:                    if (bi + 1) % log_every == 0 or done >= windows_total:
739:                        logger.info(
741:                            f"({100 * done // max(1, windows_total)}%) | {rate:.1f} win/s |
"
742:                            f"elapsed {el:.0f}s | ETA {eta:.0f}s"
750:        logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
752:        logger.info("detector cache already complete -> tracker-only re-run")
783:    logger.info(f"wrote {len(rows)} rows -> {out_csv}")
784:    logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
806:        logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
816:    logger.info(f"extracted {i} frames -> {frames_dir}")
882:    log_every_batches: int = 50,
884:    flush_every: int = 1000,
911:            log_every_batches=log_every_batches,
913:            flush_every=flush_every,
1069:    log_every_batches: int = 50,
1071:    flush_every: int = 1000,
1112:        logger.info(
1136:            logger.info(
1141:                logger.info("cache incompatible with this run -> starting fresh")
1175:            log_every = max(1, log_every_batches)
1176:            flush_n = max(1, flush_every)
1180:            logger.info(
1220:                        if len(pending) >= flush_n or done >= windows_total:
1225:                        if n_batches % log_every == 0 or done >= windows_total:
1230:                            logger.info(
1232:                                f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
1233:                                f"elapsed {el:.0f}s | ETA {eta:.0f}s"
1235:                if pending:  # safety net; the done>=windows_total branch already flushed
1241:            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
1243:            logger.info("detector cache already complete -> tracker-only re-run")
```

```
1250:def main():
1251:    ap = argparse.ArgumentParser()
1297:        "--log-every-batches", type=int, default=50, help="progress log cadence"
1305:        "--flush-every",
1332:            log_every_batches=args.log_every_batches,
1334:            flush_every=args.flush_every,
1349:            log_every_batches=args.log_every_batches,
1351:            flush_every=args.flush_every,
1373:        log_every_batches=args.log_every_batches,
1375:        flush_every=args.flush_every,
```

22. assistant (2026-07-09T06:26:20.781Z)

Root cause of GAP 3 found. `wasb_infer.py` emits its progress lines via `logger.info(...)` (lines 739?742 and 1230?1233) ? Python `logging` goes to **stderr** by default. But the native runner's live loop reads **only `proc.stdout`** (native_wasb_runner.py:385), capturing stderr only at the end via `communicate()`. So the progress lines never reach the parser and `_progress` never fires. Let me confirm the exact emission format, the two code paths (CPU vs nvdec), and how logging is configured in `wasb_infer.py`.

23. user (2026-07-09T06:26:22.810Z)

```
49      import argparse
50      import csv
51      import glob
52      import json
53      import logging
54      import os
55      import os.path as osp
56      from collections import defaultdict
57      from contextlib import contextmanager
58
59      logger = logging.getLogger(__name__)
60
61      CACHE_VERSION = 1
62      _DET_FILE = "det_raw.jsonl"
63      _MANIFEST_FILE = "manifest.json"
64
65      # Pixel-decode backends (#506). "cpu" = the historical cv2 paths (disk frames or
66      # --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
67      # resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
68      DECODE_BACKENDS = ("cpu", "nvdec_resident")
69
70      # ImageNet statistics ? MUST match WASB's build_img_transforms Normalize (the GPU-
resident
71      # frontend normalizes on-device instead of through torchvision's transform).
72      IMAGENET_MEAN = (0.485, 0.456, 0.406)
73      IMAGENET_STD = (0.229, 0.224, 0.225)
74
75
76      def _build_cfg(weights: str, sport: str, device: str = "cuda"):
77          """Compose the WASB eval config via Hydra, overriding model/weights/device.
78
```

24. user (2026-07-09T06:26:23.663Z)

```
720                             for ie in sorted(batch_results[ib].keys()):
721                                 p = img_paths[ie][ib]
722                                 fid = _frame_id(p)
723                                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
724                                 frames_payload.append([fid, plain])
725                                 det_by_fid[fid].extend(plain)
726                             pending.append({"w": win_idx, "f": frames_payload})
727                             win_idx += 1
```

```
728                         done += nb
729                         if len(pending) >= flush_n or done >= windows_total:
730                             _append_windows(det_f, pending)
731                             pending = []
732                             base_manifest["windows_done"] = done
733                             write_manifest(cache_dir, base_manifest)
734                         if (bi + 1) % log_every == 0 or done >= windows_total:
735                             el = time.time() - t0
736                             new_done = done - windows_done
737                             rate = new_done / el if el > 0 else 0.0
738                             eta = (windows_total - done) / rate if rate > 0 else 0.0
739                             logger.info(
740                                 f"{done}/{windows_total} "
741                                 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
742                                 f"elapsed {el:.0f}s | ETA {eta:.0f}s"
743                             )
744                     if pending:
745                         _append_windows(det_f, pending)
746                         base_manifest["windows_done"] = done
747                         write_manifest(cache_dir, base_manifest)
748             finally:
749                 det_f.close()
750             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
751         else:
752             logger.info("detector cache already complete -> tracker-only re-run")
753 
754         # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
755         return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
756 
757 
758     def _tracker_pass_and_write(cfg, det_by_fid, out_csv: str) -> str:
759         """Tracker re-run over the full ordered detection cache ? trajectory CSV.
760 
761         Shared verbatim by every decode backend: the tracker consumes the same per-frame
762         candidate lists regardless of how the pixels were decoded, so keeping this single
763         keeps the output contract single.
764 
765         NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
766         `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
767         the mypy false positive where the main env resolves `trackers` to the Roboflow
package
768         (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
769         subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
770         """
771         from trackers import build_tracker  # type: ignore[attr-defined]
772 
773         _ensure_np_inf()  # OnlineTracker reads np.Inf (removed in NumPy 2.0)
774         tracker = build_tracker(cfg)
```

25. assistant (2026-07-09T06:26:33.415Z)

Confirmed: progress format `{done}/{windows_total} ({pct}%) | ... win/s | elapsed ...` matches the regex ? the emission path is the problem, not the pattern. Let me confirm the `main()` logging config (does it route to stdout or stderr?) and that the nvdec branch emits identically.

26. user (2026-07-09T06:26:35.921Z)

```
1215                             pending.append({"w": win_idx, "f": frames_payload})
1216                             win_idx += 1
1217                         done += nb
```

```
1218                            n_batches += 1
1219                            batch_fids, batch_imgs = [], []
1220                            if len(pending) >= flush_n or done >= windows_total:
1221                                _append_windows(det_f, pending)
1222                                pending = []
1223                                base_manifest["windows_done"] = done
1224                                write_manifest(cache_dir, base_manifest)
1225                            if n_batches % log_every == 0 or done >= windows_total:
1226                                el = time.time() - t0
1227                                new_done = done - windows_done
1228                                rate = new_done / el if el > 0 else 0.0
1229                                eta = (windows_total - done) / rate if rate > 0 else 0.0
1230                                logger.info(
1231                                    f"{done}/{windows_total} "
1232                                    f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
1233                                    f"elapsed {el:.0f}s | ETA {eta:.0f}s"
1234                                )
1235                    if pending:  # safety net; the done>=windows_total branch already
flushed
1236                        _append_windows(det_f, pending)
1237                        base_manifest["windows_done"] = done
1238                        write_manifest(cache_dir, base_manifest)
1239                finally:
1240                    det_f.close()
1241                logger.info(f"detection done in {time.time() - t0:.0f}s; running
tracker...")
1242            else:
1243                logger.info("detector cache already complete -> tracker-only re-run")
1244        finally:
```

27. user (2026-07-09T06:26:36.566Z)

```
1250    def main():
1251        ap = argparse.ArgumentParser()
1252        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
1253        ap.add_argument(
1254            "--video", help="video file; frames are extracted internally (in WSL)"
1255        )
1256        ap.add_argument(
1257            "--frames_out_dir", help="where to extract frames when --video is used"
1258        )
1259        ap.add_argument(
1260            "--stream-video",
1261            action="store_true",
1262            help="FAST PATH: decode --video on the fly and feed frames straight to "
1263            "the detector (no PNG extraction). Output-equivalent to the disk path.",
1264        )
1265        ap.add_argument(
1266            "--decode-backend",
1267            default="cpu",
1268            choices=list(DECODE_BACKENDS),
1269            help="cpu (default) = the historical cv2 decode paths (disk frames or "
1270            "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1271            "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
1272            "~1.7x on an L4. Requires --video; detections cache separately from cpu "
1273            "(trajectory-level, not byte-level, equivalence).",
1274        )
1275        ap.add_argument("--weights", required=True)
1276        ap.add_argument("--out", required=True)
1277        ap.add_argument("--sport", default="badminton")
1278        ap.add_argument(
1279            "--device",
1280            default="cuda",
1281            choices=["cuda", "mps", "cpu"],
```

```
            help="torch device for the detector (default cuda = the WSL parity path)",
        )
        ap.add_argument(
            "--limit", type=int, default=0, help="cap #frames (for quick tests)"
        )
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument(
            "--prefetch-batches",
            type=int,
            default=0,
            help="streaming mode: overlap CPU batch assembly with GPU inference via a "
            "bounded producer thread of this queue depth (0 = off, the parity default)",
        )
        ap.add_argument(
            "--log-every-batches", type=int, default=50, help="progress log cadence"
        )
        ap.add_argument(
            "--cache-dir",
            default=None,
            help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
        )
        ap.add_argument(
            "--flush-every",
            type=int,
            default=1000,
            help="fsync the detector cache every N windows (bounds reboot loss)",
        )
        ap.add_argument(
            "--fresh", action="store_true", help="ignore any existing cache and start over"
        )
        args = ap.parse_args()

        if args.decode_backend == "nvdec_resident":
            # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither the
            # --stream-video cv2 path nor the extract-frames fallback below may run.
            if not args.video:
                raise SystemExit("--decode-backend nvdec_resident requires --video")
            if args.device != "cuda":
                raise SystemExit(
                    "--decode-backend nvdec_resident requires --device cuda "
                    "(NVDEC decode runs on the GPU)"
                )
            run_video_nvdec_resident(
                args.video,
                args.weights,
                args.out,
                args.sport,
                args.limit,
                batch_size=args.batch_size,
                log_every_batches=args.log_every_batches,
                cache_dir=args.cache_dir,
                flush_every=args.flush_every,
                fresh=args.fresh,
            )
            return

        if args.stream_video:
            if not args.video:
                raise SystemExit("--stream-video requires --video")
            run_video_streaming(
                args.video,
                args.weights,
                args.out,
                args.sport,
```

```
1347                args.limit,
1348                batch_size=args.batch_size,
1349                log_every_batches=args.log_every_batches,
1350                cache_dir=args.cache_dir,
1351                flush_every=args.flush_every,
1352                fresh=args.fresh,
1353                device=args.device,
1354                prefetch_batches=args.prefetch_batches,
1355            )
1356            return
1357
1358        frames_dir = args.frames_dir
1359        if args.video:
1360            frames_dir = extract_frames(
1361                args.video, args.frames_out_dir or (args.video + "_frames")
1362            )
1363        if not frames_dir:
1364            raise SystemExit("provide --frames_dir or --video")
1365        run(
1366            frames_dir,
1367            args.weights,
1368            args.out,
1369            args.sport,
1370            args.limit,
1371            batch_size=args.batch_size,
1372            num_workers=args.num_workers,
1373            log_every_batches=args.log_every_batches,
1374            cache_dir=args.cache_dir,
1375            flush_every=args.flush_every,
1376            fresh=args.fresh,
1377            device=args.device,
1378        )
1379
1380
1381    if __name__ == "__main__":
1382        main()
1383
```

28. assistant (2026-07-09T06:27:34.619Z)

Now let me check a decisive detail: does `wasb_infer.py` configure logging at all? If it doesn't, `logger.info(...)` sits below the root logger's default WARNING level and is **suppressed entirely** (not even reaching stderr) ? which changes the fix. Let me verify.

29. user (2026-07-09T06:27:37.227Z)

No matches found

30. user (2026-07-09T06:27:38.032Z)

```
1040                    f"expected 3-channel RGB (is output_color_type=RGB supported here?)"
1041                )
1042            return t
1043
1044        def read_chw(self, index: int):
1045            """Decoded frame ``index`` ? the normalized ``[3, inp_h, inp_w]`` float32 CUDA
1046            tensor the detector expects (warpAffine-faithful resize + ImageNet
normalize)."""
1047            import torch.nn.functional as F
1048
1049            hwc = self._decode_rgb_hwc(index)
1050            chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
1051            # padding_mode="zeros" = cv2.warpAffine's BORDER_CONSTANT(0) black fill ? see
1052            # build_affine_grid's docstring (only non-16:9 sources ever sample the border).
```

```
1053            warped = F.grid_sample(
1054                chw, self._grid, mode="bilinear", align_corners=False, padding_mode="zeros"
1055            ).squeeze(0)
1056            return (warped - self._mean) / self._std
1057
1058        def release(self):
1059            self._dec = None
1060
1061
1062    def run_video_nvdec_resident(
1063        video_path: str,
1064        weights: str,
1065        out_csv: str,
1066        sport: str = "badminton",
1067        limit: int = 0,
1068        batch_size: int = 8,
1069        log_every_batches: int = 50,
```

31. assistant (2026-07-09T06:29:51.949Z)

Plumbing check + score source. The grep already confirmed `process_video(progress=...)` forwards to `run_predict(progress=progress)` (trajectory_hybrid.py:300), so the callback IS wired ? GAP 3 is purely emission/capture. Two compounding bugs there: (1) `wasb_infer.py` **never configures logging**, so `logger.info` progress sits below root WARNING and is largely suppressed; (2) even when emitted it goes to **stderr**, but the runner reads **stdout only** (and draining stdout while stderr's 64 KB pipe fills risks a deadlock). Now let me confirm the progress plumbing has no early-out, and find where validators put a numeric "score" in `details`.

32. user (2026-07-09T06:29:53.958Z)

```
170             Persisted candidates are NOT affected ? they remain the full superset for
offline
171             re-scoring."""
172             return list(range(len(windows)))
173
174         def process_video(  # type: ignore[override]
175             self,
176             video_id: str,
177             video_path: str,
178             player_pool: Optional[List[str]] = None,
179             max_frames: Optional[int] = None,
180             progress: Optional[Callable[[str], None]] = None,
181         ) -> "Union[List[Dict[str, Any]], Failure]":
182             # Result contract (the ABC declares Success|Failure). Anything that means the
run could
183             # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
184             # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
185             # not ready, or run_predict timing out / aborting with no trajectory ? returns a
``Failure``
186             # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
187             # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
188             # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
189             label = self.display_label
190             # --- Sport profile + AI provider wiring (identical across segmenters) ---
191             sport_name = self.config.sport
192             try:
193                 sport_profile = sport_registry.get(sport_name)
194             except KeyError as e:
195                 logger.error(str(e))
196                 # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
```

```
197                 return Failure(str(e))
198
199             idx_cfg = self.config.indexing
```

33. user (2026-07-09T06:29:55.393Z)

```
<persisted-output>
Output too large (29.2KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-
Projects-badminton-highlight-indexer\e663947f-2127-4383-9905-650340ff9863\tool-
results\toolu_012XtzFntSb45bW2HyhK5Chq.txt

Preview (first 2KB):
backend\pipeline\validators\base.py:26:class ValidationResult(BaseModel):
backend\pipeline\validators\base.py-27-    validator_name: str
backend\pipeline\validators\base.py-28-    passed: bool
backend\pipeline\validators\base.py-29-    message: str
backend\pipeline\validators\base.py-30-    details: dict[str, Any] = {}
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-31-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-32-
--
backend\pipeline\validators\base.py:148:                    ValidationResult(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-149-                        validator_name=val.name,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-150-                        passed=False,
backend\pipeline\validators\base.py-151-                        message=f"Validator threw an
unexpected error: {str(e)}",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-152-                    )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-153-                )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-154-        return results, context
--
backend\pipeline\validators\blur.py:39:            return ValidationResult(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-40-                validator_name=self.name,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-41-                passed=False,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-42-                message="Failed to open video
file for blur analysis.",
...
</persisted-output>
```

34. assistant (2026-07-09T06:30:02.089Z)

Let me read the full validator-details survey to understand the `details` structure and find the score keys.

35. user (2026-07-09T06:30:03.904Z)

```
1       backend\pipeline\validators\base.py:26:class ValidationResult(BaseModel):
2       backend\pipeline\validators\base.py-27-    validator_name: str
3       backend\pipeline\validators\base.py-28-    passed: bool
4       backend\pipeline\validators\base.py-29-    message: str
5       backend\pipeline\validators\base.py-30-    details: dict[str, Any] = {}
6       C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-31-
7       C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-32-
8       --
```

```
9       backend\pipeline\validators\base.py:148:                    ValidationResult(
10      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-149-                        validator_name=val.name,
11      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-150-                        passed=False,
12      backend\pipeline\validators\base.py-151-                        message=f"Validator
threw an unexpected error: {str(e)}",
13      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-152-                    )
14      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-153-                )
15      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-154-        return results, context
16      --
17      backend\pipeline\validators\blur.py:39:            return ValidationResult(
18      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-40-                validator_name=self.name,
19      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-41-                passed=False,
20      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-42-                message="Failed to open video
file for blur analysis.",
21      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-43-            )
22      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-44-
23      backend\pipeline\validators\blur.py-45-        if meta.total_frames <= 0:
24      backend\pipeline\validators\blur.py:46:            return ValidationResult(
25      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-47-                validator_name=self.name,
passed=False, message="Invalid frame count."
26      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-48-            )
27      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-49-
28      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-50-        variances = state["variances"]
29      backend\pipeline\validators\blur.py-51-        if not variances:
30      backend\pipeline\validators\blur.py:52:            return ValidationResult(
31      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-53-                validator_name=self.name,
32      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-54-                passed=False,
33      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-55-                message="Could not read frames to
perform blur verification.",
34      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-56-            )
35      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-57-
36      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-58-        avg_variance = sum(variances) /
len(variances)
37      --
38      backend\pipeline\validators\blur.py:71:            return ValidationResult(
39      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-72-                validator_name=self.name,
40      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-73-                passed=False,
41      backend\pipeline\validators\blur.py-74-                message=f"Video quality rejected:
Focus check failed. Average sharpness score of {round(avg_variance, 1)} is below the minimum
required threshold of {min_blur_threshold}.",
42      backend\pipeline\validators\blur.py:75:                details=details,
43      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-76-            )
```

```
44      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-77-
45      backend\pipeline\validators\blur.py:78:        return ValidationResult(
46      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-79-            validator_name=self.name,
47      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-80-            passed=True,
48      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-81-            message="Video sharpness meets focus
requirements.",
49      backend\pipeline\validators\blur.py:82:            details=details,
50      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-83-        )
51      --
52      backend\pipeline\validators\format.py:27:            return ValidationResult(
53      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-28-                validator_name=self.name,
54      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-29-                passed=False,
55      backend\pipeline\validators\format.py-30-                message=f"File not found:
{video_path}",
56      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-31-            )
57      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-32-
58      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-33-        cmd = [
59      --
60      backend\pipeline\validators\format.py:54:                return ValidationResult(
61      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-55-                    validator_name=self.name,
62      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-56-                    passed=False,
63      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-57-                    message="Failed to retrieve
valid video streams from the file.",
64      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-58-                )
65      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-59-
66      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-60-            # Store the metadata in context for
subsequent validators to utilize
67      --
68      backend\pipeline\validators\format.py:75:                return ValidationResult(
69      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-76-                    validator_name=self.name,
70      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-77-                    passed=False,
71      backend\pipeline\validators\format.py-78-                    message=f"Invalid format:
Found '{fmt.get('format_name')}', but MP4 is required.",
72      backend\pipeline\validators\format.py:79:                    details=details,
73      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-80-                )
74      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-81-
75      backend\pipeline\validators\format.py:82:            return ValidationResult(
76      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-83-                validator_name=self.name,
77      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-84-                passed=True,
78      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-85-                message="Video format is valid
MP4 container.",
79      backend\pipeline\validators\format.py:86:                details=details,
```

```
80      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-87-            )
81      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-88-
82      backend\pipeline\validators\format.py-89-        except subprocess.CalledProcessError as
e:
83      backend\pipeline\validators\format.py:90:            return ValidationResult(
84      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-91-                validator_name=self.name,
85      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-92-                passed=False,
86      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-93-                message="Video stream is
corrupted or unreadable by ffprobe.",
87      backend\pipeline\validators\format.py:94:                details={"error": e.stderr},
88      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-95-            )
89      backend\pipeline\validators\format.py-96-        except subprocess.TimeoutExpired:
90      backend\pipeline\validators\format.py:97:            return ValidationResult(
91      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-98-                validator_name=self.name,
92      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-99-                passed=False,
93      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-100-                message=f"ffprobe timed out
after {timeout_label(ffprobe_timeout_sec())}.",
94      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-101-            )
95      backend\pipeline\validators\format.py-102-        except Exception as e:
96      backend\pipeline\validators\format.py:103:            return ValidationResult(
97      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-104-                validator_name=self.name,
98      C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-105-                passed=False,
99      backend\pipeline\validators\format.py-106-                message=f"Unexpected error
running ffprobe: {str(e)}",
100     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-107-            )
101     --
102     backend\pipeline\validators\pts_continuity.py:46:                return
ValidationResult(
103     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-47-
validator_name=self.name,
104     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-48-                    passed=False,
105     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-49-                    message="No packet
timestamps could be extracted from the video stream.",
106     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-50-                )
107     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-51-
108     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-52-            timestamps = []
109     --
110     backend\pipeline\validators\pts_continuity.py:66:                return
ValidationResult(
111     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-67-
validator_name=self.name,
112     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-68-                    passed=True,
113     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-69-                    message="Video is
```

```
too short to perform a PTS continuity check.",
114     backend\pipeline\validators\pts_continuity.py:70:
details={"keyframes_count": len(timestamps)},
115     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-71-               )
116     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-72-
117     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-73-           # Analyze gaps
118     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-74-           max_gap = 0.0
119     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-75-           backward_jumps = 0
120     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-76-           gaps = []
121     --
122     backend\pipeline\validators\pts_continuity.py:98:               return
ValidationResult(
123     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-99-
validator_name=self.name,
124     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-100-                   passed=False,
125     backend\pipeline\validators\pts_continuity.py-101-                   message="Video
integrity compromise: Backward timeline jumps (splicing) detected.",
126     backend\pipeline\validators\pts_continuity.py:102:                   details=details,
127     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-103-               )
128     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-104-
129     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-105-           max_gap_threshold =
config.validation.pts_max_gap_seconds
130     backend\pipeline\validators\pts_continuity.py-106-           if max_gap >
max_gap_threshold:
131     backend\pipeline\validators\pts_continuity.py:107:               return
ValidationResult(
132     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-108-
validator_name=self.name,
133     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-109-                   passed=False,
134     backend\pipeline\validators\pts_continuity.py-110-                   message=f"Video
integrity compromise: Detected a timestamp gap of {round(max_gap, 2)}s, indicating edited or
missing content.",
135     backend\pipeline\validators\pts_continuity.py:111:                   details=details,
136     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-112-               )
137     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-113-
138     backend\pipeline\validators\pts_continuity.py:114:           return ValidationResult(
139     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-115-
validator_name=self.name,
140     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-116-               passed=True,
141     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-117-               message="PTS timeline
is continuous with no abnormal gaps or backward jumps.",
142     backend\pipeline\validators\pts_continuity.py:118:               details=details,
143     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-119-           )
144     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-120-
145     backend\pipeline\validators\pts_continuity.py-121-       except
```

```
subprocess.CalledProcessError as e:
146     backend\pipeline\validators\pts_continuity.py:122:            return ValidationResult(
147     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-123-
validator_name=self.name,
148     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-124-                passed=False,
149     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-125-                message="Failed to
execute ffprobe PTS analysis.",
150     backend\pipeline\validators\pts_continuity.py:126:                details={"error":
e.stderr},
151     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-127-            )
152     backend\pipeline\validators\pts_continuity.py-128-        except
subprocess.TimeoutExpired:
153     backend\pipeline\validators\pts_continuity.py:129:            return ValidationResult(
154     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-130-
validator_name=self.name,
155     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-131-                passed=False,
156     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-132-                message=f"ffprobe PTS
analysis timed out after {timeout_label(ffprobe_timeout_sec())}.",
157     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-133-            )
158     backend\pipeline\validators\pts_continuity.py-134-        except Exception as e:
159     backend\pipeline\validators\pts_continuity.py:135:            return ValidationResult(
160     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-136-
validator_name=self.name,
161     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-137-                passed=False,
162     backend\pipeline\validators\pts_continuity.py-138-                message=f"Unexpected
error in PTS validation: {str(e)}",
163     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-139-            )
164     --
165     backend\pipeline\validators\relevance.py:80:            return ValidationResult(
166     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-81-                validator_name=self.name,
167     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-82-                passed=False,
168     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-83-                message="Failed to open
video file for relevance validation.",
169     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-84-            )
170     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-85-
171     backend\pipeline\validators\relevance.py-86-        if meta.total_frames <= 0:
172     backend\pipeline\validators\relevance.py:87:            return ValidationResult(
173     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-88-                validator_name=self.name,
passed=False, message="Invalid frame count."
174     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-89-            )
175     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-90-
176     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-91-        green_ratios = state["green_ratios"]
177     backend\pipeline\validators\relevance.py-92-        if not green_ratios:
178     backend\pipeline\validators\relevance.py:93:            return ValidationResult(
179     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
```

```
indexer\backend\pipeline\validators\relevance.py-94-                validator_name=self.name,
180     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-95-                passed=False,
181     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-96-                message="Failed to extract
frames for relevance checking.",
182     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-97-            )
183     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-98-
184     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-99-        sport_name = config.sport
185     --
186     backend\pipeline\validators\relevance.py:112:            return ValidationResult(
187     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-113-                validator_name=self.name,
188     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-114-                passed=False,
189     backend\pipeline\validators\relevance.py-115-                message=f"Video rejected:
Relevance check failed. The scene does not contain a recognizable {sport_name} court (court area
found: {round(avg_green_ratio * 100, 2)}% vs min required {round(min_court_ratio * 100, 2)}%).",
190     backend\pipeline\validators\relevance.py:116:                details=details,
191     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-117-            )
192     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-118-
193     backend\pipeline\validators\relevance.py:119:        return ValidationResult(
194     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-120-            validator_name=self.name,
195     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-121-            passed=True,
196     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-122-            message=f"Video identified as
relevant to {sport_name} ({sport_name} court structures found).",
197     backend\pipeline\validators\relevance.py:123:            details=details,
198     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-124-        )
199     --
200     backend\pipeline\validators\resolution_fps.py:68:            return ValidationResult(
201     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-69-
validator_name=self.name,
202     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-70-                passed=False,
203     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-71-                message="Video metadata
is missing. Run format_check validator first.",
204     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-72-            )
205     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-73-
206     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-74-        stream = meta["streams"][0]
207     --
208     backend\pipeline\validators\resolution_fps.py:95:            return ValidationResult(
209     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-96-
validator_name=self.name,
210     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-97-                passed=False,
211     backend\pipeline\validators\resolution_fps.py-98-                message=f"Low
resolution: Found {width}x{height}, but minimum is {min_w}x{min_h}.",
212     backend\pipeline\validators\resolution_fps.py:99:                details=details,
213     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-100-            )
```

```
214     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-101-
215     backend\pipeline\validators\resolution_fps.py-102-        if fps < min_fps:
216     backend\pipeline\validators\resolution_fps.py:103:            return ValidationResult(
217     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-104-
validator_name=self.name,
218     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-105-                passed=False,
219     backend\pipeline\validators\resolution_fps.py-106-                message=f"Low frame
rate: Found {round(fps, 2)} FPS, but minimum is {min_fps} FPS.",
220     backend\pipeline\validators\resolution_fps.py:107:                details=details,
221     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-108-            )
222     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-109-
223     backend\pipeline\validators\resolution_fps.py:110:        return ValidationResult(
224     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-111-            validator_name=self.name,
225     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-112-            passed=True,
226     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-113-            message="Video resolution
and frame rate meet minimum standards.",
227     backend\pipeline\validators\resolution_fps.py:114:            details=details,
228     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-115-        )
229     --
230     backend\pipeline\validators\scene_cut.py:62:            return ValidationResult(
231     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-63-                validator_name=self.name,
232     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-64-                passed=False,
233     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-65-                message="Failed to open
video file using OpenCV.",
234     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-66-            )
235     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-67-
236     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-68-        fps = meta.fps
237     --
238     backend\pipeline\validators\scene_cut.py:73:            return ValidationResult(
239     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-74-                validator_name=self.name,
240     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-75-                passed=False,
241     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-76-                message="Invalid video
duration.",
242     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-77-            )
243     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-78-
244     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-79-        sample_interval = max(1, int(fps /
2))
245     --
246     backend\pipeline\validators\scene_cut.py:99:            return ValidationResult(
247     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-100-                validator_name=self.name,
248     C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-101-                passed=False,
249     backend\pipeline\validators\scene_cut.py-102-                message=f"Video splicing
```

```
compromise: Detected {cuts_detected} cuts ({round(cuts_per_minute, 2)} cuts/min), which exceeds
the continuous raw file limit of {max_cuts_rate} cuts/min.",
250     backend\pipeline\validators\scene_cut.py:103:                details=details,
251
252     [Showing results with pagination = limit: 250]
```

36. assistant (2026-07-09T06:30:31.948Z)

```
The validators each put their own numeric scores in `details` (blur?sharpness, relevance?court
ratio, scene_cut?cuts/min, etc.) ? there's no uniform `score` field. Note `[r.model_dump() for r
in val_results]` (already stored to the ephemeral DB at line 306) yields exactly
`{validator_name, passed, message, details}` ? the clean thing to also put in report.json. Let
me read the `details` dicts to design a robust compact score hint for the log line.
```

37. user (2026-07-09T06:30:32.958Z)

```
1       from typing import Any, List
2
3       import cv2
4
5       from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
6       from backend.pipeline.validators.frame_sampler import SampleMeta
7
8       # Number of frames sampled evenly across the whole video.
9       _NUM_SAMPLES = 15
10
11
12      class BlurValidator(FrameSamplingValidator):
13          @property
14          def name(self) -> str:
15              return "blur_check"
16
17          @property
18          def description(self) -> str:
19              return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
20
21          def sample_indices(self, total_frames: int, fps: float) -> List[int]:
22              # Sample _NUM_SAMPLES frames evenly distributed across the video.
23              step = max(1, total_frames // _NUM_SAMPLES)
24              return [i * step for i in range(_NUM_SAMPLES)]
25
26          def _init_state(self, config: Any) -> dict[str, Any]:
27              return {"variances": []}
28
29          def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
30              gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
31              # Calculate the Laplacian variance (standard focus metric)
32              laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
33              state["variances"].append(laplacian_var)
34
35          def _finalize(
36              self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
37          ) -> ValidationResult:
38              if not meta.opened:
39                  return ValidationResult(
40                      validator_name=self.name,
41                      passed=False,
42                      message="Failed to open video file for blur analysis.",
43                  )
44
45              if meta.total_frames <= 0:
46                  return ValidationResult(
47                      validator_name=self.name, passed=False, message="Invalid frame count."
48                  )
```

```
49
50              variances = state["variances"]
51              if not variances:
52                  return ValidationResult(
53                      validator_name=self.name,
54                      passed=False,
55                      message="Could not read frames to perform blur verification.",
56                  )
57
58              avg_variance = sum(variances) / len(variances)
59
60              details = {
61                  "avg_laplacian_variance": round(avg_variance, 2),
62                  "min_laplacian_variance": round(min(variances), 2),
63                  "max_laplacian_variance": round(max(variances), 2),
64                  "sampled_frames_count": len(variances),
65              }
66
67              val_cfg = config.validation
68              min_blur_threshold = val_cfg.min_blur_threshold
69
70              if avg_variance < min_blur_threshold:
71                  return ValidationResult(
72                      validator_name=self.name,
73                      passed=False,
74                      message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
75                      details=details,
76                  )
77
78              return ValidationResult(
79                  validator_name=self.name,
80                  passed=True,
81                  message="Video sharpness meets focus requirements.",
82                  details=details,
83              )
84
```

38. user (2026-07-09T06:30:34.784Z)

```
backend\pipeline\validators\blur.py:60:        details = {
backend\pipeline\validators\blur.py-61-            "avg_laplacian_variance": round(avg_variance,
2),
backend\pipeline\validators\blur.py-62-            "min_laplacian_variance":
round(min(variances), 2),
backend\pipeline\validators\blur.py-63-            "max_laplacian_variance":
round(max(variances), 2),
backend\pipeline\validators\blur.py-64-            "sampled_frames_count": len(variances),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-65-        }
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-66-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\blur.py-67-        val_cfg = config.validation
--
backend\pipeline\validators\pts_continuity.py:89:            details = {
backend\pipeline\validators\pts_continuity.py-90-                "keyframes_count":
len(timestamps),
backend\pipeline\validators\pts_continuity.py-91-                "max_keyframe_gap_seconds":
round(max_gap, 2),
backend\pipeline\validators\pts_continuity.py-92-                "backward_jumps_detected":
backward_jumps,
backend\pipeline\validators\pts_continuity.py-93-                "first_keyframe":
timestamps[0],
```

```
backend\pipeline\validators\pts_continuity.py-94-                "last_keyframe":
timestamps[-1],
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-95-            }
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\pts_continuity.py-96-
--
backend\pipeline\validators\relevance.py:103:        details = {
backend\pipeline\validators\relevance.py-104-            "avg_green_court_ratio":
round(avg_green_ratio, 4),
backend\pipeline\validators\relevance.py-105-            "max_green_court_ratio":
round(max(green_ratios), 4),
backend\pipeline\validators\relevance.py-106-            "min_green_court_ratio":
round(min(green_ratios), 4),
backend\pipeline\validators\relevance.py-107-            "green_threshold": min_court_ratio,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-108-        }
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-109-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\relevance.py-110-        # If a certain amount of court
color is visible, consider it relevant
--
backend\pipeline\validators\format.py:66:            details = {
backend\pipeline\validators\format.py-67-                "format_name": fmt.get("format_name"),
backend\pipeline\validators\format.py-68-                "duration": float(fmt.get("duration",
0)),
backend\pipeline\validators\format.py-69-                "size_bytes": int(fmt.get("size", 0)),
backend\pipeline\validators\format.py-70-                "codec_name": stream.get("codec_name"),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-71-            }
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-72-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\format.py-73-            # Check if container name includes
mp4
--
backend\pipeline\validators\base.py:30:    details: dict[str, Any] = {}
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-31-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-32-
backend\pipeline\validators\base.py-33-class VideoValidator(ABC):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-34-    @property
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-35-    @abstractmethod
backend\pipeline\validators\base.py-36-    def name(self) -> str:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\base.py-37-        """Unique identifier for this
validator."""
--
backend\pipeline\validators\resolution_fps.py:86:        details = {"width": width, "height":
height, "fps": round(fps, 2)}
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-87-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-88-        # Check bounds from typed
config (AppConfig.validation)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-89-        val_cfg = config.validation
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-90-        min_w =
val_cfg.min_resolution_width
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
```

```
indexer\backend\pipeline\validators\resolution_fps.py-91-        min_h =
val_cfg.min_resolution_height
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-92-        min_fps = val_cfg.min_fps
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py-93-
--
backend\pipeline\validators\scene_cut.py:89:        details = {
backend\pipeline\validators\scene_cut.py-90-            "cuts_detected": cuts_detected,
backend\pipeline\validators\scene_cut.py-91-            "sampled_frames": sampled_count,
backend\pipeline\validators\scene_cut.py-92-            "cuts_per_minute":
round(cuts_per_minute, 3),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-93-        }
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-94-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-95-        val_cfg = config.validation
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\scene_cut.py-96-        max_cuts_rate =
val_cfg.max_scene_cuts_per_minute
```

39. assistant (2026-07-09T06:32:08.697Z)

Now I have the full `details` picture ? each validator has its own metric keys (blur?`avg_laplacian_variance`, relevance?`avg_green_court_ratio`, scene_cut?`cuts_per_minute`, etc.), no uniform `score`. For report.json I'll persist `[r.model_dump() ?]` (same list already going to the DB ? carries `details` with all scores); for the log line I'll derive a compact score hint from known keys.

Let me look at the test infrastructure: the module-load-by-path pattern to mirror, and any existing tests for the worker infer path and the native runner's progress streaming (added by PR #546).

40. user (2026-07-09T06:32:10.700Z)

```
1       """Tests for scripts/gates/quality/nightly_regression.py ? the pure
summary/comparison/report core.
2
3       Fully offline: no real golden corpus, no trajectory I/O, no GPU/numpy. The comparison
core
4       operates on plain summary dicts, so a regression / over-seg-guard / corpus-change can be
5       exercised deterministically. The real-corpus scoring path (``run_corpus``) is a thin
wrapper
6       over ``rally_seg_eval.evaluate`` (tested in test_rally_seg_eval.py) and is exercised e2e
on the
7       owner's machine, not here.
8       """
9
10      import importlib.util
11      import os
12      import sys
13
14      import pytest
15
16      # scripts/ isn't a package ? load the module by path. Register in sys.modules BEFORE
exec so
17      # dataclasses can resolve the `from __future__ import annotations` string annotations.
18      _SCRIPT = os.path.join(
19          os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
20          "scripts",
21          "gates",
22          "quality",
23          "nightly_regression.py",
24      )
```

```
    _spec = importlib.util.spec_from_file_location("nightly_regression", _SCRIPT)
    nr = importlib.util.module_from_spec(_spec)
    sys.modules["nightly_regression"] = nr
    _spec.loader.exec_module(nr)


    # ---------------------------------------------------------------------- #
    # Builders
    # ---------------------------------------------------------------------- #
    _PRESET = {
        "name": "x",
        "velocity_thresh": 0.02,
        "merge_gap": 2.0,
        "min_window_duration": 2.0,
        "max_window_duration": 0.0,
        "hard_cap": False,
        "inactivity_end": 0.0,
        "inactivity_rally_thresh": 0.08,
        "inactivity_min_density": 0.34,
    }


    def _summary(aggregate, per_video, preset=None, min_crossings=0, n=None):
        return {
            "n": n if n is not None else len(per_video),
            "preset": preset or dict(_PRESET),
            "min_crossings": min_crossings,
            "iou": 0.5,
            "aggregate": dict(aggregate),
            "per_video": {k: dict(v) for k, v in per_video.items()},
        }


    def _pv(tol_f1=0.3, split_count=2, merge_count=0, **extra):
        d = {
            "tol_f1": tol_f1,
            "f1@0.5": 0.4,
            "f1@0.25": 0.5,
            "merge_rate": 0.1,
            "split_count": split_count,
            "merge_count": merge_count,
            "num_pred": 10,
            "num_gt": 10,
        }
        d.update(extra)
        return d
```

41. user (2026-07-09T06:32:10.891Z)

```
tests\__init__.py
tests\test_hashing.py
tests\test_import_smoke.py
tests\test_reliability_hardening.py
tests\test_annotation_providers.py
tests\test_app_home.py
tests\test_automated_provider.py
tests\test_base.py
tests\test_bind_and_host.py
tests\test_calibrate_wasb.py
tests\test_calibration.py
tests\test_cleanup_caches.py
tests\test_court_gate.py
tests\test_db_upgrade_safety.py
tests\test_detector_keying.py
tests\test_device_context.py
```

tests\test_distill_local.py
tests\test_edge_auth.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_partial_labels.py
tests\test_eval_regression.py
tests\test_eval_stats.py
tests\test_eval_training.py
tests\test_experiment.py
tests\test_experiment_honest.py
tests\test_export_wasb_segments.py
tests\test_flywheel.py
tests\test_fusion_features.py
tests\test_fusion_hybrid.py
tests\test_gemini.py
tests\test_gemini_refine.py
tests\test_generate_highlights.py
tests\test_geometry.py
tests\test_golden_fixtures.py
tests\test_golden_fixtures_split.py
tests\test_golden_real_fixtures.py
tests\test_gt_loader.py
tests\test_hit_detector.py
tests\test_job_self_scheduling.py
tests\test_job_worker_module_seam.py
tests\test_leak_scan.py
tests\test_light_tier_no_torch.py
tests\test_logging_setup.py
tests\test_motion_segmenter.py
tests\test_output_base.py
tests\test_p2b_install.py
tests\test_per_user_eval.py
tests\test_per_user_gate.py
tests\test_per_user_regression.py
tests\test_person_detector.py
tests\test_personalization_serving.py
tests\test_profile_parity.py
tests\test_promotion.py
tests\test_providers.py
tests\test_provider_gemini.py
tests\test_rally_seg_eval_report.py
tests\test_rally_seq_proto.py
tests\test_rally_slicer.py
tests\test_rally_summary.py
tests\test_redact.py
tests\test_reingest_safety.py
tests\test_reporting.py
tests\test_reporting_harvest.py
tests\test_reporting_spec_faq.py
tests\test_results.py
tests\test_run_signals.py
tests\test_run_telemetry.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
tests\test_server_entrypoint.py
tests\test_serve_contrast.py
tests\test_single_instance.py
tests\test_splits.py
tests\test_sport_profiles.py
tests\test_stub_runner.py
tests\test_tolerance_metrics.py
tests\test_training_gen0.py

```
tests\test_validators_base.py
tests\test_variant_tuning.py
tests\test_watermark_i18n.py
tests\test_workspace.py
tests\test_wsl_tilde_quoting.py
tests\test_rally_gate.py
tests\test_rally_seg_eval.py
tests\test_windowing.py
tests\test_check_md_links.py
tests\test_freespace.py
tests\test_storage_capabilities.py
tests\test_tracknet_runner.py
tests\test_wasb_runner.py
tests\test_api_annotations.py
tests\test_annotation_proxy.py
tests\test_ffmpeg_resolver.py
tests\test_ffmpeg_timeouts.py
tests\test_video.py
tests\test_api_assets.py
(Showing 100 of 153 matching files; 53 more are not listed. Narrow the pattern or path to see
the rest.)
```

42. assistant (2026-07-09T06:32:21.143Z)

Let me find the existing tests that touch the worker infer path, the native runner, and the #534 progress/telemetry work, so I extend rather than duplicate.

43. user (2026-07-09T06:32:24.046Z)

```
tests\test_native_wasb_runner.py
```

44. user (2026-07-09T06:32:25.294Z)

```
Found 17 files
tests\test_worker.py
tests\test_native_wasb_runner.py
tests\test_config.py
tests\test_compute_backend.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py
tests\test_startup_checks.py
tests\test_fusion_golden_telemetry.py
tests\test_wasb_runner.py
tests\test_tracknet_runner.py
tests\test_wsl_tilde_quoting.py
tests\test_stub_runner.py
tests\test_run_telemetry.py
tests\test_profile_parity.py
tests\test_output_base.py
tests\test_fusion_hybrid.py
tests\test_detector_keying.py
```

45. assistant (2026-07-09T06:32:35.450Z)

Let me read the two most relevant existing test files to match their mocking patterns.

46. user (2026-07-09T06:32:36.272Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
```

```
    """

    import json
    import os

    import pytest

    from backend.config import load_config
    from backend.results import Failure
    from backend.worker import __main__ as wmain

    # ---------------------------------------------------------------- pure helpers


    @pytest.mark.parametrize(
        "raw,expected",
        [
            ("true", True),
            ("False", False),
            ("3", 3),
            ("2.5", 2.5),
            ("wasb_hybrid", "wasb_hybrid"),
        ],
    )
    def test_coerce(raw, expected):
        assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


    def test_apply_overrides_on_typed_config():
        cfg = _fresh_config()
        wmain._apply_overrides(
            cfg,
            [
                "indexing.skip_ai_handoff=true",
                "indexing.min_segment_duration=1.5",
                "sport=tennis",
            ],
        )
        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.indexing.min_segment_duration == 1.5
        assert cfg.sport == "tennis"


    def test_apply_overrides_rejects_bad_format():
        cfg = _fresh_config()
        with pytest.raises(ValueError):
            wmain._apply_overrides(cfg, ["no_equals_sign"])


    def test_write_intervals_csv(tmp_path):
        segs = [
            {
                "start_time": 2.0,
                "end_time": 5.0,
                "shot_count": 4,
                "shot_count_confidence": "LocalNoAI",
            },
            {"start_time": 9.0, "end_time": 12.5},
        ]
        out = tmp_path / "intervals.csv"
        wmain._write_intervals_csv(segs, str(out))
        lines = out.read_text().splitlines()
        assert lines[0] == "start,end,shot_count,ending_reason"
        assert lines[1].startswith("2.000,5.000,4,")
        assert lines[2].startswith("9.000,12.500,,")
```

```
71
72
73      def test_write_report_json(tmp_path):
74          out = tmp_path / "report.json"
75          wmain._write_report_json(
76              "vid1", [{"start_time": 1.0, "end_time": 2.0}], "success", str(out)
77          )
78          data = json.loads(out.read_text())
79          assert data["video_id"] == "vid1" and data["segment_count"] == 1
80          assert data["segments"][0] == {"start": 1.0, "end": 2.0}
81
82
83      def test_parser_infer_accepts_set_and_uris():
84          args = wmain.build_parser().parse_args(
85              [
86                  "infer",
87                  "--video-uri",
88                  "s3://b/v.mp4",
89                  "--out-uri",
90                  "s3://b",
91                  "--set",
92                  "indexing.skip_ai_handoff=true",
93                  "--set",
94                  "sport=tennis",
95              ]
96          )
97          assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
98          assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
99
100
101     def test_parser_infer_skip_validation_flag():
102         base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
103         # Default-OFF: the direct `worker infer` CLI still validates.
104         assert wmain.build_parser().parse_args(base).skip_validation is False
105         # The dispatcher passes it to disable the redundant worker re-validation.
106         assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation
is True
107
108
109     # ------------------------------------------------------------- cmd_infer (local,
mocked)
110
111
112     class _OKResult:
113         passed = True
114         validator_name = "fake"
115         message = "ok"
116
117         def model_dump(self):
118             return {"validator_name": "fake", "passed": True, "message": "ok"}
119
120
121     class _FailResult(_OKResult):
122         passed = False
123         message = "too blurry"
124
125
126     class _FakeSeg:
127         def __init__(self, db, config):
128             pass
129
130         def process_video(self, video_id, path, max_frames=None, progress=None):
131             # progress accepted for #534 live heartbeat compat (ignored in fake)
132             return [
133                 {
```

```
                    "start_time": 2.0,
                    "end_time": 5.0,
                    "shot_count": 4,
                    "shot_count_confidence": "LocalNoAI",
                },
                {
                    "start_time": 9.0,
                    "end_time": 12.0,
                    "shot_count": 6,
                    "shot_count_confidence": "LocalNoAI",
                },
            ]


    def _fresh_config():
        # Load defaults without touching any cwd config.json.
        import tempfile

        p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
        p.write("{}")
        p.close()
        return load_config(p.name)


    @pytest.fixture
    def mocked_pipeline(monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)


    def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        out = tmp_path / "bucket"
        scratch = tmp_path / "scratch"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
                "--segmenter",
                "wasb_hybrid",
                "--set",
                "indexing.skip_ai_handoff=true",
            ]
        )
        assert rc == 0
        intervals = out / "outputs" / "vid123" / "intervals.csv"
        report = out / "outputs" / "vid123" / "report.json"
        assert intervals.exists() and report.exists()
```

```
        rows = intervals.read_text().splitlines()
        assert len(rows) == 3  # header + 2 rallies
        assert json.loads(report.read_text())["segment_count"] == 2


    class _EmptySeg:
        """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None, progress=None):
            # progress accepted for #534 (ignored in fake)
            return []


    class _FailSeg:
        """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
        trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None, progress=None):
            # progress accepted for #534 (ignored in fake)
            return Failure(
                message="WASB detector produced no trajectory (timed out or aborted)",
                is_recoverable=True,
            )


    def test_setup_logging_levels():
        import logging

        wmain._setup_logging(False)
        assert logging.getLogger().level == logging.INFO
        wmain._setup_logging(True)
        assert logging.getLogger().level == logging.DEBUG


    def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
        # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"

        # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
        # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
        args = wmain.build_parser().parse_args(
            [
                "infer",
                "--video-uri",
```

```
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        with caplog.at_level("WARNING", logger="backend.worker"):
            rc = wmain.cmd_infer(args)
        assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
        msgs = " ".join(r.getMessage() for r in caplog.records)
        assert "0 segments" in msgs and "vidZ" in msgs
        assert "detector_impl=native" in msgs  # surfaces the resolved detector
        assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
        # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
        # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
        report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
        assert report["segment_count"] == 0 and report["status"] == "success"


    def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
        tmp_path, monkeypatch
    ):
        """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
        as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
        status="failed" so the control plane / alpha-user surface can show an error /
retry."""
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"
        done = tmp_path / "bucket" / "_dispatch" / "tok.done"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                str(done),
            ]
        )
        # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
        assert rc == 3
        # report.json is written as an explicit FAILURE, not a false empty "success".
        report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
```

```
        assert report["status"] == "failed" and report["segment_count"] == 0
        assert report["segments"] == []
        # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 3 and marker["status"] == "failed"
        assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"


    def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
monkeypatch):
        """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
flows
        through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
the two
        halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
_FailSeg stub."""
        from unittest.mock import MagicMock

        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

        runner = MagicMock()
        runner.healthcheck.return_value = (True, "env ok")
        runner.run_predict.return_value = None  # WASB timed out / aborted: no trajectory
produced

        # Force the real base process_video down to its no-trajectory abort without a
GPU/video.
        monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
(runner, {}))
        monkeypatch.setattr(
            WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
        )
        monkeypatch.setattr(
            "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
30.0
        )
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
        monkeypatch.setattr(
            wmain.validator_registry, "run_all_with_context", lambda path, cfg:
([_OKResult()], {})
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)

        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"
        done = tmp_path / "bucket" / "_dispatch" / "tok.done"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                str(done),
                "--set",
```

```
                "indexing.skip_ai_handoff=true",  # no API key needed; isolate the detector abort
            ]
        )
        assert rc == 3
        report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 3 and marker["status"] == "failed"


    def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_FailResult()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"X")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        out = tmp_path / "o"
        done = tmp_path / "o" / "_dispatch" / "tok.done"
        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--config",
                str(cfg),
                "--scratch",
                str(tmp_path / "s"),
                "--done-uri",
                str(done),
            ]
        )
        assert rc == 2
        # A validation failure is ALSO written as an explicit failure (not a missing artifact / false
        # success): report.json status="failed" + the done-marker rc 2 / status="failed".
        report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 2 and marker["status"] == "failed"


    def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
        """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the worker's
        validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
        control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
        run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with a
        status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc 2)."""
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")

        def _boom(*a, **k):  # the validators must never be reached on the skip path
            raise AssertionError("run_all_with_context must not run under --skip-validation")
```

```
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--skip-validation",
            ]
        )
        assert rc == 0  # a clip that WOULD fail re-validation still succeeds ? the gate is
trusted
        report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
        assert report["status"] == "success" and report["segment_count"] == 2


    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---


    @pytest.mark.parametrize(
        "fname,expected",
        [
            (
                "2026-06-22_GX020094.rallies.csv",
                "GX020094",
            ),  # canonical dated label (DATA_IN_GCS §7.1)
            ("GX020094.rallies.csv", "GX020094"),  # legacy bare label -> SAME clip stem
            (
                "GX020094_proxy.rallies.csv",
                "GX020094_proxy",
            ),  # _proxy is part of the name, kept
            (
                "2026-06-21_mahadevpura_1.rallies.csv",
                "mahadevpura_1",
            ),  # underscores in <name> survive
            ("v1.csv", "v1"),  # bare .csv suffix
            ("2026-06-22_GX020094.csv", "GX020094"),  # dated + bare .csv
        ],
    )
    def test_label_clip_name_strips_date_prefix(fname, expected):
        """A dated label and a bare label must both resolve to the bare videos/<name>
stem."""
        assert wmain._label_clip_name(fname) == expected


    def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
        """Regression (DATA_IN_GCS §7b blocker #1): canonical labels carry an ISO-date
prefix
        (labels/<date>_<name>.rallies.csv) while videos are bare (videos/<name>.mp4). The
label-derived
        clip name must strip the date so the video join succeeds ? otherwise every clip is
skipped and
```

```
        train aborts with <2 videos."""
        import os
        import subprocess

        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # CPU stub runner (no GPU/WSL)
        monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        # canonical DATED labels ...
        (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(
            header + "1,2.0,5.0,winner,badminton,\n"
        )
        (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(
            header + "1,3.0,6.0,winner,badminton,\n"
        )
        # ... joining to BARE-stem videos
        (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
        (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)

        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
                str(corpus),
                "--out-uri",
                str(out_dir),
                "--version",
                "0.0.4",
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
                "--trajectory-source",
                "detector",
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        assert rc == 0  # before the fix: every clip skipped -> rc 2 ("<2 videos")
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2  # both dated labels joined
        assert {s["name"] for s in specs} == {
            "GX020094",
            "mahadevpura_1",
        }  # downstream name is date-stripped
```

```


    def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
        """A dated label whose <name> has no videos/<name>.<ext> is SKIPPED (the join-miss
branch),
        while the labels that DO join are kept ? so one orphan label can't abort an
otherwise-valid run."""
        import os
        import subprocess

        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
        monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        row = "1,2.0,5.0,winner,badminton,\n"
        (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row)
        (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header + row)
        (labels_dir / "2026-06-20_orphan.rallies.csv").write_text(
            header + row
        )  # NO videos/orphan.*
        (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
        (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)

        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
                str(corpus),
                "--out-uri",
                str(out_dir),
                "--version",
                "0.0.5",
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
                "--trajectory-source",
                "detector",
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        assert (
            rc == 0
        )  # the orphan label was skipped; the two joinable clips carried the run
```

```
        specs = json.loads((scratch / "manifest.json").read_text())
        assert {s["name"] for s in specs} == {
            "GX020094",
            "mahadevpura_1",
        }  # 'orphan' absent


    def test_cmd_train_detector_skips_unprobeable_and_trajless_videos(
        tmp_path, monkeypatch
    ):
        """cmd_train's detector loop must SKIP (never crash, never guess) a video whose
fps/width won't
        probe and a video the detector yields no trajectory for, keeping the healthy
clips."""
        import os
        import subprocess

        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        row = "1,2.0,5.0,winner,badminton,\n"
        for stem in ("good1", "good2", "badfps", "notraj"):
            (labels_dir / f"2026-06-22_{stem}.rallies.csv").write_text(header + row)
            (videos_dir / f"{stem}.mp4").write_bytes(b"FAKE-" + stem.encode())

        # probe fails ONLY for badfps -> that clip is skipped (a wrong fps mis-aligns every
label).
        monkeypatch.setattr(
            wmain,
            "_probe_video_fps_width",
            lambda p: None if "badfps" in os.path.basename(p) else (30.0, 1280.0),
        )
        monkeypatch.setattr(
            wmain, "compute_video_id", lambda p: "id-" + os.path.basename(p)
        )

        class _FakeRunner:
            # writes a real trajectory for everything EXCEPT notraj (empty result ->
skipped).
            def run_predict(self, video, out_dir, video_id=None):
                if "notraj" in os.path.basename(video):
                    return ""
                traj = os.path.join(out_dir, f"{video_id}_traj.csv")
                with open(traj, "w") as f:
                    f.write("frame,x,y\n0,1,2\n")
                return traj

        monkeypatch.setattr(
            wmain, "_resolve_train_detector", lambda args, config: _FakeRunner()
        )

        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
```

```
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)

        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
                str(corpus),
                "--out-uri",
                str(out_dir),
                "--version",
                "0.0.6",
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
                "--trajectory-source",
                "detector",
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        assert rc == 0
        specs = json.loads((scratch / "manifest.json").read_text())
        assert {s["name"] for s in specs} == {"good1", "good2"}  # badfps + notraj skipped


    def test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join(
        tmp_path, monkeypatch
    ):
        """When labels join to <2 videos, train aborts cleanly with rc 2 (leave-one-video-
out needs >=2).
        This is the guard the date-prefix bug used to trip for EVERY clip (DATA_IN_GCS §7b
#1); here it
        fires legitimately because only one of two labels has a matching video."""
        import subprocess

        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
        monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        row = "1,2.0,5.0,winner,badminton,\n"
        (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row)  # joins
        (labels_dir / "2026-06-21_orphan.rallies.csv").write_text(
            header + row
        )  # no video -> skipped
        (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        # subprocess.run must NOT be reached (we abort before the harness); guard that.
        monkeypatch.setattr(
            subprocess, "run", lambda *a, **k: pytest.fail("harness ran despite <2 videos")
        )

        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
```

```
                str(corpus),
                "--out-uri",
                str(tmp_path / "out"),
                "--version",
                "0.0.7",
                "--config",
                str(cfg),
                "--scratch",
                str(tmp_path / "s"),
                "--trajectory-source",
                "detector",
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        assert rc == 2  # only 1 of 2 labels joined -> below the leave-one-video-out floor


    def test_cmd_train_orchestration(tmp_path, monkeypatch):
        import os
        import subprocess

        labels_dir = tmp_path / "corpus" / "labels"
        labels_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(
            header + "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n"
        )
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)

        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
                str(tmp_path / "corpus"),
                "--out-uri",
                str(out_dir),
                "--version",
                "0.0.1",
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
            ]
        )
        assert rc == 0
        # bundle pushed to <out-uri>/weights/<version>/
        assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
```

```
        assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
        # the manifest fed to the harness enumerated both labeled videos with a trajectory
each
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2
        assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


    def test_parser_train_detector_args():
        args = wmain.build_parser().parse_args(
            [
                "train",
                "--corpus-uri",
                "gcs://b",
                "--out-uri",
                "gcs://b",
                "--version",
                "1.0.0",
                "--trajectory-source",
                "detector",
                "--segmenter",
                "fusion_hybrid",
            ]
        )
        assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"


    def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
        """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
With
        RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
runner)."""
        import os
        import subprocess

        monkeypatch.setenv(
            "RALLY_DETECTOR_IMPL", "stub"
        )  # resolve a CPU stub runner (no GPU/WSL)
        # Fake bytes aren't decodable, so stub the (strict, fail-loud) ffprobe probe to a
real value.
        monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
        (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
        (videos_dir / "v1.srt").write_text(
            "1\n00:00:00,000 --> 00:00:01,000\nx\n"
        )  # sidecar: must NOT shadow v1.mp4
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
```

```
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)

        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
                str(corpus),
                "--out-uri",
                str(out_dir),
                "--version",
                "0.0.2",
                "--config",
                str(cfg),
                "--scratch",
                str(scratch),
                "--trajectory-source",
                "detector",
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        assert rc == 0
        specs = json.loads((scratch / "manifest.json").read_text())
        assert (
            len(specs) == 2
        )  # the v1.srt sidecar did NOT shadow v1.mp4 (extension filter)
        # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
        assert all(os.path.exists(s["trajectory"]) for s in specs)
        assert all(
            s["trajectory"].endswith("_stub.csv") for s in specs
        )  # the stub runner's output


    def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(
        tmp_path, monkeypatch
    ):
        """detector_impl=native for a family with no native runner must return rc 2 (clean),
not
        propagate a raw NotImplementedError traceback out of the worker."""
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        (labels_dir).mkdir(parents=True)
        (corpus / "videos").mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        rc = wmain.main(
            [
                "train",
                "--corpus-uri",
                str(corpus),
                "--out-uri",
                str(tmp_path / "out"),
                "--version",
                "0.0.3",
                "--config",
                str(cfg),
```

```
                "--scratch",
                str(tmp_path / "s"),
                "--trajectory-source",
                "detector",
                "--segmenter",
                "tracknet_hybrid",
            ]
        )
        assert (
            rc == 2
        )  # graceful: _resolve_train_detector caught NotImplementedError, returned None


    def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
        """The strict ffprobe probe must return None (caller skips) for a non-video file,
never guess."""
        bad = tmp_path / "not_a_video.mp4"
        bad.write_bytes(b"NOTAVIDEO")
        assert wmain._probe_video_fps_width(str(bad)) is None


    # ------------------------------------------------------------ compile subcommand (#521 ?
L4 stitch)


    def test_parser_compile_accepts_spec_and_done_uris():
        args = wmain.build_parser().parse_args(
            [
                "compile",
                "--spec-uri",
                "gs://b/_compile/tok/spec.json",
                "--out-uri",
                "gs://b",
                "--done-uri",
                "gs://b/_compile/tok.done",
                "--set",
                "storage.output_root=gs://b",
            ]
        )
        assert args.cmd == "compile"
        assert args.spec_uri == "gs://b/_compile/tok/spec.json"
        assert args.done_uri == "gs://b/_compile/tok.done"
        assert args.set == ["storage.output_root=gs://b"]


    def _write_compile_spec(
        bucket, video_id, source, name, encoder="h264_nvenc", rendition="original"
    ):
        spec = {
            "video_id": video_id,
            "video_uri": str(source),
            "output_name": name,
            "rendition": rendition,
            "locale": None,
            "time_pairs": [[2.0, 6.0], [10.0, 14.0]],
            "stitching": {"encoder": encoder, "watermark": {"enabled": False}},
        }
        spec_dir = os.path.join(str(bucket), "_compile", "tok")
        os.makedirs(spec_dir, exist_ok=True)
        spec_uri = os.path.join(spec_dir, "spec.json")
        with open(spec_uri, "w", encoding="utf-8") as f:
            json.dump(spec, f)
        return spec_uri


```

```
    def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
        """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core (VideoCompiler stubbed so
        no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's bucket-poll."""
        import backend.orchestration as orch

        bucket = tmp_path / "bucket"
        bucket.mkdir()
        source = tmp_path / "src.mp4"
        source.write_bytes(b"SRC")
        spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
        done = str(bucket / "_compile" / "tok.done")

        seen = {}

        def _fake_stitch(
            cfg,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
            locale,
            notify,
            rendition="original",
            require_mirror=False,
            **kw,
        ):
            seen["encoder"] = stitching.encoder
            seen["pairs"] = time_pairs
            seen["source"] = source_ref
            seen["rendition"] = rendition
            seen["require_mirror"] = require_mirror
            notify("stitching")
            return {
                "status": "success",
                "video_id": video_id,
                "download_url": f"/api/highlights/{video_id}/{out_name}",
                "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
                "filename": out_name,
                "rendition": rendition,
            }

        monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)

        cfg = tmp_path / "c.json"
        cfg.write_text('{"storage": {"backend": "local"}}')
        rc = wmain.cmd_compile(
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--done-uri",
                    done,
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--set",
                    f"storage.output_root={bucket}",
                ]
```

```
            )
        )
        assert rc == 0
        # the shared stitch core got the resolved pairs + the NVENC encoder from the spec
        assert seen["encoder"] == "h264_nvenc"
        assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0)]
        assert seen["source"] == str(source)
        assert seen["require_mirror"] is True
        assert seen["rendition"] == "original"
        marker = json.loads(open(done, encoding="utf-8").read())
        assert marker["returncode"] == 0 and marker["status"] == "success"
        assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"
        assert marker["filename"] == "reel.mp4"
        assert marker["rendition"] == "original"


    def test_cmd_compile_sd_rendition_reaches_stitch_and_marker(tmp_path, monkeypatch):
        """The worker honors the staged SD rendition directly; it does not build an original
reel first."""
        import backend.orchestration as orch

        bucket = tmp_path / "bucket"
        bucket.mkdir()
        source = tmp_path / "src.mp4"
        source.write_bytes(b"SRC")
        spec_uri = _write_compile_spec(
            bucket, "vidReel", source, "reel-sd.mp4", rendition="sd"
        )
        done = str(bucket / "_compile" / "tok.done")
        seen = {}

        def _fake_stitch(*args, **kwargs):
            seen["rendition"] = kwargs.get("rendition") or (args[-1] if args else
"original")
            return {
                "status": "success",
                "video_id": "vidReel",
                "download_url": "/api/highlights/vidReel/reel-sd.mp4",
                "reel_uri": "gs://b/highlights/vidReel/reel-sd.mp4",
                "filename": "reel-sd.mp4",
                "rendition": args[-1],
            }

        monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)

        cfg = tmp_path / "c.json"
        cfg.write_text('{"storage": {"backend": "local"}}')
        rc = wmain.cmd_compile(
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--done-uri",
                    done,
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                ]
            )
        )

```

```
        assert rc == 0
        assert seen["rendition"] == "sd"
        marker = json.loads(open(done, encoding="utf-8").read())
        assert marker["filename"] == "reel-sd.mp4"
        assert marker["rendition"] == "sd"


    def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
        """A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the dispatcher's
poll
        terminates and the SPA sees a clean failure."""
        import backend.orchestration as orch

        bucket = tmp_path / "bucket"
        bucket.mkdir()
        source = tmp_path / "src.mp4"
        source.write_bytes(b"SRC")
        spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
        done = str(bucket / "_compile" / "tok.done")

        monkeypatch.setattr(
            orch,
            "_stitch_reel",
            lambda *a, **k: {"status": "failed", "message": "ffmpeg blew up", "video_id":
"vidReel"},
        )
        cfg = tmp_path / "c.json"
        cfg.write_text('{"storage": {"backend": "local"}}')
        rc = wmain.cmd_compile(
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--done-uri",
                    done,
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                ]
            )
        )
        assert rc == 3
        marker = json.loads(open(done, encoding="utf-8").read())
        assert marker["returncode"] == 3 and marker["status"] == "failed"


    def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
        """DEFAULT-OFF: a direct-CLI stitch with no --done-uri writes no marker."""
        import backend.orchestration as orch

        bucket = tmp_path / "bucket"
        bucket.mkdir()
        source = tmp_path / "src.mp4"
        source.write_bytes(b"SRC")
        spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")

        monkeypatch.setattr(
            orch, "_stitch_reel", lambda *a, **k: {"status": "success", "video_id":
"vidReel"}
        )
        cfg = tmp_path / "c.json"
```

```
        cfg.write_text('{"storage": {"backend": "local"}}')
        rc = wmain.cmd_compile(
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--config",
                    str(cfg),
                ]
            )
        )
        assert rc == 0
        assert not (bucket / "_compile" / "tok.done").exists()


    def test_cmd_compile_fails_when_reel_upload_fails(tmp_path, monkeypatch):
        """Review finding (silent data loss): drive cmd_compile through the REAL
_stitch_reel (VideoCompiler
        stubbed so no ffmpeg) with a failing reel mirror-upload. The worker has no local
fallback, so it MUST
        return rc 3 + a FAILED marker ? never a 'success' marker for a reel the SPA can't
download (404)."""
        import backend.main as bmain

        bucket = tmp_path / "bucket"
        bucket.mkdir()
        source = tmp_path / "src.mp4"
        source.write_bytes(b"SRC")
        # output_root is a BUCKET URI so reel_object_uri resolves + the mirror is attempted
(then fails)
        spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
        done = str(bucket / "_compile" / "tok.done")

        class _FakeCompiler:
            def __init__(self, output_dir, *a, **k):
                self.output_dir = output_dir
                os.makedirs(output_dir, exist_ok=True)

            def compile_highlights(self, src, pairs, name):
                p = os.path.join(self.output_dir, os.path.basename(name))
                with open(p, "wb") as f:
                    f.write(b"REELBYTES")
                return p

        monkeypatch.setattr(bmain, "VideoCompiler", _FakeCompiler)
        monkeypatch.setattr(wmain.storage, "acquire_local_input", lambda ref, sc:
str(source))
        monkeypatch.setattr(wmain.storage, "cleanup_acquired_local_input", lambda *a, **k:
None)

        def _put(src_path, uri, **k):
            # the reel mirror to the bucket fails; the tiny marker put (a different uri)
still succeeds
            if uri.endswith("reel.mp4"):
                raise RuntimeError("gcs 503 mid-upload")
            # marker + any other put: copy locally so the marker is written
            import shutil

            os.makedirs(os.path.dirname(uri), exist_ok=True)
            shutil.copy(src_path, uri)
```

```
            return uri

        monkeypatch.setattr(wmain.storage, "put", _put)

        cfg = tmp_path / "c.json"
        cfg.write_text('{"storage": {"backend": "local"}}')
        rc = wmain.cmd_compile(
            wmain.build_parser().parse_args(
                [
                    "compile",
                    "--spec-uri",
                    spec_uri,
                    "--out-uri",
                    str(bucket),
                    "--done-uri",
                    done,
                    "--config",
                    str(cfg),
                    "--scratch",
                    str(tmp_path / "s"),
                    "--set",
                    "storage.output_root=gs://testbucket525",
                ]
            )
        )
        assert rc == 3  # NOT a false success
        marker = json.loads(open(done, encoding="utf-8").read())
        assert marker["returncode"] == 3 and marker["status"] == "failed"
        assert "upload" in (marker.get("message") or "").lower()


    def test_cmd_infer_respects_detector_progress_sample_config(tmp_path, monkeypatch):
        """#534: worker cmd_infer reads the configurable sampling value and the path
        remains stable when sampling is set via --set (no crash, config reaches the
closure)."""
        class _NoopSeg:
            def __init__(self, db, config):
                pass
            def process_video(self, video_id, path, max_frames=None, progress=None):
                if progress:
                    progress("detector 50% (2500/5000)")
                return [{"start_time": 0.0, "end_time": 1.0}]

        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _NoopSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda p, c: ([], {}),
        )

        # Track that storage.put is reachable (we don't assert count here to keep test
stable)
        called = []
        monkeypatch.setattr(wmain.storage, "put", lambda *a, **k: called.append(1))

        video = tmp_path / "in.mp4"
        video.write_bytes(b"fake")
        out = tmp_path / "bucket"

        # Explicit small sample value must be accepted by the run
        rc = wmain.main(
            [
                "infer",
                "--video-uri",
```

```
1304                str(video),
1305                "--out-uri",
1306                str(out),
1307                "--scratch",
1308                str(tmp_path / "s"),
1309                "--segmenter",
1310                "wasb_hybrid",
1311                "--skip-validation",
1312                "--set",
1313                "indexing.detector_progress_sample_sec=0.1",
1314            ]
1315        )
1316        assert rc == 0
1317        # At least the final serialize push happened (or the progress one)
1318        assert len(called) >= 0  # non-crashing is the main assertion here
1319
1320        # Large value also accepted
1321        rc = wmain.main(
1322            [
1323                "infer",
1324                "--video-uri",
1325                str(video),
1326                "--out-uri",
1327                str(out),
1328                "--scratch",
1329                str(tmp_path / "s2"),
1330                "--segmenter",
1331                "wasb_hybrid",
1332                "--skip-validation",
1333                "--set",
1334                "indexing.detector_progress_sample_sec=999",
1335            ]
1336        )
1337        assert rc == 0
1338
```

47. assistant (2026-07-09T06:33:10.595Z)

`test_worker.py` gives me the exact mocking patterns (and confirms the fakes lack a `details` attr, so my score-hint must use `getattr(r, "details", {})`). Now let me read the native runner test to see how the #546 progress-streaming path is currently tested.

48. user (2026-07-09T06:33:11.986Z)

```
1       """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3       The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4       the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5       (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6       activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7       name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8       """
9
10      import logging
11      import os
12      import types
13      from unittest.mock import patch
14
15      import pytest
16
17      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin
18      from backend.pipeline.detectors.native_wasb_runner import (
```

```
        NativeWasbConfig,
        NativeWasbRunner,
    )


    def _proc(rc=0, stdout="", stderr=""):
        return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)


    # --- identity / no-WSL guarantee
-----------------------------------------------------
    def test_is_a_detector_runner():
        assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)


    def test_is_not_a_wsl_runner():
        """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
        r = NativeWasbRunner(NativeWasbConfig())
        assert not isinstance(r, WslCommandMixin)
        assert not hasattr(r, "_wsl_bash")


    def test_reuses_model_agnostic_windowing():
        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

        assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
        assert (
            NativeWasbRunner.trajectory_to_action_windows
            is TrackNetRunner.trajectory_to_action_windows
        )


    # --- config resolution (env overrides win over config over default)
---------------------
    def test_from_indexing_cfg_defaults():
        cfg = NativeWasbConfig.from_indexing_cfg({})
        assert cfg.device == "cuda" and cfg.python_bin == "python"
        # weights_path now comes from the Pydantic model default
(WasbIndexConfig.weights_path)
        assert cfg.weights_path.endswith("wasb_badminton_best.pth.tar")
        assert cfg.repo_dir == "~/models/WASB-SBDT"
        # Native default is STREAMING (perf; bit-identical to disk, measured on a real GPU).
        assert cfg.stream_video is True and cfg.keep_frames is False
        # ?but an explicit stream_video=False is honoured (e.g. a container cv2 can't seek).
        assert (
            NativeWasbConfig.from_indexing_cfg(
                {"wasb": {"stream_video": False}}
            ).stream_video
            is False
        )


    def test_from_indexing_cfg_reads_config_block():
        cfg = NativeWasbConfig.from_indexing_cfg(
            {
                "wasb": {
                    "weights_path": "/m/w.pth.tar",
                    "device": "cpu",
                    "python_bin": "/envs/wasb/bin/python",
                    "repo_dir": "/m/WASB-SBDT",
                    "stream_video": True,
                    "keep_frames": True,
                    "sport": "tennis",
                }
            }
```

```
        )
        assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
        assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
        assert (
            cfg.stream_video is True and cfg.keep_frames is True and cfg.sport == "tennis"
        )


    def test_env_overrides_beat_config(monkeypatch):
        monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
        monkeypatch.setenv("WASB_DEVICE", "mps")
        monkeypatch.setenv("WASB_PYTHON", "/env/py")
        monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
        monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
        cfg = NativeWasbConfig.from_indexing_cfg(
            {
                "wasb": {
                    "weights_path": "/cfg/w",
                    "device": "cpu",
                    "python_bin": "/cfg/py",
                    "repo_dir": "/cfg/repo",
                }
            }
        )
        assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
        assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
        assert cfg.scratch_dir == "/env/scratch"


    # --- healthcheck
-----------------------------------------------------------------------
    def _ok_cfg(tmp_path, **kw):
        w = tmp_path / "w.pth.tar"
        w.write_text("weights")
        (tmp_path / "repo" / "src").mkdir(parents=True)
        return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)


    def test_healthcheck_ok(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda True")
        ):
            ok, msg = r.healthcheck()
        assert ok and "torch" in msg


    def test_healthcheck_empty_weights():
        ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
        assert not ok and "weights" in msg.lower()


    def test_healthcheck_weights_missing(tmp_path):
        (tmp_path / "repo" / "src").mkdir(parents=True)
        cfg = NativeWasbConfig(
            weights_path=str(tmp_path / "nope.pth"), repo_dir=str(tmp_path / "repo")
        )
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "not found" in msg.lower()


    def test_healthcheck_repo_missing(tmp_path):
        w = tmp_path / "w.pth.tar"
        w.write_text("x")
        cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
```

```
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "repo" in msg.lower()


    def test_healthcheck_torch_import_fails(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path))
        with patch.object(
            r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError: torch")
        ):
            ok, msg = r.healthcheck()
        assert not ok and "failed" in msg.lower()


    def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")
        ):
            ok, msg = r.healthcheck()
        assert not ok and "cuda" in msg.lower()


    def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")
        ):
            ok, msg = r.healthcheck()
        assert ok  # cpu run does not require a GPU


    def test_healthcheck_python_not_found(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
        with patch.object(r, "_run", side_effect=FileNotFoundError()):
            ok, msg = r.healthcheck()
        assert not ok and "python" in msg.lower()


    # --- M4: device="auto" CPU fallback (DeviceContext)
-----------------------------------
    def test_healthcheck_unknown_device_fails_fast(tmp_path):
        """A typo'd device fails the healthcheck with a clear error, BEFORE the torch probe
subprocess
        (beats a cryptic argparse rc=2 from wasb_infer.py at run time)."""
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="gpu"))
        with patch.object(r, "_run") as spy:
            ok, msg = r.healthcheck()
        assert not ok and "unknown device" in msg.lower() and "gpu" in msg
        spy.assert_not_called()


    def test_healthcheck_auto_uses_cuda_when_available(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="auto"))
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda True")
        ):
            ok, _msg = r.healthcheck()
        assert ok
        assert r._cuda_available is True
        assert r._effective_device() == "cuda"  # auto resolves to the GPU when present


    def test_healthcheck_auto_falls_back_to_cpu_with_warning(tmp_path, caplog):
        """device=auto on a GPU-less box does NOT hard-fail (unlike device=cuda) ? it
resolves to cpu
```

```
        and warns loudly. This is the M4 portability win for a no-GPU Linux / cpu-serving box."""
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="auto"))
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")
        ), caplog.at_level(
            logging.WARNING, logger="backend.pipeline.detectors.native_wasb_runner"
        ):
            ok, _msg = r.healthcheck()
        assert ok  # auto does not require a GPU
        assert r._effective_device() == "cpu"
        assert any("FALLING BACK TO CPU" in rec.message for rec in caplog.records)


    def test_healthcheck_caches_probe_for_run_predict(tmp_path):
        """healthcheck's CUDA probe is cached so run_predict reuses it (no second subprocess)."""
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="auto"))
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")
        ) as spy:
            r.healthcheck()
            assert spy.call_count == 1
            assert r._effective_device() == "cpu"  # uses the cache, no extra probe
            assert spy.call_count == 1


    def test_run_predict_auto_threads_cpu_when_no_gpu(tmp_path):
        """run_predict with device=auto + no GPU: the lazy probe (mocked empty stdout = no cuda)
        resolves to cpu, and 'auto' is NEVER sent to wasb_infer.py."""
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="auto"), tmp_path
        )
        assert argv[argv.index("--device") + 1] == "cpu"
        assert "auto" not in argv


    def test_run_predict_auto_threads_cuda_when_gpu(tmp_path):
        """run_predict with device=auto + a GPU: the probe reports cuda True ? --device cuda."""
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth", device="auto"))
        key = "clip__abc123abc123"
        (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")

        def _run_side(argv, cwd=None):
            if "-c" in argv:  # the CUDA probe
                return _proc(0, stdout="cuda True")
            return _proc(0)  # the inference

        with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
            r, "_run", side_effect=_run_side
        ) as spy, patch.object(r, "_rm_rf"):
            out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
        assert out is not None
        infer_argv = next(
            c.args[0] for c in spy.call_args_list if "wasb_infer.py" in c.args[0]
        )
        assert infer_argv[infer_argv.index("--device") + 1] == "cuda"


    def test_run_predict_explicit_cuda_does_not_probe(tmp_path):
        """An explicit device=cuda resolves to itself with NO probe subprocess ? unchanged behaviour
        (a single _run call: the inference)."""
```

```
        r, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path
        )
        assert argv[argv.index("--device") + 1] == "cuda"
        assert r._cuda_available is None  # never probed for an explicit device


    # --- run_predict
-----------------------------------------------------------------------
    def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
        """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
        r = NativeWasbRunner(cfg)
        key = f"clip__{vid[:12]}"
        (tmp_path / f"{key}_wasb.csv").write_text(
            "Frame,Visibility,X,Y\n"
        )  # success = CSV exists
        with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
            r, "_run", return_value=_proc(0)
        ) as run_spy, patch.object(r, "_rm_rf") as rm_spy:
            out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
        argv = run_spy.call_args[0][0]
        cwd = run_spy.call_args.kwargs.get("cwd")
        return r, out, rm_spy, argv, cwd, key


    def test_run_predict_builds_native_argv_no_wsl(tmp_path):
        # Pin disk mode here (default is now streaming) so this also covers the
--frames_out_dir path.
        _, out, rm, argv, cwd, key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="cuda", stream_video=False),
            tmp_path,
        )
        assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
        # No WSL / conda plumbing anywhere in the command.
        assert "wsl" not in argv
        assert not any("conda" in a or "/mnt/" in a for a in argv)
        # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
        assert argv[0] == "python" and argv[1] == "wasb_infer.py"
        assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
        assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
        assert argv[argv.index("--out") + 1] == out
        # Disk mode extracts frames + cleans them on success.
        assert "--frames_out_dir" in argv and "--stream-video" not in argv
        assert cwd.endswith(os.path.join("src"))
        rm.assert_called_once()


    def test_run_predict_default_is_stream_mode_no_frames_dir(tmp_path):
        # No stream_video set -> the NATIVE DEFAULT (streaming): no PNG extraction, nothing
to clean.
        _, out, rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth"), tmp_path
        )
        assert out is not None
        assert "--stream-video" in argv and "--frames_out_dir" not in argv
        rm.assert_not_called()  # streaming never materializes a frame cache


    def test_run_predict_omits_gpu_feed_flags_by_default(tmp_path):
        """batch_size/prefetch_batches default to 0 ? the flags are NOT passed, so
wasb_infer.py's
        parity defaults (batch 8, no prefetch) and the byte-parity guarantee are
untouched."""
```

```
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth"), tmp_path
        )
        assert "--batch-size" not in argv
        assert "--prefetch-batches" not in argv


    def test_run_predict_passes_log_every_batches_for_progress_sampling(tmp_path):
        """#534: log_every_batches (emission cadence for detector %) is configurable and
        threaded into the native argv so the sidecar sampling rate can be controlled end-to-
end."""
        cfg = NativeWasbConfig(weights_path="/m/w.pth", log_every_batches=10)
        _, _out, _rm, argv, _cwd, _key = _drive_success(cfg, tmp_path)
        assert "--log-every-batches" in argv
        assert argv[argv.index("--log-every-batches") + 1] == "10"

        # With the new configurable default (50), the flag is always passed for native
        # so the emission rate for progress/sidecar sampling is explicit and overridable.
        cfg_default = NativeWasbConfig(weights_path="/m/w.pth")
        _, _out, _rm, argv_def, _cwd, _key = _drive_success(cfg_default, tmp_path)
        assert "--log-every-batches" in argv_def
        assert argv_def[argv_def.index("--log-every-batches") + 1] == "50"


    def test_run_predict_threads_gpu_feed_tuning_when_set(tmp_path):
        """The cloud-serving preset's GPU-feed tuning reaches the wasb_infer.py argv."""
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(
                weights_path="/m/w.pth", batch_size=64, prefetch_batches=4
            ),
            tmp_path,
        )
        assert argv[argv.index("--batch-size") + 1] == "64"
        assert argv[argv.index("--prefetch-batches") + 1] == "4"


    def test_run_predict_frames_dir_uses_scratch_and_is_cleaned(tmp_path):
        """Frames extract under scratch_dir (the box's big NVMe) when set, and the SAME dir
is the
        one cleaned on success ? a regression that dropped scratch_dir (frames -> small
system disk)
        or diverged the cleanup target would be a disk-fill leak."""
        scratch = tmp_path / "scratch"
        r, out, rm, argv, _cwd, key = _drive_success(
            NativeWasbConfig(
                weights_path="/m/w.pth", scratch_dir=str(scratch), stream_video=False
            ),
            tmp_path,
        )
        expected_frames = os.path.join(str(scratch), f"{key}_frames")
        assert argv[argv.index("--frames_out_dir") + 1] == expected_frames
        rm.assert_called_once_with(expected_frames)  # cleanup target == extraction target


    def test_run_predict_frames_dir_defaults_next_to_output(tmp_path):
        _, out, rm, argv, _cwd, key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", stream_video=False), tmp_path
        )
        expected_frames = os.path.join(os.path.abspath(str(tmp_path)), f"{key}_frames")
        assert argv[argv.index("--frames_out_dir") + 1] == expected_frames
        rm.assert_called_once_with(expected_frames)


    def test_copy_infer_script_copies_byte_identical(tmp_path):
        """Unmocked: the native runner stages the SAME wasb_infer.py the WSL runner runs
```

```
(the
        byte-parity precondition) into <repo>/src/, byte-for-byte."""
        import filecmp

        from backend.pipeline.detectors import native_wasb_runner as nwr

        r = NativeWasbRunner(NativeWasbConfig(repo_dir=str(tmp_path / "repo")))
        assert r._copy_infer_script() is True
        dest = tmp_path / "repo" / "src" / "wasb_infer.py"
        assert dest.exists() and filecmp.cmp(nwr._INFER_PY, str(dest), shallow=False)


    def test_copy_infer_script_false_on_oserror(tmp_path, monkeypatch):
        r = NativeWasbRunner(NativeWasbConfig(repo_dir=str(tmp_path / "repo")))
        monkeypatch.setattr(
            "backend.pipeline.detectors.native_wasb_runner.shutil.copy",
            lambda *a, **k: (_ for _ in ()).throw(OSError("no space")),
        )
        assert r._copy_infer_script() is False


    def test_run_predict_device_threaded_cpu(tmp_path):
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path
        )
        assert argv[argv.index("--device") + 1] == "cpu"


    def test_run_predict_keep_frames_skips_cleanup(tmp_path):
        # Disk mode (stream_video=False) so the keep_frames branch is the reason cleanup is
skipped
        # (under the default stream mode there are no frames to clean, which would be
vacuous).
        _, _out, rm, _argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True, stream_video=False),
            tmp_path,
        )
        rm.assert_not_called()


    def test_run_predict_none_when_copy_infer_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=False):
            assert (
                r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
                is None
            )


    def test_run_predict_none_when_subprocess_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
            r, "_run", return_value=_proc(1, stderr="CUDA error")
        ):
            assert (
                r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
                is None
            )


    def test_run_predict_none_when_csv_absent(tmp_path):
        # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
            r, "_run", return_value=_proc(0)
```

```
        ):
            assert (
                r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
                is None
            )


    def test_run_predict_times_out_returns_none(tmp_path):
        import subprocess

        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
            r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x", timeout=1)
        ):
            assert (
                r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
                is None
            )


    def test_csv_name_matches_wsl_contract(tmp_path):
        """The native CSV name must equal the WSL runner's so downstream + the parity diff
line up."""
        _, out, _rm, _argv, _cwd, key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth"), tmp_path
        )
        assert os.path.basename(out) == f"{key}_wasb.csv"


    # --- #506: GPU-resident NVDEC decode backend
-------------------------------------------
    def test_run_predict_omits_decode_backend_flag_by_default(tmp_path):
        """decode_backend defaults to "cpu" ? the flag is NOT passed, so the default argv
        (and its byte-parity guarantee) is unchanged by #506."""
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth"), tmp_path
        )
        assert "--decode-backend" not in argv


    def test_run_predict_nvdec_resident_threads_flag_and_skips_cv2_paths(tmp_path):
        """nvdec_resident decodes on the GPU inside wasb_infer.py: the flag is threaded and
        NEITHER cv2 path flag (--stream-video / --frames_out_dir) rides along; there is no
        frame cache to clean."""
        _, out, rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", decode_backend="nvdec_resident"),
            tmp_path,
        )
        assert out is not None
        assert argv[argv.index("--decode-backend") + 1] == "nvdec_resident"
        assert "--stream-video" not in argv and "--frames_out_dir" not in argv
        assert argv[argv.index("--device") + 1] == "cuda"
        rm.assert_not_called()


    def test_run_predict_nvdec_resident_auto_without_gpu_aborts_before_subprocess(tmp_path):
        """device=auto resolving to cpu cannot host NVDEC decode: run_predict fails BEFORE
        launching the (paid) inference subprocess ? only the CUDA probe runs."""
        r = NativeWasbRunner(
            NativeWasbConfig(
                weights_path="/m/w.pth", device="auto", decode_backend="nvdec_resident"
            )
        )
        with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda False")
```

```
        ) as spy:
            out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
        assert out is None
        assert spy.call_count == 1  # the probe only; wasb_infer.py never launched


    def test_healthcheck_rejects_unknown_decode_backend(tmp_path):
        """A typo'd decode_backend fails fast with a clear message, before any
subprocess."""
        r = NativeWasbRunner(_ok_cfg(tmp_path, decode_backend="nvdec"))
        with patch.object(r, "_run") as spy:
            ok, msg = r.healthcheck()
        assert not ok and "decode_backend" in msg
        spy.assert_not_called()


    def test_healthcheck_nvdec_resident_requires_effective_cuda(tmp_path):
        """auto?cpu fallback is fine for the cpu decode path but NOT for nvdec_resident:
        the healthcheck must hard-fail instead of warn-and-proceed."""
        r = NativeWasbRunner(
            _ok_cfg(tmp_path, device="auto", decode_backend="nvdec_resident")
        )
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda False")
        ):
            ok, msg = r.healthcheck()
        assert not ok and "nvdec_resident" in msg


    def test_healthcheck_nvdec_resident_ok_with_cuda_and_pynvc(tmp_path):
        r = NativeWasbRunner(
            _ok_cfg(tmp_path, device="cuda", decode_backend="nvdec_resident")
        )
        with patch.object(
            r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda True")
        ) as spy:
            ok, _msg = r.healthcheck()
        assert ok
        # Both probes ran: the torch/cuda one AND the PyNvVideoCodec import.
        assert spy.call_count == 2
        assert any("PyNvVideoCodec" in c.args[0][-1] for c in spy.call_args_list)


    def test_healthcheck_nvdec_resident_requires_pynvc(tmp_path):
        """A wasb env without PyNvVideoCodec must fail the healthcheck, not the paid
        inference subprocess (the common misconfig: env built without the package)."""
        r = NativeWasbRunner(
            _ok_cfg(tmp_path, device="cuda", decode_backend="nvdec_resident")
        )

        def _run_side(argv, cwd=None):
            if "PyNvVideoCodec" in argv[-1]:
                return _proc(1, stderr="ModuleNotFoundError: No module named
'PyNvVideoCodec'")
            return _proc(0, stdout="torch 2.6.0 cuda True")

        with patch.object(r, "_run", side_effect=_run_side):
            ok, msg = r.healthcheck()
        assert not ok and "PyNvVideoCodec" in msg


    def test_run_predict_nvdec_resident_omits_prefetch_batches(tmp_path):
        """prefetch overlaps CPU batch assembly with GPU compute ? inapplicable to the
        GPU-resident decoder, so the dead knob must not ride into the argv (batch-size,
        which DOES batch nvdec windows, still threads)."""
```

```
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(
                weights_path="/m/w.pth",
                decode_backend="nvdec_resident",
                batch_size=64,
                prefetch_batches=4,
            ),
            tmp_path,
        )
        assert "--prefetch-batches" not in argv
        assert argv[argv.index("--batch-size") + 1] == "64"


    # --- the detector_impl='native' seam (parallels the stub seam)
-------------------------
    def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

        runner, params = WasbHybridSegmenter(None, {})._resolve_runner(
            {"detector_impl": "native"}
        )
        assert isinstance(runner, NativeWasbRunner)
        assert params.get("detector_impl") == "native"


    def test_resolve_runner_native_env_overrides_config(monkeypatch):
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

        runner, _ = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "auto"})
        assert isinstance(runner, NativeWasbRunner)


    def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

        runner, params = FusionSegmenter(None, {})._resolve_runner(
            {"detector_impl": "native"}
        )
        assert isinstance(runner, NativeWasbRunner)
        assert params.get("fusion_substrate") == "wasb"


    def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

        with pytest.raises(NotImplementedError):
            FusionSegmenter(None, {})._resolve_runner(
                {"detector_impl": "native", "fusion": {"substrate": "tracknet"}}
            )


    def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter

        with pytest.raises(NotImplementedError):
            TrackNetHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})


    # --- progress callback for live detector heartbeat (#534)
-------------------------------
```

```
    def test_run_predict_streams_and_calls_progress_on_pct_lines(tmp_path, caplog):
        """When progress= is supplied, run_predict switches to Popen streaming and forwards
        parsed "done/total (pct%)" lines (the format already emitted by wasb_infer) as
        "detector XX% (D/T)" callbacks. Non-matching lines are still logged but do not call
progress.
        The fast _run path is used when progress=None (byte-identical for tests)."""
        import types

        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner

        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        key = "clip__abc123abc123"
        (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")

        lines = [
            "some startup noise",
            "1234/5000 (24%) | 2.3 win/s | elapsed 45s | ETA 120s",
            "2500/5000 (50%)",
            "final line",
        ]

        class _FakeStdout:
            def __init__(self, data):
                self._data = data
                self._i = 0

            def __iter__(self):
                return self

            def __next__(self):
                if self._i >= len(self._data):
                    raise StopIteration
                val = self._data[self._i]
                self._i += 1
                return val + "\n"

        received = []

        def _fake_popen(argv, cwd=None, stdout=None, stderr=None, text=None):
            # simulate the proc
            proc = types.SimpleNamespace(
                stdout=_FakeStdout(lines),
                returncode=0,
                poll=lambda: 0,
                communicate=lambda: ("", ""),
                kill=lambda: None,
            )
            return proc

        with patch.object(r, "_copy_infer_script", return_value=True), patch(
            "backend.pipeline.detectors.native_wasb_runner.subprocess.Popen",
            side_effect=_fake_popen,
        ), patch.object(r, "_rm_rf"), caplog.at_level(logging.INFO):
            out = r.run_predict(
                r"C:\v\clip.mp4",
                str(tmp_path),
                video_id="abc123abc123",
                progress=received.append,
            )

        assert out is not None
        # Only the two matching % lines should have produced callbacks (exact format from
parse)
        assert received == ["detector 24% (1234/5000)", "detector 50% (2500/5000)"]
        # The runner still logged the lines (via logger.info inside the loop)
```

```
701         assert any("1234/5000 (24%)" in rec.getMessage() for rec in caplog.records)
702
```

49. assistant (2026-07-09T06:33:45.217Z)

This is the exact streaming test I need to extend. My runner change (merge stderr?stdout, `proc.wait()` instead of `communicate()`) means I'll update this test's fake to accept `**kwargs` and expose `wait()`. Before changing signatures, let me confirm nothing else in the codebase calls the two report helpers, and re-read the precise cmd_infer region I'll edit.

50. user (2026-07-09T06:33:47.911Z)

```
backend\worker\__main__.py:99:def _write_report_json(
backend\worker\__main__.py:125:def _serialize_and_push_outputs(
backend\worker\__main__.py:145:    _write_report_json(
backend\worker\__main__.py:318:            _serialize_and_push_outputs(
backend\worker\__main__.py:480:    pushed = _serialize_and_push_outputs(
tests\test_worker.py:73:def test_write_report_json(tmp_path):
tests\test_worker.py:75:    wmain._write_report_json(
```

51. assistant (2026-07-09T06:36:27.072Z)

I now have full understanding of all three gaps and the test patterns. Let me confirm my root-cause analysis and plan, then implement.

Root causes:
- **GAP 3** is two compounding bugs: (1) `wasb_infer.py` never configures logging, so its `logger.info` progress lines sit below root=WARNING and are suppressed; (2) even when emitted they go to stderr, but `native_wasb_runner.py` reads only stdout (and draining stdout while stderr's pipe fills risks a deadlock).
- **GAP 1/2** are in `cmd_infer`: no per-stage logging on success, no timing, and neither timings nor validator verdicts reach the pushed `report.json`.

Let me mark this phase and start implementing. First, the emission fix in `wasb_infer.py`.

52. user (2026-07-09T06:36:28.209Z)

Chapter marked: "Implement observability fixes". Continue your current work.

53. user (2026-07-09T06:36:33.576Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

54. user (2026-07-09T06:36:43.242Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-07-09T06:36:47.708Z)

Now the capture fix in `native_wasb_runner.py`: add `deque`, merge stderr?stdout, use `proc.wait()`, and keep a bounded tail for the failure message.

56. user (2026-07-09T06:36:49.992Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

57. user (2026-07-09T06:37:12.073Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

58. assistant (2026-07-09T06:37:18.279Z)

Now the worker `__main__.py`. First, extend `_write_report_json` to persist `timings` + `validation`:

59. user (2026-07-09T06:37:26.669Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

60. assistant (2026-07-09T06:37:34.216Z)

Now `_serialize_and_push_outputs` ? thread the two new args through:

61. user (2026-07-09T06:37:44.362Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

62. assistant (2026-07-09T06:37:52.823Z)

Now add the validation-summary helper just before `cmd_infer`:

63. user (2026-07-09T06:38:00.852Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

64. assistant (2026-07-09T06:38:18.174Z)

Now the core `cmd_infer` flow ? checksum timing + log, validation timing + PASS/FAIL log, shared `validation_payload`, and threading both into `_fail`:

65. user (2026-07-09T06:38:36.574Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

66. assistant (2026-07-09T06:38:45.251Z)

Now thread `timings` + `validation_payload` into the `_fail` closure's push:

67. user (2026-07-09T06:38:48.553Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

68. assistant (2026-07-09T06:39:04.409Z)

Now add segmentation timing + the per-stage log line:

69. user (2026-07-09T06:39:08.765Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-

indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

70. assistant (2026-07-09T06:39:18.115Z)

Finally, thread both into the success-path push:

71. user (2026-07-09T06:39:21.535Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

72. assistant (2026-07-09T06:39:30.985Z)

Let me re-read the modified `cmd_infer` end-to-end to verify the closure ordering and that nothing is broken:

73. user (2026-07-09T06:39:31.724Z)

```
288         with no numeric detail and to the lightweight fakes used in tests (no ``details``
attribute)."""
289         name = getattr(r, "validator_name", "?")
290         ok = bool(getattr(r, "passed", False))
291         details = getattr(r, "details", None) or {}
292         hint = ""
293         for k in _VALIDATOR_SCORE_KEYS:
294             if k in details:
295                 hint = f"({k}={details[k]})"
296                 break
297         return f"{name}={'ok' if ok else 'FAIL'}{hint}"
298
299
300     def cmd_infer(args: argparse.Namespace) -> int:
301         config = load_config(args.config) if args.config else load_config()
302         _apply_overrides(config, args.set)
303         if not _run_startup_checks(config):
304             return 2
305
306         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
307         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
308         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
309         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
310         compute = get_compute_backend(config)
311         if compute is not None:
312             return _dispatch_remote(compute, args)
313
314         storage_cfg = config.storage.model_dump()
315
316         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
317         os.makedirs(scratch, exist_ok=True)
318         config.output.output_dir = scratch
319
320         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
321         local_video = storage.fetch(
322             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
323         )
324         print(f"[worker] pulled {args.video_uri} -> {local_video}")
325
```

```
326         # Per-stage wall-clock (#534 follow-up, 2026-07-09 live CUJ): each stage is timed
with
327         # perf_counter and persisted into report.json, so a runlog shows time-per-stage
instead of the
328         # earlier black hole (a ~10-min segmentation recorded nothing anywhere). Built
incrementally so
329         # a mid-run _fail() still reports the stages that DID run.
330         timings: dict[str, float] = {}
331
332         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
333         _t = time.perf_counter()
334         video_id = compute_video_id(local_video)
335         timings["checksum_s"] = round(time.perf_counter() - _t, 3)
336         print(f"[worker] checksum: video_id={video_id} ({timings['checksum_s']:.1f}s)")
337
338         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
339         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
340         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
341         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
342         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
343         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
344         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
345         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
346         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
347         #
348         # When it DOES run, log the verdict loudly (PASS or FAIL): on the cloud-serving path
349         # phase_placement.validation=cloud_run_l4 means the worker really does run the full
suite, but a
350         # PASS previously logged nothing at all ? users reasonably concluded validation was
skipped.
351         if getattr(args, "skip_validation", False):
352             print(
353                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
354                 "gate and already validated this input."
355             )
356             val_results, val_context = [], {}
357             timings["validation_s"] = 0.0
358         else:
359             n_checks = len(getattr(validator_registry, "_validators", []))
360             print(f"[worker] validating ({n_checks} checks)?")
361             _t = time.perf_counter()
362             val_results, val_context = validator_registry.run_all_with_context(
363                 local_video, config
364             )
365             timings["validation_s"] = round(time.perf_counter() - _t, 3)
366             verdict = "FAILED" if any(not r.passed for r in val_results) else "PASSED"
367             summary = " ".join(_val_token(r) for r in val_results) or "(no checks ran)"
368             print(
369                 f"[worker] validation {verdict} in {timings['validation_s']:.1f}s ?
{summary}"
370             )
371         # The per-validator verdicts (name/passed/message/details) ? stored BOTH in the
ephemeral worker
372         # DB (below) and, now, in the pushed report.json (so a runlog captures validator
outcomes).
```

```
        validation_payload = [r.model_dump() for r in val_results]
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            validation_payload,
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                    timings=timings,
                    validation=validation_payload,
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
```

```
            segmenter_actual = seg_name
            out_local = os.path.join(scratch, "outputs", video_id)  # for sidecar +
telemetry
            os.makedirs(out_local, exist_ok=True)

            # Live telemetry black-box for L4 detector runs (#534): velocity + GPU/CPU/RAM
samples.
            # Written under outputs/ so the final push will mirror it; snapshots driven from
detector %.
            telem: Optional[RunTelemetry] = None
            try:
                telem = RunTelemetry(
                    os.path.join(out_local, "run_telemetry.jsonl"),
                    label=f"l4-infer-{video_id[:8]}",
                )
            except Exception:  # noqa: BLE001 - telemetry must never break a production run
                telem = None

            out_prefix = getattr(args, "out_uri", None)
            if out_prefix:
                out_prefix = out_prefix.rstrip("/")

            # Configurable sampling for sidecar upload + telemetry snapshots (#534).
            # Local sidecar file is always kept up-to-date (cheap). Only the costly remote
put
            # and nvidia-smi-using snapshot are rate-limited. 0/negative = every callback.
            sample_sec = 15.0
            try:
                idx_cfg = getattr(config, "indexing", None)
                if idx_cfg is not None:
                    raw = getattr(idx_cfg, "detector_progress_sample_sec", 15.0)
                    sample_sec = float(raw) if raw is not None else 15.0
            except Exception:  # noqa: BLE001
                sample_sec = 15.0

            last_sample_t = [0.0]

            def _progress(s: str) -> None:
                print(f"[worker] progress: {s}")
                # Write sidecar for cloud/L4 poll to surface live detector progress in job
status (#534).
                # CP will read outputs/<video_id>/*_detector_progress.json during poll.
                # The *local* file is always written so the latest state is available in
scratch.
                now = time.time()
                do_sample = sample_sec <= 0 or (now - last_sample_t[0] >= sample_sec)

                pfile = os.path.join(out_local, f"{video_id}_detector_progress.json")
                try:
                    with open(pfile, "w") as pf:
                        json.dump({"msg": s, "t": now}, pf)
                except Exception:
                    pass  # best effort

                heavy_done = False
                # LIVE push to storage (the part visible to CP poll during the run).
                if out_prefix and do_sample:
                    try:
                        storage.put(
                            pfile,
                            f"{out_prefix}/outputs/{video_id}/{os.path.basename(pfile)}",
                            config=storage_cfg,
                        )
                        heavy_done = True
                    except Exception:
```

```
490                         pass  # best-effort live heartbeat
491
492                 # Drive RunTelemetry snapshots (includes nvidia-smi + file rewrite) only on
sample.
493                 if telem and s and "detector" in (s or "") and do_sample:
494                     try:
495                         m = re.search(r"(\d+)/(\d+)", s)
496                         done = int(m.group(1)) if m else None
497                         total = int(m.group(2)) if m else None
498                         telem.snapshot(done=done, total=total, phase="detector")
499                         heavy_done = True
500                     except Exception:  # noqa: BLE001
501                         pass
502
503                 if heavy_done:
504                     last_sample_t[0] = now
505
506             seg_t = time.perf_counter()
507             result = segmenter_cls(db, config).process_video(
508                 video_id,
509                 local_video,
510                 max_frames=args.max_frames,
511                 progress=_progress,
512             )
513             segmentation_ran = True
514             timings["segmentation_s"] = round(time.perf_counter() - seg_t, 3)
515             if isinstance(result, Failure):
516                 print(
517                     f"[worker] segmentation ({segmenter_actual}) FAILED after "
```

74. assistant (2026-07-09T06:39:40.753Z)

Let me verify the remainder of the segmentation block and the tail:

75. user (2026-07-09T06:39:42.135Z)

```
515             if isinstance(result, Failure):
516                 print(
517                     f"[worker] segmentation ({segmenter_actual}) FAILED after "
518                     f"{timings['segmentation_s']:.1f}s ? {result.message}"
519                 )
520                 return _fail(3)
521             segments = result.value if hasattr(result, "value") else result
522             status = "success"
523             print(
524                 f"[worker] segmentation ({segmenter_actual}) done in "
525                 f"{timings['segmentation_s']:.1f}s ? {len(segments)} rallies"
526             )
527             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
528             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
529             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
530             if not segments:
531                 detector_impl = (
532                     os.environ.get("RALLY_DETECTOR_IMPL")
533                     or getattr(config.indexing, "detector_impl", None)
534                     or "auto"
535                 )
536                 logger.warning(
537                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
538                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
```

```
539                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
540                     "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
541                     "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
542                     "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
543                     "detections, or the input resolution differs from the tuned proxy
resolution. "
544                     "Re-run with -v for the native command + frame counts.",
545                     video_id,
546                     segmenter_actual,
547                     detector_impl,
548                 )
549         finally:
550             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
551             emit_indexer_report(
552                 db=db,
553                 video_id=video_id,
554                 video_path=local_video,
555                 config=config,
556                 val_results=val_results,
557                 val_context=val_context,
558                 segmenter_requested=args.segmenter,
559                 segmenter_actual=segmenter_actual,
560                 was_reingest=False,
561                 segmentation_ran=segmentation_ran,
562             )
563
564         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
565         pushed = _serialize_and_push_outputs(
566             scratch,
567             args.out_uri,
568             video_id,
569             segments,
570             status,
571             storage_cfg,
572             str(getattr(args, "video_uri", "") or ""),
573             timings=timings,
574             validation=validation_payload,
```

76. assistant (2026-07-09T06:41:41.558Z)

The worker `cmd_infer` flow is correct ? `timings` and `validation_payload` are defined before the `_fail` closure and threaded into every push. Now let me check how `wasb_infer.py` is imported by existing tests and whether it has heavy module-level imports (to pick the right test approach for the emission fix).

77. user (2026-07-09T06:41:43.814Z)

```
49:import argparse
50:import csv
51:import glob
52:import json
53:import logging
54:import os
55:import os.path as osp
56:import sys
57:from collections import defaultdict
58:from contextlib import contextmanager
```

78. user (2026-07-09T06:41:44.981Z)

```
28:from backend.pipeline.detectors import wasb_infer as wi
339:        spec = importlib.util.spec_from_file_location("wasb_infer_staged", staged)
```

79. assistant (2026-07-09T06:42:57.360Z)

Module-level imports are stdlib-only, so `wasb_infer` imports cleanly. To make the emission fix independently testable (and to avoid mutating global logging inside `main()` untestably), let me extract the logging config into a small `_configure_logging()` helper ? mirroring the worker's tested `_setup_logging` pattern.

80. user (2026-07-09T06:43:06.244Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

81. assistant (2026-07-09T06:43:17.924Z)

Now the tests. First, update the existing streaming test's fake to match the new Popen call (`bufsize`, `wait()`):

82. user (2026-07-09T06:43:21.858Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

83. assistant (2026-07-09T06:43:34.600Z)

Now append the new native-runner tests (stderr-merge, failure-tail) and the `wasb_infer` emission test:

84. user (2026-07-09T06:43:54.400Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

85. assistant (2026-07-09T06:44:05.255Z)

Now the worker tests. First add the `types` import:

86. user (2026-07-09T06:44:07.265Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

87. assistant (2026-07-09T06:44:10.099Z)

Now add the pure-helper tests (report serialization + `_val_token`) after the existing `test_write_report_json`:

88. user (2026-07-09T06:44:23.470Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

89. assistant (2026-07-09T06:45:04.435Z)

Now the two end-to-end `cmd_infer` tests (PASS logs + report shape; FAIL timings/logging), placed after `test_cmd_infer_local_roundtrip`:

90. user (2026-07-09T06:45:20.781Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

91. assistant (2026-07-09T06:45:37.235Z)

```
All edits are in. Now let me run the verification gates. First, a quick environment probe
(Python, ruff, mypy, pytest versions) so I know what's installed.
```

92. user (2026-07-09T06:45:40.826Z)

```
Python 3.13.13
--- ruff ---
ruff 0.15.16
--- mypy ---
mypy 2.1.0 (compiled: yes)
--- pytest ---
pytest 8.4.2
```

93. assistant (2026-07-09T06:45:50.249Z)

```
Toolchain matches CI exactly (ruff 0.15.16, mypy 2.1.0) ? no installs needed. Let me run the
directly-affected tests first for fast feedback.
```

94. user (2026-07-09T06:45:59.740Z)

```
.................................................................... [ 70%]
..............................                                       [100%]
102 passed in 6.20s
```

95. assistant (2026-07-09T06:46:08.988Z)

```
All 102 pass. Now ruff and mypy:
```

96. user (2026-07-09T06:46:09.921Z)

```
All checks passed!
```

97. user (2026-07-09T06:46:19.068Z)

```
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
```

98. assistant (2026-07-09T06:46:39.042Z)

```
Ruff clean, mypy clean (150 files, only informational notes). Now the full suite with coverage.
Let me first confirm `run_all_tests.py` passes through the `--cov` flags so I invoke it
correctly.
```

99. user (2026-07-09T06:46:40.170Z)

```
1       #!/usr/bin/env python
2       """Run the FULL test suite for the badminton-highlight-indexer.
3
4       A single, discoverable entry point so you (or a reviewer / CI) can verify the
5       whole suite before opening a PR:
6
```

```
    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = (
        "available"
        if _installed("obsreport")
        else "NOT installed ? reporting tests will SKIP"
    )
    bar = "=" * 72
    print(bar)
    print(" Full test suite:  " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)

    rc = subprocess.run(cmd).returncode

    print(bar)
    print(
        " RESULT: "
        + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
    )
    print(bar)
    return rc


if __name__ == "__main__":
    raise SystemExit(main())

```

100. assistant (2026-07-09T06:46:48.443Z)

`run_all_tests.py` passes args straight through to pytest. Running the full suite with coverage
now (this is the pre-PR gate):

101. user (2026-07-09T06:53:51.216Z)

```
backend\pipeline\validators\format.py                36     16    56%
backend\pipeline\validators\frame_sampler.py         54      1    98%
backend\pipeline\validators\pts_continuity.py        56     39    30%
backend\pipeline\validators\relevance.py             51     23    55%
backend\pipeline\validators\resolution_fps.py        50     23    54%
backend\pipeline\validators\scene_cut.py             57     38    33%
backend\providers\__init__.py                        16      1    94%
backend\providers\base.py                             9      1    89%
backend\providers\gemini.py                         137     21    85%
backend\reporting\__init__.py                        26      2    92%
backend\reporting\harvest.py                        111      8    93%
backend\reporting\run_signals.py                     14      0   100%
backend\results.py                                   27      0   100%
backend\server_runtime.py                            57      2    96%
backend\sports\__init__.py                           17      1    94%
backend\sports\badminton.py                          13      1    92%
backend\sports\base.py                               12      3    75%
backend\storage\__init__.py                         128     16    88%
backend\storage\base.py                              16      1    94%
backend\storage\capabilities.py                      66     11    83%
backend\storage\gcs.py                               80      3    96%
backend\storage\gdrive.py                            61      3    95%
backend\storage\gdrive_api.py                       105      6    94%
backend\storage\local.py                             49      3    94%
backend\storage\ref.py                               49      0   100%
backend\storage\s3.py                                47      5    89%
backend\tools\__init__.py                             0      0   100%
backend\tools\export.py                              75      2    97%
backend\tools\rally_slicer.py                       105     47    55%
backend\utils\__init__.py                             0      0   100%
backend\utils\annotation_proxy.py                    64     11    83%
backend\utils\app_home.py                            40      0   100%
backend\utils\ffmpeg.py                              47      3    94%
backend\utils\freespace.py                           32      2    94%
backend\utils\geometry.py                            29      0   100%
backend\utils\hashing.py                              9      0   100%
backend\utils\logging_setup.py                       11      0   100%
backend\utils\paths.py                               44      3    93%
backend\utils\redact.py                              16      0   100%
backend\utils\run_telemetry.py                      126     12    90%
backend\utils\single_instance.py                     43     11    74%
backend\utils\video.py                               34      3    91%
backend\utils\workspace.py                           49      0   100%
backend\worker\__init__.py                            0      0   100%
backend\worker\__main__.py                          457     43    91%
training\__init__.py                                  0      0   100%
training\gen0\__init__.py                             0      0   100%
training\gen0\harness.py                            130      9    93%
----------------------------------------------------------------------
TOTAL                                             13582   2114    84%
Required test coverage of 70% reached. Total coverage: 84.44%
1965 passed, 1 skipped, 57 warnings in 417.23s (0:06:57)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

102. assistant (2026-07-09T06:54:28.020Z)

Full suite: **1965 passed, 1 skipped**, coverage **84.44%** (?70% floor), worker `__main__.py`
at 91%. No `test_storage.py` failures ? #548 (already on origin/master) resolved them. ruff and

mypy clean.

One backward-compat check before I commit: I added `timings`/`validation` keys to `report.json`.
Let me verify the control-plane's reader doesn't reject unknown keys (e.g. a Pydantic model with
`extra="forbid"`).

103. user (2026-07-09T06:54:30.399Z)

```
111:    if report.get("output_dir"):
314:            uri, os.path.join(td, ref.name or "report.json"), config=storage_config
360:def probe_process_cache(db, cfg, video_id: str) -> bool:
381:        report_uri = f"{output_root}/outputs/{video_id}/report.json"
385:        return bool(report and report.get("status") == "success")
395:def cached_process_result(
399:    the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
429:            report_uri = f"{outputs_uri}/report.json"
433:            if not report or report.get("status") != "success":
441:                video_path = str(report.get("video_uri") or "") or _first_object_under(
493:    if not probe_process_cache(db, cfg, vid):
495:    return cached_process_result(db, cfg, vid, "") is not None
1685:            result.report.get("message")
1691:        result.report.get("download_url")
1704:        "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
1707:        "reel_uri": str(result.report.get("reel_uri") or ""),
1708:        "filename": str(result.report.get("filename") or out_name),
1709:        "rendition": str(result.report.get("rendition") or rendition),
```

104. assistant (2026-07-09T06:54:43.594Z)

Confirmed ? the CP reads `report.json` via plain `report.get(...)`, so the additive
`timings`/`validation` keys are ignored by existing readers. Backward compatible.

Let me self-review the full diff before committing.

105. user (2026-07-09T06:54:49.901Z)

```
 backend/pipeline/detectors/native_wasb_runner.py |  57 +++++---
 backend/pipeline/detectors/wasb_infer.py         |  22 +++
 backend/worker/__main__.py                       | 135 ++++++++++++++----
 tests/test_native_wasb_runner.py                 | 110 +++++++++++++-
 tests/test_worker.py                             | 172 +++++++++++++++++++++
 5 files changed, 450 insertions(+), 46 deletions(-)
======== FULL DIFF: detectors ========
diff --git a/backend/pipeline/detectors/native_wasb_runner.py
b/backend/pipeline/detectors/native_wasb_runner.py
index 81ebbfd..1d80828 100644
--- a/backend/pipeline/detectors/native_wasb_runner.py
+++ b/backend/pipeline/detectors/native_wasb_runner.py
@@ -39,6 +39,7 @@ import os
 import re
 import shutil
 import subprocess
+from collections import deque
 from dataclasses import dataclass
 from typing import Callable, List, Optional, Tuple

@@ -369,41 +370,57 @@ class NativeWasbRunner(DetectorRunner):
                 # Fast path: capture at end (existing behaviour, byte-identical for tests).
                 res = self._run(argv, cwd=self._repo_src())
             else:
-                # Live progress path (#534): stream stdout so we can parse the periodic
-                # "done/total (pct%) ..." lines already emitted by wasb_infer.py and call
-                # the callback. This gives the SPA a real heartbeat during the long GPU pass
-                # instead of a spinner. Still respects timeout.
```

```
+                # Live progress path (#534): stream the child's output so we can parse the
+                # periodic "done/total (pct%) ..." lines wasb_infer emits and fire the callback
?
+                # a real heartbeat during the long GPU pass instead of a spinner.
+                #
+                # MERGE stderr INTO stdout (stderr=STDOUT). wasb_infer routes its logging
(progress
+                # included) to a stream, and third-party libs may also write to stderr. Reading
+                # stdout alone would (a) miss any progress that lands on stderr and (b) risk a
+                # DEADLOCK: draining only stdout while the child fills its ~64 KB stderr pipe
blocks
+                # the child on the stderr write, which stops its stdout too ? the run then
hangs
+                # until timeout. One merged pipe removes both failure modes (the 2026-07-09
live CUJ
+                # produced zero heartbeat lines). bufsize=1 = line-buffered so the heartbeat is
+                # timely, not block-delayed. A bounded tail keeps the last lines for the
failure
+                # message, since there is no separate stderr to read on error.
                 proc = subprocess.Popen(
                     argv,
                     cwd=self._repo_src(),
                     stdout=subprocess.PIPE,
-                    stderr=subprocess.PIPE,
+                    stderr=subprocess.STDOUT,
                     text=True,
+                    bufsize=1,
                 )
                 assert proc.stdout is not None
+                tail: "deque[str]" = deque(maxlen=200)
                 try:
                     for line in proc.stdout:
                         line = line.rstrip()
-                        if line:
-                            # Parse existing wasb_infer progress lines for detector heartbeat.
-                            # Example: "1234/5000 (24%) | 2.3 win/s | elapsed 45s | ETA 120s"
-                            m = re.search(r"(\d+)/(\d+)\s+\((\d+)%\)", line)
-                            if m:
-                                done, total, pct = m.groups()
-                                try:
-                                    progress(f"detector {pct}% ({done}/{total})")
-                                except Exception:  # noqa: BLE001
-                                    pass  # progress must never kill the run
-                            # Also log for the container/runlog.
-                            logger.info(line)
-                    # Wait for completion (communicate for stderr/returncode).
-                    _, stderr = proc.communicate()
+                        if not line:
+                            continue
+                        tail.append(line)
+                        # Parse wasb_infer progress lines for the detector heartbeat.
+                        # Example: "1234/5000 (24%) | 2.3 win/s | elapsed 45s | ETA 120s"
+                        m = re.search(r"(\d+)/(\d+)\s+\((\d+)%\)", line)
+                        if m:
+                            done, total, pct = m.groups()
+                            try:
+                                progress(f"detector {pct}% ({done}/{total})")
+                            except Exception:  # noqa: BLE001
+                                pass  # progress must never kill the run
+                        # Mirror to the container/runlog.
+                        logger.info(line)
+                    proc.wait()
                     res = subprocess.CompletedProcess(
-                        args=argv, returncode=proc.returncode, stdout="", stderr=stderr or ""
+                        args=argv,
```

```
+                        returncode=proc.returncode,
+                        stdout="",
+                        stderr="\n".join(tail),
                     )
                 finally:
                     if proc.poll() is None:
                         proc.kill()
+                        proc.wait()
         except FileNotFoundError:
             logger.error(
                 "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
diff --git a/backend/pipeline/detectors/wasb_infer.py b/backend/pipeline/detectors/wasb_infer.py
index ef43816..13c38a4 100644
--- a/backend/pipeline/detectors/wasb_infer.py
+++ b/backend/pipeline/detectors/wasb_infer.py
@@ -53,6 +53,7 @@ import json
 import logging
 import os
 import os.path as osp
+import sys
 from collections import defaultdict
 from contextlib import contextmanager

@@ -1247,7 +1248,28 @@ def run_video_nvdec_resident(
     return _tracker_pass_and_write(cfg, det_by_fid, out_csv)


+def _configure_logging() -> None:
+    """Own logging config for this SUBPROCESS entrypoint (run as ``python wasb_infer.py ?`` by
the
+    WSL / native runners). Without it the module logger inherits the root default (WARNING) and
+    EVERY ``logger.info()`` below is silently dropped ? including the periodic "done/total
(pct%)"
+    progress lines the native runner parses for the #534 live detector heartbeat. That is
exactly
+    why a ~10-min L4 pass produced zero progress lines, zero telemetry snapshots, and no
+    detector_progress sidecar (2026-07-09 live CUJ).
+
+    Route to STDOUT ? the stream ``NativeWasbRunner`` streams + parses ? and ``force=True`` so
a
+    library that pre-configured logging can't leave us pinned at WARNING. ``StreamHandler``
flushes
+    per record, so the heartbeat is timely rather than block-buffered."""
+    logging.basicConfig(
+        level=logging.INFO,
+        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
+        datefmt="%H:%M:%S",
+        stream=sys.stdout,
+        force=True,
+    )
+
+
 def main():
+    _configure_logging()
     ap = argparse.ArgumentParser()
     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
     ap.add_argument(
```

106. assistant (2026-07-09T06:54:56.474Z)

The detector diffs are clean. Now the worker diff:

107. user (2026-07-09T06:54:57.494Z)

```
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
```

```
index faccb9f..1692d84 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -97,29 +97,40 @@ def _write_intervals_csv(segments: List[dict], path: str) -> None:


 def _write_report_json(
-    video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
+    video_id: str,
+    segments: List[dict],
+    status: str,
+    path: str,
+    video_uri: str = "",
+    timings: Optional[dict] = None,
+    validation: Optional[List[dict]] = None,
 ) -> None:
-    with open(path, "w") as f:
-        json.dump(
+    payload: dict[str, Any] = {
+        "video_id": video_id,
+        "status": status,
+        "segment_count": len(segments),
+        # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
+        # control plane re-hydrating from outputs/<md5>/ restores the video row
+        # with its truthful path ? without this, only the segments recover.
+        "video_uri": video_uri,
+        "segments": [
             {
-                "video_id": video_id,
-                "status": status,
-                "segment_count": len(segments),
-                # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
-                # control plane re-hydrating from outputs/<md5>/ restores the video row
-                # with its truthful path ? without this, only the segments recover.
-                "video_uri": video_uri,
-                "segments": [
-                    {
-                        "start": float(s.get("start_time", 0.0)),
-                        "end": float(s.get("end_time", 0.0)),
-                    }
-                    for s in segments
-                ],
-            },
-            f,
-            indent=2,
-        )
+                "start": float(s.get("start_time", 0.0)),
+                "end": float(s.get("end_time", 0.0)),
+            }
+            for s in segments
+        ],
+    }
+    # Observability (#534 follow-up, 2026-07-09 live CUJ): persist per-stage wall-clock AND the
+    # validator verdicts the worker actually computed. Previously both were invisible on a PASS
?
+    # the results lived only in the ephemeral worker DB and never reached this pushed report,
so a
+    # passing validation looked skipped and a ~10-min segmentation recorded no timing anywhere.
+    if timings:
+        payload["timings"] = timings
+    if validation is not None:
+        payload["validation"] = validation
+    with open(path, "w") as f:
+        json.dump(payload, f, indent=2)
```

```
 def _serialize_and_push_outputs(
@@ -130,6 +141,8 @@ def _serialize_and_push_outputs(
     status: str,
     storage_cfg: dict,
     video_uri: str = "",
+    timings: Optional[dict] = None,
+    validation: Optional[List[dict]] = None,
 ) -> List[str]:
     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and push both
to
     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
@@ -138,7 +151,10 @@ def _serialize_and_push_outputs(
     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
     report as "no usable result" ? retry, and the restart-recovery poll waits for report.json
to
-    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
+    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
+
+    ``timings`` (per-stage wall-clock) and ``validation`` (the per-validator verdicts) are
folded
+    into report.json for observability (see :func:`_write_report_json`)."""
     out_local = os.path.join(scratch, "outputs", video_id)
     os.makedirs(out_local, exist_ok=True)
     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
@@ -148,6 +164,8 @@ def _serialize_and_push_outputs(
         status,
         os.path.join(out_local, "report.json"),
         video_uri=video_uri,
+        timings=timings,
+        validation=validation,
     )
     out_prefix = out_uri.rstrip("/")
     pushed: List[str] = []
@@ -249,6 +267,36 @@ def _write_done_marker(
         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


+# Headline metric key per validator (in ``ValidationResult.details``) for the one-line
validation
+# summary log. The FULL details dict is persisted in report.json; this is only the at-a-glance
+# score for the human-readable ``[worker] validation ?`` line. First key present wins.
+_VALIDATOR_SCORE_KEYS = (
+    "avg_laplacian_variance",  # blur_check ? sharpness
+    "avg_green_court_ratio",  # relevance ? court coverage
+    "cuts_per_minute",  # scene_cut
+    "max_keyframe_gap_seconds",  # pts_continuity
+    "fps",  # resolution_fps
+    "duration",  # format_check
+)
+
+
+def _val_token(r: Any) -> str:
+    """One ``name=ok`` / ``name=FAIL(score)`` token for the validation-summary log line.
+
+    ``score`` is a best-effort headline metric pulled from ``details`` (see
+    :data:`_VALIDATOR_SCORE_KEYS`) and omitted when absent ? so this is robust to a validator
+    with no numeric detail and to the lightweight fakes used in tests (no ``details``
attribute)."""
+    name = getattr(r, "validator_name", "?")
+    ok = bool(getattr(r, "passed", False))
+    details = getattr(r, "details", None) or {}
+    hint = ""
+    for k in _VALIDATOR_SCORE_KEYS:
```

```
+        if k in details:
+            hint = f"({k}={details[k]})"
+            break
+    return f"{name}={'ok' if ok else 'FAIL'}{hint}"
+
+
 def cmd_infer(args: argparse.Namespace) -> int:
     config = load_config(args.config) if args.config else load_config()
     _apply_overrides(config, args.set)
@@ -275,8 +323,17 @@ def cmd_infer(args: argparse.Namespace) -> int:
     )
     print(f"[worker] pulled {args.video_uri} -> {local_video}")

+    # Per-stage wall-clock (#534 follow-up, 2026-07-09 live CUJ): each stage is timed with
+    # perf_counter and persisted into report.json, so a runlog shows time-per-stage instead of
the
+    # earlier black hole (a ~10-min segmentation recorded nothing anywhere). Built
incrementally so
+    # a mid-run _fail() still reports the stages that DID run.
+    timings: dict[str, float] = {}
+
     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
+    _t = time.perf_counter()
     video_id = compute_video_id(local_video)
+    timings["checksum_s"] = round(time.perf_counter() - _t, 3)
+    print(f"[worker] checksum: video_id={video_id} ({timings['checksum_s']:.1f}s)")

     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
@@ -287,23 +344,40 @@ def cmd_infer(args: argparse.Namespace) -> int:
     # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
     # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+    #
+    # When it DOES run, log the verdict loudly (PASS or FAIL): on the cloud-serving path
+    # phase_placement.validation=cloud_run_l4 means the worker really does run the full suite,
but a
+    # PASS previously logged nothing at all ? users reasonably concluded validation was
skipped.
     if getattr(args, "skip_validation", False):
         print(
             "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
             "gate and already validated this input."
         )
         val_results, val_context = [], {}
+        timings["validation_s"] = 0.0
     else:
+        n_checks = len(getattr(validator_registry, "_validators", []))
+        print(f"[worker] validating ({n_checks} checks)?")
+        _t = time.perf_counter()
         val_results, val_context = validator_registry.run_all_with_context(
             local_video, config
         )
+        timings["validation_s"] = round(time.perf_counter() - _t, 3)
+        verdict = "FAILED" if any(not r.passed for r in val_results) else "PASSED"
+        summary = " ".join(_val_token(r) for r in val_results) or "(no checks ran)"
+        print(
+            f"[worker] validation {verdict} in {timings['validation_s']:.1f}s ? {summary}"
+        )
+    # The per-validator verdicts (name/passed/message/details) ? stored BOTH in the ephemeral
worker
```

```
+    # DB (below) and, now, in the pushed report.json (so a runlog captures validator outcomes).
+    validation_payload = [r.model_dump() for r in val_results]
     failures = [r for r in val_results if not r.passed]
     db = Database(os.path.join(scratch, config.output.db_name))
     db.add_video(
         video_id,
         local_video,
         "failed" if failures else "processed",
-        [r.model_dump() for r in val_results],
+        validation_payload,
     )

     def _fail(rc: int) -> int:
@@ -323,6 +397,8 @@ def cmd_infer(args: argparse.Namespace) -> int:
                 "failed",
                 storage_cfg,
                 str(getattr(args, "video_uri", "") or ""),
+                timings=timings,
+                validation=validation_payload,
             )
         except Exception as e:  # never mask the real failure on an artifact-push hiccup
             logger.warning("[worker] could not push failure artifacts: %s", e)
@@ -427,6 +503,7 @@ def cmd_infer(args: argparse.Namespace) -> int:
             if heavy_done:
                 last_sample_t[0] = now

+        seg_t = time.perf_counter()
         result = segmenter_cls(db, config).process_video(
             video_id,
             local_video,
@@ -434,11 +511,19 @@ def cmd_infer(args: argparse.Namespace) -> int:
             progress=_progress,
         )
         segmentation_ran = True
+        timings["segmentation_s"] = round(time.perf_counter() - seg_t, 3)
         if isinstance(result, Failure):
-            print(f"[worker] segmenter FAILED: {result.message}")
+            print(
+                f"[worker] segmentation ({segmenter_actual}) FAILED after "
+                f"{timings['segmentation_s']:.1f}s ? {result.message}"
+            )
             return _fail(3)
         segments = result.value if hasattr(result, "value") else result
         status = "success"
+        print(
+            f"[worker] segmentation ({segmenter_actual}) done in "
+            f"{timings['segmentation_s']:.1f}s ? {len(segments)} rallies"
+        )
         # Loud, never-silent diagnostic: a 0-segment success is almost always a misconfig, not
an
         # empty match ? the cold-start failure mode (the substrate detector yielded no usable
         # trajectories). Explain it so a run is analysable from the log alone (re-run -v for
more).
@@ -485,6 +570,8 @@ def cmd_infer(args: argparse.Namespace) -> int:
         status,
         storage_cfg,
         str(getattr(args, "video_uri", "") or ""),
+        timings=timings,
+        validation=validation_payload,
     )

     print(
```

108. assistant (2026-07-09T06:55:26.742Z)

The worker diff is clean and all three gaps are addressed. Everything verifies: 1965 passed, ruff + mypy clean, coverage 84.44%, backward-compatible report.json. Let me re-verify branch safety (concurrent-session protocol), then stage explicit paths and commit.

109. user (2026-07-09T06:55:30.559Z)

```
=== current branch (must be mine) ===
fix/worker-l4-infer-observability
=== origin/master now ===
00e8f92
=== commits on my branch vs origin/master (must be EMPTY pre-commit) ===
(end)
```

110. assistant (2026-07-09T06:55:38.395Z)

Branch safety confirmed: on my branch, origin/master unchanged at `00e8f92`, clean base. Staging explicit paths only (never `-A`) and committing:

111. user (2026-07-09T06:55:40.690Z)

```
=== staged ===
M  backend/pipeline/detectors/native_wasb_runner.py
M  backend/pipeline/detectors/wasb_infer.py
M  backend/worker/__main__.py
M  tests/test_native_wasb_runner.py
M  tests/test_worker.py
?? "SESSION_HANDOFF (1).md"
=== confirm HEAD is mine ===
fix/worker-l4-infer-observability
```

112. assistant (2026-07-09T06:55:46.688Z)

Exactly my 5 files staged (the stray untracked handoff file is not). Committing:

113. user (2026-07-09T06:56:01.038Z)

```
[fix/worker-l4-infer-observability b3dff78] fix(worker): make the cloud-serving L4 infer path
observable
 5 files changed, 450 insertions(+), 46 deletions(-)
```

114. assistant (2026-07-09T06:56:09.330Z)

Committed as `b3dff78` (5 files, +450/?46). Verifying only my commit rides on origin/master, then pushing:

115. user (2026-07-09T06:56:13.499Z)

```
=== commits vs origin/master (must be ONLY mine) ===
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
=== push ===
remote:
remote: Create a pull request for 'fix/worker-l4-infer-observability' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-
indexer/pull/new/fix/worker-l4-infer-observability
remote:
branch 'fix/worker-l4-infer-observability' set up to track 'origin/fix/worker-l4-infer-
observability'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/worker-l4-infer-observability -> fix/worker-l4-infer-observability
```

116. assistant (2026-07-09T06:56:23.201Z)

Pushed cleanly with only my commit. Now opening the PR to master:

117. user (2026-07-09T06:56:44.981Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/550

118. assistant (2026-07-09T06:57:03.723Z)

PR #550 is open: https://github.com/Khelsutra/badminton-highlight-indexer/pull/550

Let me record the one non-derivable fact for future sessions ? that the live-L4 heartbeat still needs an owner-driven CUJ to confirm end-to-end ? as a memory with an index pointer.

119. assistant (2026-07-09T06:57:19.282Z)

```
---
name: pr550-worker-l4-observability
description: "PR #550 (open) makes the cloud-serving L4 worker infer path observable; live-L4
heartbeat still needs an owner CUJ"
metadata:
  type: project
---
```

PR #550 ? cloud-serving L4 infer observability (OPEN, 2026-07-09)

Branch `fix/worker-l4-infer-observability` cut from `origin/master @ 00e8f92`; commit `b3dff78`.
Fixes three linked observability gaps on `backend/worker/__main__.py`'s infer path, surfaced by the
2026-07-09 live CUJ (video GX010155, deployed image `sha256:1838c07f?`, worker `rally-worker-dpssz`).
Because `phase_placement.validation/segment/compile` are all `cloud_run_l4`, the worker runs the full
validator suite + segmentation but none of it was observable.

- **GAP 1 (validation invisible on PASS):** now logs `[worker] validating (N checks)?` +
  `[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`, and the per-validator verdicts
  are folded into the pushed `report.json` under `validation` (previously only in the ephemeral DB).
- **GAP 2 (no per-stage timing):** checksum/validation/segmentation each timed with
  `time.perf_counter()`, logged, and persisted as a `timings` block in `report.json` (built
  incrementally so a mid-run `_fail()` still records the stages that ran).
- **GAP 3 (#534 heartbeat never fired on native-L4):** TWO compounding bugs ?
  (a) `wasb_infer.py` never configured logging so its `logger.info` `done/total (pct%)` progress sat
  below root=WARNING and was dropped; (b) even when emitted it goes to **stderr** but
  `NativeWasbRunner` read only **stdout** (and draining stdout while stderr's ~64 KB pipe fills risks
  a deadlock). Fix: `wasb_infer._configure_logging()` ? INFO to **stdout**; runner merges
  stderr?stdout (line-buffered) + bounded failure tail. Callback now fires ? `run_telemetry.jsonl`
  snapshots + `[worker] progress:` heartbeat + `<video_id>_detector_progress.json` on GCS.

`report.json` changes are additive; CP reads it via `report.get(...)` (`probe_process_cache` / `cached_process_result`) so existing readers are unaffected.

Verified locally (Windows): `run_all_tests.py --cov=backend --cov=training --cov-fail-under=70`
? 1965 passed / 1 skipped, coverage 84.44% (worker `__main__.py` 91%); ruff 0.15.16 + mypy 2.1.0 clean.
No `test_storage.py` failures ? #548 (already on master) fixed those on Windows.

OPEN follow-up (not verifiable offline): the live-L4 heartbeat end-to-end (real GPU + `wasb` env +
Cloud Run L4) needs an **owner-driven CUJ** to confirm the telemetry snapshots / progress sidecar /

stdout heartbeat actually land. The offline tests lock the emission config, the stdout parse+merge
contract, and the report/timing serialization ? but not the real-hardware behavior. Related: [[ci-windows-coverage-gap]] (why the local Windows run matters), [[live-deployment-state]] (deploy the
image after merge to make this live).

120. user (2026-07-09T06:57:19.327Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md (file state is current in your context ? no need to Read it back)

121. user (2026-07-09T06:57:24.977Z)

1 # Memory index
2
3 - ? **Start here (GPU-resident rollout):** session-handoff-gpu-resident.md ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.
4 - ? **LIVE deploy (2026-07-09):** live-deployment-state.md ? CP rev **00023-spm** + worker on digest **1838c07f** (master **00e8f92**; ships #531/#535/#537/#538/#539/#529/#534/#548/#547 over 304af7b). **CP DOWNGRADED 4/16 ? 2 vCPU/8 GiB** (safe via #539 acquire-gating; val/seg/compile all on L4). Image-only bump preserves the baked cloud-serving/phase_placement config. Verified 200 on /api/config. Rollback ? digest **7b62a8df** @ 4/16 or rev **00022-29w**. Release-process gaps ? **#532**.
5 - ? **CLOSED 2026-07-08:** followup-validation-latency-live-cuj.md ? live SPA CUJ filled both CF-gated cells on rev **00019** (validation wall **183 s** PASS + reel compiled; fresh clip, corroborates not re-measures Video-1). Docs **PR #522**. Surfaced stitch bottleneck ? issues **#521** (compile?L4 + config placement) & **#520** (debug tracing).
6
7 - ? **Perf gate + nightly reorg (PR #547 MERGED 2026-07-09, squash `00e8f92f`):** perf-release-gate.md ? local release perf gate (prod vs HEAD, per-stage GPU timing, **stitch-only gating**) at `scripts/gates/perf/`; nightly CLIs consolidated into `scripts/gates/{perf,quality}/`; OS-aware (WSL on Windows, `detector_impl=native` on Linux). Finding: release **perf-neutral** (#531 within local noise). Related: the pre-existing Windows-only `test_storage` #535 failure (`task_2e0a3b82`) ? **FIXED in PR #548** (ci-windows-coverage-gap.md).
8
9 - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under `khelsutra-guru/badminton-highlight-indexer`.
10 - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`); diagnose-first, run long work detached.

122. user (2026-07-09T06:57:37.686Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

123. assistant (2026-07-09T06:57:59.835Z)

Done. PR [#550](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550) is open against `master`, branched from `origin/master @ 00e8f92`.

What was wrong and what I changed

All three gaps are fixed in one PR (`backend/worker/__main__.py`, `backend/pipeline/detectors/native_wasb_runner.py`, `backend/pipeline/detectors/wasb_infer.py`):

GAP 1 ? validation invisible on PASS. The worker now logs `[worker] validating (N checks)?` then `[worker] validation PASSED/FAILED in Ys ? <name>=ok/FAIL(score)?`, and the per-validator

verdicts land in the pushed `report.json` under `validation` (previously only in the ephemeral worker DB).

GAP 2 ? no per-stage timing. checksum / validation / segmentation are each timed with `perf_counter()`, logged, and persisted as a `timings` block in `report.json` ? built incrementally so a mid-run failure still records the stages that ran.

GAP 3 ? the #534 heartbeat never fired. This turned out to be **two compounding bugs**, which is why the CUJ saw *zero* snapshots:
- **Emission:** [`wasb_infer.py`](backend/pipeline/detectors/wasb_infer.py) never configured logging, so its `done/total (pct%)` progress went through `logger.info()` while root stayed at `WARNING` ? the lines were dropped entirely, not just misrouted.
- **Capture:** even when emitted they go to **stderr**, but [`native_wasb_runner.py`](backend/pipeline/detectors/native_wasb_runner.py) streamed **only stdout** (and draining stdout while the child fills its ~64 KB stderr pipe risks a deadlock).

Fix: `wasb_infer` configures INFO logging to **stdout** at entry, and the runner **merges stderr?stdout** (line-buffered) with a bounded tail for failure diagnostics. The callback now fires ? `run_telemetry.jsonl` snapshots, the `[worker] progress:` heartbeat, and the `_detector_progress.json` sidecar all flow.

`report.json` changes are additive ? the control-plane reads it via `report.get(...)`, so existing readers are unaffected.

Verified locally (Windows)

- `run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? **1965 passed, 1 skipped**, coverage **84.44%** (worker `__main__.py` 91%). No `test_storage.py` failures ? #548 is already on master.
- `ruff check .` (0.15.16) and `mypy backend training` (2.1.0) ? both clean.
- Added offline tests for the report/timing serialization, the `_val_token` score hint, the stderr-merge + failure-tail streaming path, and the `wasb_infer` stdout logging config.

Not verified (stated plainly)

The **live-L4 heartbeat end-to-end** ? real GPU + `wasb` conda env + Cloud Run L4 ? can't be exercised offline. The unit tests lock the emission config, the stdout parse/merge contract, and the serialization, but confirming the actual telemetry/sidecar/heartbeat on a real pass needs an **owner-driven CUJ**. I noted this in the PR body and in memory.

124. user (2026-07-09T07:12:46.258Z)

<cross-session-message from="local_5780afdd-5e0d-4851-8707-9826eb374e87" name="API performance comparison testing" encoded="1">
Heads-up before you start editing: your cwd `C:\Users\avidu\Projects\badminton-highlight-indexer` is the near-empty STUB (not a git repo). The REAL code is at `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` (see the `actual-repo-location` memory). Do all work there.

Concrete anchors for this task in the real repo:
- `backend/worker/__main__.py` ? validation runs ~line 290 (`validator_registry.run_all_with_context(...)`) but is SILENT on pass and results go only to the ephemeral DB, never report.json. Add per-stage log lines + wall-clock timing (validation / segment / compile) so runlogs can capture "validation took Ns". Also emit a pass summary, not just FAILED/SKIPPED.
- The `_progress` callback (~line 388, wired `progress=_progress` ~line 434) writes `&lt;video_id&gt;_detector_progress.json` + prints `[worker] progress:` ? but on the NATIVE-L4 path it never fires (heartbeat dead: run_telemetry.jsonl was just a 116-byte header on the 07-09 live CUJ). Wire the progress callback through `backend/pipeline/detectors/native_wasb_runner.py` so it actually emits on the native path.

Git safety (shared checkout, concurrent sessions): `git -C &lt;real-repo&gt; fetch origin` then `git checkout -b &lt;name&gt; origin/master`; stage explicit paths (never `git add -A`); before PR, `git log --oneline origin/master..HEAD` must list ONLY your commits. Full local gate before LGTM: run_all_tests.py (--cov-fail-under=70), ruff (pinned 0.15.16) + mypy clean. End commits with the Co-Authored-By: Claude Opus 4.8 trailer.

</cross-session-message>

125. assistant (2026-07-09T07:13:11.926Z)

This heads-up describes the task I've **already completed and shipped** ? it reads as pre-start context, but the work is done. Let me quickly confirm my branch and PR are still intact (concurrent-session sanity check) before responding.

126. user (2026-07-09T07:13:18.368Z)

```
From https://github.com/Khelsutra/badminton-highlight-indexer
   00e8f92..6cfcb37  master     -> origin/master
=== branch ===
fix/worker-l4-infer-observability
=== my commit vs origin/master ===
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
=== PR ===
#550 [OPEN] fix(worker): make the cloud-serving L4 infer path observable (fix/worker-l4-infer-observability)
```

127. assistant (2026-07-09T07:13:26.390Z)

My PR #550 is intact ? but `origin/master` moved (`00e8f92 ? 6cfcb37`) while I worked, so a sibling session merged something. Since the sibling that messaged me was working on the same task, I need to check whether they shipped a duplicate. Let me see what landed.

128. user (2026-07-09T07:13:28.471Z)

```
=== new commits on master ===
6cfcb37 docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ (2026-07-09) (#549)

=== files touched by those commits ===
 ..._cloud-serving_cp-2vcpu-downgrade_2026-07-09.md | 90 ++++++++++++++++++++++
 1 file changed, 90 insertions(+)
```

129. assistant (2026-07-09T07:13:55.478Z)

Confirmed ? no duplication or conflict. The new master commit is just [#549](https://github.com/Khelsutra/badminton-highlight-indexer/pull/549), a docs/runlog file, disjoint from my five files, so PR #550 still merges cleanly.

This task is already complete ? the heads-up arrived after I'd shipped it. What the sibling flagged matches exactly what I did, in the real repo (`khelsutra-guru/badminton-highlight-indexer`), branched from `origin/master @ 00e8f92`:

- **GAP 1** (validation silent on pass) ? added `[worker] validating (N checks)?` + a `PASSED/FAILED` summary with per-validator `name=ok/FAIL(score)`, and the verdicts now go into `report.json`, not just the ephemeral DB.
- **GAP 3** (native-L4 heartbeat dead ? the 116-byte header you saw) ? root-caused to two compounding bugs: `wasb_infer.py` never configured logging (progress `logger.info` suppressed under root=WARNING) **and** it emits to stderr while `native_wasb_runner.py` read only stdout. Fixed both (INFO?stdout in `wasb_infer`, stderr?stdout merge + line-buffering in the runner), so `_progress` fires ? `run_telemetry.jsonl` snapshots + `[worker] progress:` + the `_detector_progress.json` sidecar.
- Git safety followed (branched from `origin/master`, explicit path staging, only my commit `b3dff78` on the branch); full gate green (**1965 passed, 84.44% cov**, ruff 0.15.16 + mypy clean).

One scope nuance worth flagging: your note mentioned per-stage timing for "validation / segment / **compile**". My PR times **checksum / validation / segmentation** in the **infer** path (`cmd_infer`) ? which is what the task spec listed (GAP 2). The **compile** stage lives in a separate subcommand (`cmd_compile`, the #521 L4 stitch) and wasn't in this task's scope; its observability is tracked under #520/#521. It's a ~10-line follow-up to add the same `timings`/log pattern to `cmd_compile` if you want it in this PR ? say the word and I'll fold it

in. Otherwise #550 is ready for review as-is.

130. user (2026-07-09T07:23:07.651Z)

pls review comments on PR

131. assistant (2026-07-09T07:23:17.050Z)

The user wants me to review the comments on PR #550. Let me fetch all of them ? inline review comments, review summaries, and the conversation.

132. user (2026-07-09T07:23:24.505Z)

```
======== REVIEWS (summaries) ========
--- avidullu [COMMENTED] @ 2026-07-09T07:23:07Z
Review submitted: 1 bug, 3 suggestions, 1 nit. Overall approve-with-notes on the three L4
observability fixes; the main residual risk is the streaming Popen path not enforcing
timeout_sec.


======== INLINE REVIEW COMMENTS ========
--- avidullu on backend/pipeline/detectors/native_wasb_runner.py:386 (2026-07-09T07:22:34Z)
**[bug]** The live-progress branch (entered whenever `progress is not None`) launches the child
with `subprocess.Popen` and then blocks on `for line in proc.stdout` / `proc.wait()` with no
timeout. `NativeWasbConfig.timeout_sec` is only applied on the fast path via `_run()` ?
`subprocess.run(..., timeout=...)`. The worker always passes a progress callback
(`trajectory_hybrid` ? `run_predict(..., progress=progress)`), so the cloud native-L4 path this
PR is about never enforces the configured 30?minute kill. The surrounding `except
subprocess.TimeoutExpired` is therefore dead code for production streaming, and the new
comment?s ?hangs until timeout? framing is misleading. A child that stops producing output (or
wedges after closing pipes) can block the worker indefinitely.

**Suggestion:** Enforce `timeout_sec` on the streaming path as well ? e.g. track `deadline =
time.monotonic() + timeout`, use a reader thread/`select` with a short poll, or
`proc.wait(timeout=remaining)` in a loop and `kill()` + raise/return on expiry (mirroring
`_run`). Add a unit test that a fake Popen which never closes stdout is killed when
`timeout_sec` is small.

--- avidullu on backend/worker/__main__.py:359 (2026-07-09T07:22:34Z)
**[suggestion]** `n_checks = len(getattr(validator_registry, "_validators", []))` reaches into a
private registry attribute. That works today, but it is not part of the public validator-
registry API and will silently report `0 checks` if the internal field is renamed or if
validators are stored differently.

**Suggestion:** Prefer a public helper (e.g. `len(validator_registry)` / `list_registered()` if
one exists or is added next to `register`), or log without a count (`[worker] validating?`) to
avoid coupling to private state.

--- avidullu on backend/worker/__main__.py:369 (2026-07-09T07:22:34Z)
**[suggestion]** On validation failure the new summary line (`validation FAILED in ? ?
name=FAIL(score)`) is always followed by the pre-existing failures print later in `cmd_infer`
(`validation FAILED: name: message`). Operators get two nearly-duplicate FAIL lines with
different formats in one runlog, which slightly muddies the ?loud single verdict? goal.

**Suggestion:** Either drop/replace the older failures print now that the summary line exists,
or make the new line the sole PASS path log and keep the detailed `name: message` form only on
FAIL (not both).

--- avidullu on backend/worker/__main__.py:330 (2026-07-09T07:22:34Z)
**[suggestion]** Stage timings cover checksum / validation / segmentation but not the storage
pull (`storage.fetch` of the input video), which is often a large fraction of cloud wall-clock
for multi?GB clips. The same mid-run `_fail()` path therefore still omits the stage that may
dominate before checksum starts.
```

Suggestion: Time the fetch into something like `timings["fetch_s"]` (and optionally push) so runlogs explain full end-to-end latency, not only post-download stages.

--- avidullu on backend/worker/__main__.py:374 (2026-07-09T07:22:34Z)
[nit] On `--skip-validation`, `validation_payload` is `[]` and is still written as `"validation": []` (because the gate is `is not None`). That is backward-compatible and distinct from omitting the key, but a later CP/P3 consumer cannot tell ?dispatcher already validated (skipped here)? from ?worker ran zero validators.?

Suggestion: Optionally add `"validation_skipped": true` (or omit `validation` when skipped) so future control-plane ingest of worker verdicts does not treat an empty list as an authoritative empty suite. Not blocking for this observability-only PR.

======== ISSUE/CONVERSATION COMMENTS ========

133. assistant (2026-07-09T07:27:39.132Z)

The owner (avidullu) left a substantive review: 1 bug, 3 suggestions, 1 nit. Let me address all of them ? the timeout bug is the "main residual risk" and must be fixed. First, a quick check for the `__len__` truthiness concern before I add a public registry accessor.

134. user (2026-07-09T07:27:41.647Z)

No matches found

135. assistant (2026-07-09T07:27:49.126Z)

No boolean use of the registry anywhere ? `__len__` is safe. Now let me implement all the review fixes.

Fix 1 (the bug): enforce `timeout_sec` on the streaming path. Add `threading`, and use a watchdog Timer that kills the child at the deadline ? the kill closes stdout, the read loop hits EOF, and I raise `TimeoutExpired` through the existing handler (making it live for this path, not dead).

136. user (2026-07-09T07:27:51.321Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

137. user (2026-07-09T07:28:15.033Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

138. assistant (2026-07-09T07:28:21.101Z)

Let me verify the full streaming block + exception handlers are coherent after the edit:

139. user (2026-07-09T07:28:21.878Z)

```
367
368             res: Optional[subprocess.CompletedProcess] = None
369             try:
370                 if progress is None:
371                     # Fast path: capture at end (existing behaviour, byte-identical for
tests).
372                     res = self._run(argv, cwd=self._repo_src())
373                 else:
374                     # Live progress path (#534): stream the child's output so we can parse
the
```

```
                    # periodic "done/total (pct%) ..." lines wasb_infer emits and fire the
callback ?
                    # a real heartbeat during the long GPU pass instead of a spinner.
                    #
                    # MERGE stderr INTO stdout (stderr=STDOUT). wasb_infer routes its
logging (progress
                    # included) to a stream, and third-party libs may also write to stderr.
Reading
                    # stdout alone would (a) miss any progress that lands on stderr and (b)
risk a
                    # DEADLOCK: draining only stdout while the child fills its ~64 KB stderr
pipe blocks
                    # the child on the stderr write, which stops its stdout too ? the run
then hangs
                    # until timeout. One merged pipe removes both failure modes (the
2026-07-09 live CUJ
                    # produced zero heartbeat lines). bufsize=1 = line-buffered so the
heartbeat is
                    # timely, not block-delayed. A bounded tail keeps the last lines for the
failure
                    # message, since there is no separate stderr to read on error.
                    proc = subprocess.Popen(
                        argv,
                        cwd=self._repo_src(),
                        stdout=subprocess.PIPE,
                        stderr=subprocess.STDOUT,
                        text=True,
                        bufsize=1,
                    )
                    assert proc.stdout is not None
                    tail: "deque[str]" = deque(maxlen=200)
                    # TIMEOUT: the blocking ``for line in proc.stdout`` read has NO built-in
deadline, so a
                    # wedged child (hung GPU call, or one that closed its pipes without
exiting) would
                    # block the worker indefinitely ? the fast ``_run()`` path's
``timeout_sec`` is not in
                    # force here, and this IS the production path (the worker always passes
a progress
                    # callback). A watchdog Timer kills the child at ``timeout_sec``; the
kill closes its
                    # stdout, the read loop hits EOF, and we surface a timeout via the SAME
                    # ``TimeoutExpired`` handler as ``_run`` (return None, log "timed out
after Ns").
                    timed_out = threading.Event()
                    watchdog: Optional[threading.Timer] = None
                    if self.cfg.timeout_sec:

                        def _kill_on_timeout() -> None:
                            timed_out.set()
                            try:
                                proc.kill()
                            except Exception:  # noqa: BLE001 - the run is already being
torn down
                                pass

                        watchdog = threading.Timer(self.cfg.timeout_sec, _kill_on_timeout)
                        watchdog.daemon = True
                        watchdog.start()
                    try:
                        for line in proc.stdout:
                            line = line.rstrip()
                            if not line:
                                continue
                            tail.append(line)
```

```
424                             # Parse wasb_infer progress lines for the detector heartbeat.
425                             # Example: "1234/5000 (24%) | 2.3 win/s | elapsed 45s | ETA
120s"
426                             m = re.search(r"(\d+)/(\d+)\s+\((\d+)%\)", line)
427                             if m:
428                                 done, total, pct = m.groups()
429                                 try:
430                                     progress(f"detector {pct}% ({done}/{total})")
431                                 except Exception:  # noqa: BLE001
432                                     pass  # progress must never kill the run
433                             # Mirror to the container/runlog.
434                             logger.info(line)
435                         proc.wait()
436                         res = subprocess.CompletedProcess(
437                             args=argv,
438                             returncode=proc.returncode,
439                             stdout="",
440                             stderr="\n".join(tail),
441                         )
```

140. user (2026-07-09T07:28:25.926Z)

```
441                         )
442                     finally:
443                         if watchdog is not None:
444                             watchdog.cancel()
445                         if proc.poll() is None:
446                             proc.kill()
447                             proc.wait()
448                     if timed_out.is_set():
449                         # Route through the outer handler so the streaming path enforces
timeout_sec
450                         # exactly like _run (no longer dead code for the production
native-L4 path).
451                         raise subprocess.TimeoutExpired(argv, self.cfg.timeout_sec)
452             except FileNotFoundError:
453                 logger.error(
454                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
455                 )
456                 return None
457             except subprocess.TimeoutExpired:
458                 logger.error(
459                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec
460                 )
461                 return None
462
463             if res is None or res.returncode != 0:
464                 logger.error(
465                     "WASB native inference failed:\n%s",
466                     (getattr(res, "stderr", None) or getattr(res, "stdout", None) or
"")[-2000:],
467                 )
468                 return None
469             if not os.path.exists(out_csv):
470                 logger.error(
```

141. assistant (2026-07-09T07:28:35.419Z)

The timeout fix is coherent ? it routes through the existing `TimeoutExpired` handler, making it live for the streaming path.

Fix 2: public registry count. Add `__len__` to `ValidationRegistry`:

142. user (2026-07-09T07:28:38.508Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\validators\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

143. assistant (2026-07-09T07:28:53.025Z)

Fix 4: time the storage pull (`fetch_s`) ? it can dominate wall-clock for multi-GB clips, and the mid-run `_fail()` should capture it. Moving `timings` init above the pull:

144. user (2026-07-09T07:29:02.247Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

145. assistant (2026-07-09T07:29:18.210Z)

Fixes 2, 3, 5 in the validation block: public `len(validator_registry)`, a `skipped` flag driving `validation_skipped` in the report (instead of an ambiguous `[]`), and setting up to reword the duplicate FAIL line:

146. user (2026-07-09T07:29:27.378Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

147. assistant (2026-07-09T07:29:34.873Z)

Fix 3: reword the duplicate FAIL line so it complements the verdict summary instead of repeating "validation FAILED":

148. user (2026-07-09T07:29:38.121Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

149. assistant (2026-07-09T07:29:44.498Z)

Fix 5: `validation_skipped` in the report writer. Add the param and emit it:

150. user (2026-07-09T07:29:56.241Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

151. assistant (2026-07-09T07:30:05.470Z)

Now thread `validation_skipped` through `_serialize_and_push_outputs`:

152. user (2026-07-09T07:30:16.328Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

153. assistant (2026-07-09T07:30:25.428Z)

Now update the two push sites (the `_fail` closure and the success path) to pass `report_validation` + `validation_skipped`:

154. user (2026-07-09T07:30:28.286Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

155. user (2026-07-09T07:30:34.294Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

156. assistant (2026-07-09T07:31:24.945Z)

```
All source fixes are in. Now let me update and add the tests. Let me check the validators-base
test file's style first, then update the worker/native tests.
```

157. user (2026-07-09T07:31:25.948Z)

```
1       """Tests for the validation registry, focused on the backward-compatible
2       `run_all_with_context` added for observability reporting."""
3
4       from backend.config.models import AppConfig
5       from backend.pipeline.validators.base import (
6           ValidationRegistry,
7           ValidationResult,
8           VideoValidator,
9       )
10
11
12      class _ProducerValidator(VideoValidator):
13          @property
14          def name(self):
15              return "producer"
16
17          @property
18          def description(self):
19              return "writes ffprobe_meta into the shared context"
20
21          def validate(self, video_path, config, context):
22              context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
23              return ValidationResult(validator_name=self.name, passed=True, message="ok")
24
25
26      class _ConsumerValidator(VideoValidator):
27          @property
28          def name(self):
29              return "consumer"
30
31          @property
32          def description(self):
33              return "reads ffprobe_meta the producer cached"
34
35          def validate(self, video_path, config, context):
36              seen = "ffprobe_meta" in context
37              return ValidationResult(
38                  validator_name=self.name, passed=seen, message=f"saw_meta={seen}"
39              )
40
```

158. user (2026-07-09T07:31:33.870Z)

```
        @property
        def description(self):
```

```
            return "Tests safe instance registration for validators with complex __init__
state."

        def validate(self, video_path, config, context):
            # Use the state to "prove" it was initialized once
            return ValidationResult(
                validator_name=self.name,
                passed=len(self._heavy_state) > 0,
                message="ok",
            )

    r = ValidationRegistry()
    v = ComplexValidator()
    r.register(v)

    results, _ = r.run_all_with_context("v.mp4", {})
    assert len(results) == 1
    assert results[0].validator_name == "complex-validator"
    assert results[0].passed is True
```

159. user (2026-07-09T07:31:39.542Z)

```
appended
```

160. assistant (2026-07-09T07:31:45.170Z)

```
Now the native-runner timeout test ? appending after the streaming tests:
```

161. user (2026-07-09T07:31:54.282Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_native_wasb_runner.py has been updated successfully. (file state is current
in your context ? no need to Read it back)
```

162. assistant (2026-07-09T07:32:11.313Z)

```
Now update the worker tests for `fetch_s`, the de-duplicated FAIL line, and the
`validation_skipped` marker. First the PASS test's timings set:
```

163. user (2026-07-09T07:32:14.615Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

164. user (2026-07-09T07:32:23.095Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

165. assistant (2026-07-09T07:32:26.317Z)

```
Now the `validation_skipped` assertions ? extend the omit test and the existing skip-bypass
test:
```

166. user (2026-07-09T07:32:28.808Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

167. user (2026-07-09T07:32:35.046Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

168. assistant (2026-07-09T07:32:40.429Z)

All fixes and tests are in. Let me run the affected test files, then the full gate.

169. user (2026-07-09T07:32:48.605Z)

```
.................................................................... [ 66%]
..................................                                   [100%]
108 passed in 4.78s
```

170. assistant (2026-07-09T07:32:57.709Z)

108 pass (including the 0.3s timeout test). Now ruff + mypy:

171. user (2026-07-09T07:32:59.444Z)

```
All checks passed!
```

172. user (2026-07-09T07:33:03.636Z)

```
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
```

173. assistant (2026-07-09T07:33:11.457Z)

Ruff + mypy clean. Now the full suite with coverage (the pre-push gate):

174. user (2026-07-09T07:39:57.340Z)

```
backend\worker\__init__.py                            0      0   100%
backend\worker\__main__.py                          463     43    91%
training\__init__.py                                  0      0   100%
training\gen0\__init__.py                             0      0   100%
training\gen0\harness.py                            130      9    93%
-----------------------------------------------------------------------
TOTAL                                             13607   2116    84%
Required test coverage of 70% reached. Total coverage: 84.45%
1967 passed, 1 skipped, 57 warnings in 401.43s (0:06:41)
=======================================================================
 RESULT: PASS - all tests green
=======================================================================
```

175. assistant (2026-07-09T07:40:14.658Z)

Full suite: **1967 passed, 1 skipped**, coverage **84.45%**, worker `__main__.py` 91%. ruff + mypy clean. All five review items addressed and verified.

Let me re-verify branch safety, then stage explicit paths and commit the review fixes.

176. user (2026-07-09T07:40:19.736Z)

```
=== branch ===
fix/worker-l4-infer-observability
=== dirty files ===
 M backend/pipeline/detectors/native_wasb_runner.py
 M backend/pipeline/validators/base.py
```

```
 M backend/worker/__main__.py
 M tests/test_native_wasb_runner.py
 M tests/test_validators_base.py
 M tests/test_worker.py
?? "SESSION_HANDOFF (1).md"
=== my commits vs origin/master (b3dff78 should be the only one so far) ===
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
```

177. assistant (2026-07-09T07:40:25.184Z)

Clean state. Staging the 6 explicit paths (the stray untracked handoff excluded) and committing:

178. user (2026-07-09T07:40:26.909Z)

```
warning: in the working copy of 'tests/test_validators_base.py', LF will be replaced by CRLF the
next time Git touches it
=== staged ===
M  backend/pipeline/detectors/native_wasb_runner.py
M  backend/pipeline/validators/base.py
M  backend/worker/__main__.py
M  tests/test_native_wasb_runner.py
M  tests/test_validators_base.py
M  tests/test_worker.py
?? "SESSION_HANDOFF (1).md"
```

179. assistant (2026-07-09T07:40:31.707Z)

Staged (the LF/CRLF warning is benign normalization). Committing:

180. user (2026-07-09T07:40:43.070Z)

```
[fix/worker-l4-infer-observability e54e4a1] fix(worker): address PR review ? streaming timeout +
observability polish
 6 files changed, 153 insertions(+), 19 deletions(-)
```

181. assistant (2026-07-09T07:40:46.972Z)

Committed `e54e4a1`. Verifying my commits and pushing:

182. user (2026-07-09T07:40:50.493Z)

```
=== my commits vs origin/master (both mine) ===
e54e4a1 fix(worker): address PR review ? streaming timeout + observability polish
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
=== push ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   b3dff78..e54e4a1  fix/worker-l4-infer-observability -> fix/worker-l4-infer-observability
```

183. assistant (2026-07-09T07:40:57.703Z)

Both commits pushed cleanly. Now let me post a reply on the PR mapping each review finding to its resolution:

184. user (2026-07-09T07:41:12.136Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#issuecomment-4922778714

185. assistant (2026-07-09T07:41:22.879Z)

Review response posted. Let me update the memory to reflect the review round for future sessions.

186. user (2026-07-09T07:41:31.872Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

187. user (2026-07-09T07:41:40.466Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

188. assistant (2026-07-09T07:41:58.251Z)

Done. I reviewed the owner's review on [PR #550](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550) ? **1 bug, 3 suggestions, 1 nit** ? and addressed all five in commit `e54e4a1` (pushed), with a per-item [response comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#issuecomment-4922778714).

The findings and how I resolved each

? [BUG] streaming path ignored `timeout_sec` ? the real risk. The live-progress branch is the *production* native-L4 path (the worker always passes a callback), yet `for line in proc.stdout` / `wait()` had no deadline ? a wedged child could hang the worker forever, and the `except subprocess.TimeoutExpired` was dead code there. Fixed with a watchdog `threading.Timer` that kills the child at `timeout_sec`; the kill closes stdout ? the read hits EOF ? I raise through the **same** `TimeoutExpired` handler as `_run` (`return None`, logs "timed out after Ns"). New test drives a fake child that blocks until the watchdog kills it.

[suggestion] private `_validators` access ? added `ValidationRegistry.__len__` and use `len(validator_registry)` (verified nothing relies on registry truthiness).

[suggestion] two near-duplicate FAIL lines ? the detail print is reworded to `[worker] validation rejected by: ?`, complementing the `validation FAILED in Ys ? ?` verdict line; test locks a single-verdict occurrence.

[suggestion] pull stage untimed ? added `timings["fetch_s"]` (often the dominant term for multi-GB clips; a mid-run `_fail()` now captures it).

[nit] ambiguous `validation: []` on skip ? skip now writes `validation_skipped: true` and omits the array, so a consumer can tell "dispatcher already validated" from "zero-validator suite".

Verification

Re-ran the full gate: **1967 passed / 1 skipped**, coverage **84.45%** (floor 70), ruff 0.15.16 + mypy 2.1.0 clean. Two commits on the branch, both mine, on top of `origin/master`.

The one unchanged caveat I flagged in the response: the **live-L4 heartbeat end-to-end** still needs an owner-driven CUJ to confirm on real hardware ? everything else is verified offline.

189. user (2026-07-09T07:50:09.883Z)

ptal more comments on the PR

190. assistant (2026-07-09T07:50:14.932Z)

Let me fetch the latest comments and reviews on the PR.

191. user (2026-07-09T07:50:21.784Z)

```
======== REVIEWS (summaries, chronological) ========
--- avidullu [COMMENTED] @ 2026-07-09T07:23:07Z
Review submitted: 1 bug, 3 suggestions, 1 nit. Overall approve-with-notes on the three L4 observability fixes; the main residual risk is the streaming Popen path not enforcing timeout_sec.
```

--- avidullu [COMMENTED] @ 2026-07-09T07:47:45Z
Second-pass review submitted: all five prior items resolved. One residual suggestion on a narrow timeout-watchdog race (success can lose to a late Timer fire); not a ship-blocker at the default 1800s budget.

======== INLINE REVIEW COMMENTS ========
--- avidullu on backend/pipeline/detectors/native_wasb_runner.py:387 @ 2026-07-09T07:22:34Z (commit b3dff78)
[bug] The live-progress branch (entered whenever `progress is not None`) launches the child with `subprocess.Popen` and then blocks on `for line in proc.stdout` / `proc.wait()` with no timeout. `NativeWasbConfig.timeout_sec` is only applied on the fast path via `_run()` ? `subprocess.run(..., timeout=...)`. The worker always passes a progress callback (`trajectory_hybrid` ? `run_predict(..., progress=progress)`), so the cloud native-L4 path this PR is about never enforces the configured 30?minute kill. The surrounding `except subprocess.TimeoutExpired` is therefore dead code for production streaming, and the new comment?s ?hangs until timeout? framing is misleading. A child that stops producing output (or wedges after closing pipes) can block the worker indefinitely.

Suggestion: Enforce `timeout_sec` on the streaming path as well ? e.g. track `deadline = time.monotonic() + timeout`, use a reader thread/`select` with a short poll, or `proc.wait(timeout=remaining)` in a loop and `kill()` + raise/return on expiry (mirroring `_run`). Add a unit test that a fake Popen which never closes stdout is killed when `timeout_sec` is small.

--- avidullu on backend/worker/__main__.py:359 @ 2026-07-09T07:22:34Z (commit b3dff78)
[suggestion] `n_checks = len(getattr(validator_registry, "_validators", []))` reaches into a private registry attribute. That works today, but it is not part of the public validator-registry API and will silently report `0 checks` if the internal field is renamed or if validators are stored differently.

Suggestion: Prefer a public helper (e.g. `len(validator_registry)` / `list_registered()` if one exists or is added next to `register`), or log without a count (`[worker] validating?`) to avoid coupling to private state.

--- avidullu on backend/worker/__main__.py:384 @ 2026-07-09T07:22:34Z (commit b3dff78)
[suggestion] On validation failure the new summary line (`validation FAILED in ? ? name=FAIL(score)`) is always followed by the pre-existing failures print later in `cmd_infer` (`validation FAILED: name: message`). Operators get two nearly-duplicate FAIL lines with different formats in one runlog, which slightly muddies the ?loud single verdict? goal.

Suggestion: Either drop/replace the older failures print now that the summary line exists, or make the new line the sole PASS path log and keep the detailed `name: message` form only on FAIL (not both).

--- avidullu on backend/worker/__main__.py:332 @ 2026-07-09T07:22:34Z (commit b3dff78)
[suggestion] Stage timings cover checksum / validation / segmentation but not the storage pull (`storage.fetch` of the input video), which is often a large fraction of cloud wall-clock for multi?GB clips. The same mid-run `_fail()` path therefore still omits the stage that may dominate before checksum starts.

Suggestion: Time the fetch into something like `timings["fetch_s"]` (and optionally push) so runlogs explain full end-to-end latency, not only post-download stages.

--- avidullu on backend/worker/__main__.py:392 @ 2026-07-09T07:22:34Z (commit b3dff78)
[nit] On `--skip-validation`, `validation_payload` is `[]` and is still written as `"validation": []` (because the gate is `is not None`). That is backward-compatible and distinct from omitting the key, but a later CP/P3 consumer cannot tell ?dispatcher already validated (skipped here)? from ?worker ran zero validators.?

Suggestion: Optionally add `"validation_skipped": true` (or omit `validation` when skipped) so future control-plane ingest of worker verdicts does not treat an empty list as an authoritative empty suite. Not blocking for this observability-only PR.

--- avidullu on backend/pipeline/detectors/native_wasb_runner.py:409 @ 2026-07-09T07:47:13Z

(commit e54e4a1)
[suggestion] The watchdog callback always does `timed_out.set()` before `proc.kill()`, and the post-loop check is an unconditional `if timed_out.is_set(): raise TimeoutExpired`. If the child exits successfully (rc=0, CSV written) in the narrow window after `proc.wait()` / before `watchdog.cancel()`, a late timer fire can still set `timed_out` (while `Popen.kill()` is a no-op once `returncode` is set). The run is then reported as a timeout (`return None`) despite a clean completion ? success loses the race to the timer. Cancel-on-completion and daemon Timer are otherwise correct; kill?EOF?raise path works for a truly wedged child; no timer leak when the process dies first.

Suggestion: Only claim timeout when the child was still alive (and/or when completion was not clean), e.g. set the flag only if `proc.poll() is None` before kill, and/or after the try/finally treat success as winning the race: `if timed_out.is_set() and not (res is not None and res.returncode == 0): raise TimeoutExpired(...)`. Optionally log the merged `tail` on the timeout path (today `res.stderr` is built then discarded by the raise) for post-mortem context.

======== ISSUE/CONVERSATION COMMENTS ========
--- avidullu @ 2026-07-09T07:41:13Z
Thanks for the review ? all five addressed in `e54e4a1`.

[bug] streaming path ignored `timeout_sec` (`native_wasb_runner.py`). Fixed. The live-progress branch now arms a watchdog `threading.Timer(timeout_sec, ?)` that `proc.kill()`s the child at the deadline; the kill closes stdout ? the `for line in proc.stdout` read hits EOF ? I `raise subprocess.TimeoutExpired` so it flows through the **same** `except` handler as `_run` (`return None`, `"timed out after Ns"`). So that handler is no longer dead code for the production native-L4 path, and a silent/wedged child is killed instead of hanging. New test `test_run_predict_streaming_enforces_timeout` drives a fake child whose stdout read blocks until the watchdog kills it and asserts `out is None` + the kill fired + the timeout was logged.

[suggestion] private `_validators` access (`__main__.py:359`). Fixed. Added a public `ValidationRegistry.__len__` and switched to `len(validator_registry)`. (Confirmed nothing uses the registry in a boolean context, so `__len__` introduces no truthiness surprise.) Test: `test_registry_len_counts_registered_validators`.

[suggestion] two near-duplicate FAIL lines (`__main__.py:369`). Fixed. The verdict line stays `[worker] validation FAILED in Ys ? name=FAIL(score)`; the pre-existing detail print is reworded to `[worker] validation rejected by: name: message` so it complements rather than repeats. The FAIL test now asserts `count("[worker] validation FAILED") == 1` plus the distinct `rejected by:` line.

[suggestion] pull stage untimed (`__main__.py:330`). Fixed. `storage.fetch` is now timed into `timings["fetch_s"]` (and logged on the pull line), so a mid-run `_fail()` explains latency that accrued before checksum ? often the dominant term for multi-GB clips.

[nit] ambiguous `validation: []` on skip (`__main__.py:374`). Fixed. On `--skip-validation` the report now carries `validation_skipped: true` and omits the `validation` array, so a CP/P3 consumer can distinguish "dispatcher already validated" from "worker ran a zero-validator suite". Tests updated for the skip report shape.

Local gate re-run green: **1967 passed / 1 skipped**, coverage **84.45%** (floor 70), ruff 0.15.16 + mypy 2.1.0 clean. The live-L4 heartbeat end-to-end still needs an owner CUJ (unchanged caveat).

192. assistant (2026-07-09T07:52:32.557Z)

Good ? the second pass confirms all five prior items resolved, with **one new, narrow correctness finding**: a watchdog race where a clean success finishing right at the deadline could be misreported as a timeout (the timer sets `timed_out` after `res` is built, and my post-check raises unconditionally). It's real, so I'll fix it properly ? success must win the race. Let me apply the reviewer's suggested guards.

193. user (2026-07-09T07:52:37.709Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-

indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

194. assistant (2026-07-09T07:52:40.990Z)

Now the post-check ? success wins the race, and carry the tail for post-mortem:

195. user (2026-07-09T07:52:45.051Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

196. assistant (2026-07-09T07:52:48.385Z)

Now the handler ? log the tail on timeout:

197. user (2026-07-09T07:52:52.031Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

198. assistant (2026-07-09T07:53:16.376Z)

Now update the timeout test to also verify the tail is surfaced, and add a test for the success-wins-the-race case:

199. user (2026-07-09T07:53:32.338Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_native_wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

200. assistant (2026-07-09T07:53:41.890Z)

Let me run the affected tests plus ruff/mypy:

201. user (2026-07-09T07:53:55.488Z)

```
..................................................................... [ 66%]
.....................................                               [100%]
109 passed in 4.81s
======== RUFF ========
All checks passed!
======== MYPY ========
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
```

202. assistant (2026-07-09T07:54:03.087Z)

109 passed (+1 race test), ruff + mypy clean. Running the full suite as the pre-push gate:

203. user (2026-07-09T08:00:36.295Z)

```
training\gen0\harness.py                          130      9    93%
```

```
----------------------------------------------------------------------
TOTAL                                              13609   2114    84%
Required test coverage of 70% reached. Total coverage: 84.47%
1968 passed, 1 skipped, 57 warnings in 388.81s (0:06:28)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

204. assistant (2026-07-09T08:00:42.502Z)

Full suite green: **1968 passed, 1 skipped**, coverage **84.47%**. Let me self-review the final race-fix diff, then commit.

205. user (2026-07-09T08:00:43.706Z)

```
diff --git a/backend/pipeline/detectors/native_wasb_runner.py
b/backend/pipeline/detectors/native_wasb_runner.py
index 5cbf3a4..d48aea5 100644
--- a/backend/pipeline/detectors/native_wasb_runner.py
+++ b/backend/pipeline/detectors/native_wasb_runner.py
@@ -406,11 +406,15 @@ class NativeWasbRunner(DetectorRunner):
                 if self.cfg.timeout_sec:

                     def _kill_on_timeout() -> None:
-                        timed_out.set()
-                        try:
-                            proc.kill()
-                        except Exception:  # noqa: BLE001 - the run is already being torn down
-                            pass
+                        # Only claim a timeout if the child is STILL running: a late fire in
the
+                        # window after a clean exit must not flag `timed_out` (proc.kill() is a
+                        # no-op once the process has a returncode), so a success can't be
misreported.
+                        if proc.poll() is None:
+                            timed_out.set()
+                            try:
+                                proc.kill()
+                            except Exception:  # noqa: BLE001 - the run is already tearing down
+                                pass

                     watchdog = threading.Timer(self.cfg.timeout_sec, _kill_on_timeout)
                     watchdog.daemon = True
@@ -445,18 +449,29 @@ class NativeWasbRunner(DetectorRunner):
                     if proc.poll() is None:
                         proc.kill()
                         proc.wait()
-                if timed_out.is_set():
-                    # Route through the outer handler so the streaming path enforces
timeout_sec
-                    # exactly like _run (no longer dead code for the production native-L4
path).
-                    raise subprocess.TimeoutExpired(argv, self.cfg.timeout_sec)
+                # A CLEAN completion wins the race against a late timer fire: only surface a
timeout
+                # when the run did NOT finish cleanly (rc 0). Route through the outer handler
so the
+                # streaming path enforces timeout_sec exactly like _run (that handler is no
longer
+                # dead code for the production native-L4 path); carry the merged tail for a
post-mortem.
+                if timed_out.is_set() and not (res is not None and res.returncode == 0):
+                    raise subprocess.TimeoutExpired(
+                        argv, self.cfg.timeout_sec, stderr="\n".join(tail)
+                    )
```

```
         except FileNotFoundError:
             logger.error(
                 "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
             )
             return None
-        except subprocess.TimeoutExpired:
+        except subprocess.TimeoutExpired as e:
+            # Surface the merged output tail (the streaming path passes it as `stderr`) so a
timeout
+            # is diagnosable ? not just "timed out after Ns".
+            detail = next(
+                (p for p in (e.stderr, e.output) if isinstance(p, str) and p), ""
+            )
             logger.error(
-                "WASB native inference timed out after %ss.", self.cfg.timeout_sec
+                "WASB native inference timed out after %ss.%s",
+                self.cfg.timeout_sec,
+                f"\nlast output:\n{detail[-2000:]}" if detail else "",
             )
             return None
```

206. assistant (2026-07-09T08:00:49.144Z)

Clean and coherent ? all three parts of the finding addressed. Let me verify branch state, then commit and push.

207. user (2026-07-09T08:00:53.826Z)

```
=== branch ===
fix/worker-l4-infer-observability
=== dirty (mine) ===
 M backend/pipeline/detectors/native_wasb_runner.py
 M tests/test_native_wasb_runner.py
?? "SESSION_HANDOFF (1).md"
=== my commits vs origin/master ===
e54e4a1 fix(worker): address PR review ? streaming timeout + observability polish
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
```

208. user (2026-07-09T08:01:05.599Z)

```
[fix/worker-l4-infer-observability f7f0c82] fix(detectors): close the timeout-watchdog race
(success must win a late fire)
 2 files changed, 87 insertions(+), 13 deletions(-)
```

209. user (2026-07-09T08:01:11.972Z)

```
=== my commits vs origin/master (all mine) ===
f7f0c82 fix(detectors): close the timeout-watchdog race (success must win a late fire)
e54e4a1 fix(worker): address PR review ? streaming timeout + observability polish
b3dff78 fix(worker): make the cloud-serving L4 infer path observable
=== push ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   e54e4a1..f7f0c82  fix/worker-l4-infer-observability -> fix/worker-l4-infer-observability
```

210. assistant (2026-07-09T08:01:15.188Z)

Pushed (`f7f0c82`). Now posting the response to the race finding:

211. user (2026-07-09T08:01:26.086Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#issuecomment-4922925046

212. assistant (2026-07-09T08:01:31.476Z)

Comment posted. Let me update the memory with the race fix.

213. user (2026-07-09T08:01:39.139Z)

<tool_use_error>String to replace not found in file.
String: `Ys ? ?`). (4) time the storage pull ? `timings["fetch_s"]` (often the dominant term for multi-GB
clips; mid-run `_fail()` now captures it). (5) skip path writes `validation_skipped: true` (omits the
`validation` array) instead of an ambiguous `validation: []`.
(note: Edit also tried swapping \uXXXX escapes and their characters; neither form matched, so the mismatch is likely elsewhere in old_string. Re-read the file and copy the exact surrounding text.)</tool_use_error>

214. assistant (2026-07-09T08:01:42.228Z)

Let me re-read the end of the memory file to match exactly.

215. user (2026-07-09T08:01:43.523Z)

```
36      ? 1967 passed / 1 skipped, coverage 84.45% (worker `__main__.py` 91%); ruff 0.15.16 +
mypy 2.1.0 clean.
37      No `test_storage.py` failures ? #548 (already on master) fixed those on Windows.
38
39      **Review round addressed (commit `e54e4a1`)** ? owner review (1 bug, 3 suggestions, 1
nit) all fixed:
40      (1) **BUG:** the streaming (progress) path ? the PRODUCTION native-L4 path, worker
always passes a
41      callback ? enforced NO `timeout_sec`; a wedged child could hang forever. Now a watchdog
42      `threading.Timer` kills the child at the deadline ? EOF ? raise through the same
`TimeoutExpired`
43      handler as `_run` (that handler was dead code for streaming before). (2)
`len(validator_registry)`
44      via a new `ValidationRegistry.__len__` (no more private `_validators`). (3) de-duped
FAIL logs ? the
45      detail line is now `[worker] validation rejected by: ?` (verdict line keeps `validation
FAILED in
46      Ys ? ?`). (4) time the storage pull ? `timings["fetch_s"]` (often the dominant term for
multi-GB
47      clips; mid-run `_fail()` now captures it). (5) skip path writes `validation_skipped:
true` (omits the
```

216. user (2026-07-09T08:01:46.861Z)

```
47      clips; mid-run `_fail()` now captures it). (5) skip path writes `validation_skipped:
true` (omits the
48      `validation` array) instead of an ambiguous `validation: []`.
49
50      **OPEN follow-up (not verifiable offline):** the live-L4 heartbeat end-to-end (real GPU
+ `wasb` env +
51      Cloud Run L4) needs an **owner-driven CUJ** to confirm the telemetry snapshots /
progress sidecar /
52      stdout heartbeat actually land. The offline tests lock the emission config, the stdout
parse+merge
```

217. user (2026-07-09T08:01:55.224Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

218. user (2026-07-09T08:02:05.677Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-

indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

219. assistant (2026-07-09T08:02:21.690Z)

The second pass is handled. Here's the state of PR [#550](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550):

Second-pass review outcome

The owner confirmed **all five prior items resolved**, with one new **narrow correctness finding** on the timeout watchdog I'd just added ? a genuine race:

The race: a clean child completion (`rc=0`, CSV written) that finished in the window *before* `watchdog.cancel()` could still catch a late Timer fire. The timer set `timed_out`, and my unconditional `if timed_out.is_set(): raise` then reported that **success as a timeout**. The reviewer correctly noted the `kill()` was the harmless part (no-op once the child has a returncode) ? the *flag* was the bug.

Fix (`f7f0c82`)

Two guards so a clean completion deterministically wins:
1. **Watchdog only flags a live child** ? `if proc.poll() is None: timed_out.set(); proc.kill()`, so a late fire after a clean exit never sets the flag.
2. **Success wins the post-check** ? `if timed_out.is_set() and not (res is not None and res.returncode == 0): raise`, so an `rc=0` run is returned as success regardless.

Plus the post-mortem suggestion: the merged output `tail` now rides into `TimeoutExpired(stderr=?)` and gets logged (`"?timed out after Ns. last output: ?"`) instead of being built-then-discarded.

Tests: new `test_run_predict_streaming_clean_exit_beats_late_timer` (forces `timed_out` set + `rc=0` ? asserts success survives); the wedged-child timeout test now also asserts the tail is surfaced. A truly wedged child still returns `None`.

Verification

Full suite **1968 passed / 1 skipped**, coverage **84.47%** (floor 70), ruff 0.15.16 + mypy 2.1.0 clean. Pushed and [replied on the PR](https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#issuecomment-4922925046).

Both review passes were **COMMENT-only** (not blocking `CHANGES_REQUESTED`), so the branch is in good shape ? the remaining gate is the owner's merge decision plus the live-L4 heartbeat CUJ that can only be confirmed on real hardware.

220. user (2026-07-09T08:02:23.878Z)

pls merge the PR after responding to the reviewer and ensuring all tests and CI pass

221. assistant (2026-07-09T08:02:42.722Z)

Merging is a hard-to-reverse, outward-facing action, so let me verify thoroughly first: check for any new reviewer comments, the formal review decision, and ? critically ? the CI status and mergeability.

222. user (2026-07-09T08:02:55.322Z)

```
======== PR STATE ========
#550 state=OPEN draft=false
mergeable=MERGEABLE mergeState=UNSTABLE reviewDecision=
head=fix/worker-l4-infer-observability base=master commits=3

======== CI CHECKS ========
Built wheel smoke (install + launch)    pass    1m17s   https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/29003359679/job/86068967570
Golden fixtures cross-OS (macos-latest) pass    22s     https://github.com/Khelsutra/badminton-
```

```
highlight-indexer/actions/runs/29003359679/job/86068967725
Leak scan (PII / attribution guard)     pass    7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967582
Markdown link check (live docs) pass    8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967535
Mypy (lenient baseline) pass    31s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967609
Syntax sweep + import smoke (no GPU stack)      pass    34s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967564
Full test suite (offline, mocked)       pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967543
Dependency license scan (no copyleft in runtime deps)   pass    36s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967639
Golden fixtures cross-OS (ubuntu-latest)        pass    25s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967572
Golden fixtures cross-OS (windows-latest)       pass    56s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967610
Ruff lint       pass    20s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967538

======== NEW REVIEWS (since last) ========
--- avidullu [COMMENTED] @ 2026-07-09T07:23:07Z
--- avidullu [COMMENTED] @ 2026-07-09T07:47:45Z

======== LATEST CONVERSATION COMMENTS (authors + time) ========
--- avidullu @ 2026-07-09T07:41:13Z
--- avidullu @ 2026-07-09T08:01:27Z
======== LATEST INLINE COMMENTS (after f7f0c82) ========
--- avidullu on backend/pipeline/detectors/native_wasb_runner.py:387 @ 2026-07-09T07:22:34Z
--- avidullu on backend/worker/__main__.py:359 @ 2026-07-09T07:22:34Z
--- avidullu on backend/worker/__main__.py:384 @ 2026-07-09T07:22:34Z
--- avidullu on backend/worker/__main__.py:332 @ 2026-07-09T07:22:34Z
--- avidullu on backend/worker/__main__.py:392 @ 2026-07-09T07:22:34Z
--- avidullu on backend/pipeline/detectors/native_wasb_runner.py:409 @ 2026-07-09T07:47:13Z
```

223. assistant (2026-07-09T08:03:45.373Z)

```
PR is `MERGEABLE` but `mergeState=UNSTABLE` because the **"Full test suite (offline, mocked)" CI
check is still PENDING** ? all 10 other checks pass. I've already responded to both review
passes (no new reviewer comments since). I must not merge while CI is pending, so let me wait
for that check to finish.
```

224. user (2026-07-09T08:05:09.803Z)

```
Full test suite (offline, mocked)       pass    3m33s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967543
Golden fixtures cross-OS (macos-latest) pass    22s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967725
Golden fixtures cross-OS (ubuntu-latest)        pass    25s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967572
Golden fixtures cross-OS (windows-latest)       pass    56s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967610
Leak scan (PII / attribution guard)     pass    7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967582
Markdown link check (live docs) pass    8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967535
Mypy (lenient baseline) pass    31s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967609
Ruff lint       pass    20s     https://github.com/Khelsutra/badminton-highlight-
```

```
indexer/actions/runs/29003359679/job/86068967538
Syntax sweep + import smoke (no GPU stack)      pass    34s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967564
Built wheel smoke (install + launch)    pass    1m17s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967570
Dependency license scan (no copyleft in runtime deps)   pass    36s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967639
Full test suite (offline, mocked)       pass    3m33s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967543
Golden fixtures cross-OS (macos-latest) pass    22s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967725
Golden fixtures cross-OS (ubuntu-latest)        pass    25s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967572
Golden fixtures cross-OS (windows-latest)       pass    56s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967610
Leak scan (PII / attribution guard)     pass    7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967582
Markdown link check (live docs) pass    8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/29003359679/job/86068967535
Mypy (lenient baseline) pass    31s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967609
Ruff lint       pass    20s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967538
Syntax sweep + import smoke (no GPU stack)      pass    34s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/29003359679/job/86068967564
=== exit: 0 ===
```

225. assistant (2026-07-09T08:06:10.817Z)

All 11 CI checks pass (Full test suite: 3m33s). Let me re-confirm the merge state flipped to CLEAN now that CI is green, and double-check the base hasn't moved.

226. user (2026-07-09T08:06:18.556Z)

```
mergeable=MERGEABLE mergeState=CLEAN reviewDecision= state=OPEN
=== base master head ===
6cfcb37
=== does the repo auto-delete merged branches? checking merge_commit settings via API ===
squash_merge_allowed=true delete_branch_on_merge=true
```

227. assistant (2026-07-09T08:06:36.707Z)

Merge state is **CLEAN**, `mergeable`, no blocking review decision, squash allowed, and `delete_branch_on_merge=true` (GitHub will auto-delete the remote branch ? so I'll skip `--delete-branch` to avoid disturbing this shared local checkout). Let me post a brief closing comment, then squash-merge (matching the repo's `(#NNN)` squash convention).

228. user (2026-07-09T08:06:43.225Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/550#issuecomment-4922966425

229. assistant (2026-07-09T08:07:05.777Z)

Closing comment posted. Now squash-merging with a full commit message (co-author trailer included, matching the repo's `(#NNN)` convention). Not passing `--delete-branch` ? the repo auto-deletes the remote branch, and I want to leave this shared local checkout untouched.

230. user (2026-07-09T08:07:17.718Z)

(Bash completed with no output)

231. assistant (2026-07-09T08:07:20.893Z)

Let me verify the merge landed on master.

232. user (2026-07-09T08:07:28.761Z)

```
=== PR state ===
Unknown JSON field: "merged"
Available fields:
  additions
  assignees
  author
  autoMergeRequest
  baseRefName
  baseRefOid
  body
  changedFiles
  closed
  closedAt
  closingIssuesReferences
  comments
  commits
  createdAt
  deletions
  files
  fullDatabaseId
  headRefName
  headRefOid
  headRepository
  headRepositoryOwner
  id
  isCrossRepository
  isDraft
  labels
  latestReviews
  maintainerCanModify
  mergeCommit
  mergeStateStatus
  mergeable
  mergedAt
  mergedBy
  milestone
  number
  potentialMergeCommit
  projectCards
  projectItems
  reactionGroups
  reviewDecision
  reviewRequests
  reviews
  state
  statusCheckRollup
  title
  updatedAt
  url
=== fetch + master HEAD ===
From https://github.com/Khelsutra/badminton-highlight-indexer
   6cfcb37..01d1081  master     -> origin/master
01d1081 fix(worker): make the cloud-serving L4 infer path observable (#550)
6cfcb37 docs(runlog): CP 2vCPU/8Gi downgrade + 7.84 GiB live CUJ (2026-07-09) (#549)
00e8f92 feat(gates): local release perf gate + consolidate nightly into scripts/gates/ (#547)
=== remote branch still there? ===
(empty above = remote branch auto-deleted)
```

233. assistant (2026-07-09T08:07:38.810Z)

Merge succeeded ? `origin/master` is now `01d1081` = "fix(worker): make the cloud-serving L4 infer path observable (#550)", and the remote branch auto-deleted. Let me confirm the PR shows MERGED and the squash landed exactly my files (no sibling work absorbed).

234. user (2026-07-09T08:07:42.593Z)

```
=== PR state ===
state=MERGED mergedAt=2026-07-09T08:07:18Z by=avidullu sha=01d108112
=== files in the squash commit (must be ONLY my 7) ===
01d1081 fix(worker): make the cloud-serving L4 infer path observable (#550)
 backend/pipeline/detectors/native_wasb_runner.py | 104 ++++++++---
 backend/pipeline/detectors/wasb_infer.py         |  22 +++
 backend/pipeline/validators/base.py              |   5 +
 backend/worker/__main__.py                       | 164 +++++++++++++---
 tests/test_native_wasb_runner.py                 | 223 ++++++++++++++++++++-
 tests/test_validators_base.py                    |  10 +
 tests/test_worker.py                             | 186 ++++++++++++++++++
 7 files changed, 663 insertions(+), 51 deletions(-)
```

235. assistant (2026-07-09T08:07:52.843Z)

Merge confirmed and clean ? PR **MERGED** (squash `01d1081`), exactly my 7 files, no sibling work absorbed. Let me update the memory to reflect the merged state.

236. user (2026-07-09T08:07:53.956Z)

```
1       ---
2       name: pr550-worker-l4-observability
3       description: "PR #550 (open) makes the cloud-serving L4 worker infer path observable;
live-L4 heartbeat still needs an owner CUJ"
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: e663947f-2127-4383-9905-650340ff9863
8       ---
9
10      # PR #550 ? cloud-serving L4 infer observability (OPEN, 2026-07-09)
11
12      Branch `fix/worker-l4-infer-observability` cut from `origin/master @ 00e8f92`; commit
`b3dff78`.
```

237. user (2026-07-09T08:07:58.499Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

238. user (2026-07-09T08:08:06.310Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr550-worker-l4-observability.md has been updated successfully. (file state is current in your context ? no need to Read it back)

239. user (2026-07-09T08:08:17.310Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

240. assistant (2026-07-09T08:08:33.167Z)

Merged. Here's the final state of PR [#550](https://github.com/Khelsutra/badminton-highlight-

indexer/pull/550):

? Merged to master

- **Squash commit `01d1081`** ? "fix(worker): make the cloud-serving L4 infer path observable (#550)".
- Merged by avidullu at 2026-07-09T08:07Z. Remote branch auto-deleted (repo setting); I left the shared local checkout untouched.
- The squash touched **exactly the 7 files** of this PR ? no sibling work absorbed.

What I verified before merging

- **Reviewer feedback**: both COMMENT-only review passes were fully resolved (5 items + the watchdog-race follow-up); no new reviewer comments outstanding. Posted a closing comment summarizing the resolution.
- **CI**: waited for the pending check, then confirmed **all 11 checks green** ? Full test suite (offline/mocked, 3m33s), cross-OS golden fixtures (ubuntu/macos/windows), ruff, mypy, wheel smoke, syntax/import sweep, leak + license scans.
- **Local gate**: 1968 passed / 1 skipped, coverage 84.47% (floor 70), ruff 0.15.16 + mypy 2.1.0 clean.
- **Merge state**: `MERGEABLE` / `CLEAN`, no blocking review decision.

?? One thing to flag ? merged ? deployed

The code is on master but **not live yet**. The serving worker/control-plane still run the pre-#550 image (digest `1838c07f` = master `00e8f92`). To make the new observability actually take effect ? and to run the **live-L4 heartbeat CUJ** that's the one thing CI can't cover (real GPU + `wasb` env + Cloud Run L4) ? the worker image needs a rebuild + redeploy. I've recorded this in memory as the open follow-up. Want me to prep that deploy, or leave it for the next release cycle?